TW128: A Tree-Structured Duplex Authenticated Encryption Mode

Coda Hale

Hale Institute for Bicycle Appreciation

Abstract. AEAD tags are often treated as compact commitments to the full encryption context, but standard deployed modes such as AES-GCM and ChaCha20-Poly1305 do not provide this property. We present TW128, a nonce-based authenticated encryption mode with associated data designed around the stronger tag-centric requirement: given a candidate opening, the authentication tag should itself behave like a digest of the full (K, U, A, M) context, including the key.

TW128 adapts the KangarooTwelve tree-hashing pattern to keyed duplex encryption over Keccak-p[1600, 12]. A serial root duplex binds the key, nonce, associated data, and first message chunk, then aggregates tags from independently keyed leaf duplexes for the remaining chunks. The leaf decomposition is canonical, while the number of leaves evaluated at once is an implementation choice; scalar, SIMD, and multicore implementations therefore produce identical ciphertexts and tags without exposing the degree of parallelism as a mode parameter.

The final root tag is both the AEAD authentication tag and the object of the main security analysis. In the public permutation model, via an intermediate random oracle and the flat sponge lift, we prove that the wrapper returning this tag is a secure hash of the full input: collision-, second-preimage-, and preimage-resistant in the standard hash-function sense. The same root tag analysis yields AEAD-facing committing security, including full-input CMTDs-4. The root-centered structure also gives integrity under release of unverified plaintext in the sense of [ABL+14]. As baseline AEAD properties, we prove concrete multi-user indistinguishability and derive all-in-one authenticated encryption as the single-user corollary. At the implemented parameters, all stated notions meet a 128-bit security target. Vectorized `amd64` and `arm64` implementations recover the scalable performance profile of the tree design: in bulk they reach 48 Gbit/s, with the AVX-512 and `arm64` paths delivering 51–76% of hardware-accelerated AES-128-GCM throughput.

Keywords: authenticated encryption · committing security · sponge constructions · duplex constructions · Keccak

1 Introduction

Authenticated encryption with associated data (AEAD) is often used as though its tag were a compact digest of the whole encryption context: a short value binding the key, nonce, associated data, and message. Ordinary AEAD security does not give that property. AES-GCM and ChaCha20-Poly1305, for example, are not key-committing: an adversary can craft a single ciphertext that decrypts to valid plaintexts under two or more keys [DGRW18, ADG+22]. The problem is structural. These modes authenticate with algebraic MACs whose outputs can be made to validate under another context, while a digest is expected to resist both collisions and preimages.

This mismatch has practical consequences. The "invisible salamander" attack breaks Facebook's attachment franking scheme [DGRW18]; related attacks construct ciphertexts that open to different well-formed files under different keys [ADG+22]; partitioning oracles recover passwords from protocols built on non-committing AEAD [LGR21]; and context discovery attacks find valid decryption contexts for ciphertexts under standard modes such as CCM, EAX, and SIV, defeating the preimage side of the digest intuition [MLGR23]. These attacks differ in surface form, but they expose the same design gap: applications compare, store, or reason about the tag as a compact commitment, while the mode proves only ordinary authenticity.

Recent work formalizes parts of this expectation. Compactly committing AEAD was introduced for message franking [GLR17], encryptment builds commitment into a dedicated primitive [DGRW18], and the committing hierarchy of [BH22] asks whether a ciphertext and tag can have another valid opening. Those are AEAD-facing notions over ciphertexts. The object applications often treat as the commitment, however, is the tag itself. This paper therefore takes the tag as the primary security object: we analyze the root tag with standard hash-function notions, then recover the AEAD committing games as consequences of the same root tag analysis. These are opening-based statements: the candidate opening includes the key, nonce, associated data, and message, so the tag is not claimed to be a public digest of the visible AEAD inputs alone.

TW128 (*TreeWrap*, with 128 denoting the 128-bit security target) is designed around that tag-centric requirement. It adapts the KangarooTwelve [BDP+16] tree-hashing pattern to keyed duplex encryption over Keccak-p[1600, 12]. A serial root duplex binds the key, nonce, associated data, and first message chunk. Independently keyed leaf duplexes encrypt the remaining chunks and return leaf tags, which the root aggregates before emitting the final root tag. The result is an AEAD mode whose authentication tag is also the natural digest point for the full (K, U, A, M) context.

The tree shape also separates transcript compatibility from implementation parallelism. The root/leaf decomposition and leaf order are fixed by the mode, but the number of leaves evaluated at once is not. Scalar, SIMD, and multicore implementations therefore produce identical ciphertexts and tags without negotiating a parallelism degree or exposing it on the wire. Single-chunk messages use only the root duplex, matching KangarooTwelve's short-input behavior and avoiding tree overhead (Section 8.4).

The main security result is that the wrapper $\mathsf{H}_{\mathsf{TW128}}$ returning the final root tag is a secure hash of the full input in the sense of [RS04]: collision-, second-preimage-, and preimage-resistant. The same proof spine yields AEAD-facing committing security, including full-input $\mathsf{CMTDs\text{-}4}$. The root-centered structure also gives $\mathsf{INT\text{-}RUP}$ integrity [ABL+14]: even when an adversary can obtain unverified stream plaintext, fresh accepting ciphertexts remain bounded by the same tag-guessing and collision terms. We also prove the baseline AEAD security bounds needed of a nonce-based mode: concrete multi-user indistinguishability [BT16] in the public permutation model, with all-in-one authenticated encryption [RS06] as the single-user corollary.

At the implemented parameters ($k = c = \nu = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$), every stated advantage is at most 2^{-128} for adversaries running up to $N_{\mathsf{tot}} \leq 2^{63}$ Keccak-p[1600, 12] calls, on the order of 2^{71} bytes of processed data. The cap is a data-scale choice rather than a security ceiling: the dominant terms are birthday-type on the 256-bit capacity and become vacuous only near 2^{128} calls (Section 7.1). Optimized `amd64` and `arm64` implementations preserve the canonical transcript while evaluating leaf duplexes in batches; in bulk, TW128 reaches 48 Gbit/s with AVX-512 and 39 Gbit/s on an Apple M4 Pro, 51–76% of hardware-accelerated AES-128-GCM throughput (Section 8).

1.1 Our Contributions

- **A tag-first tree AEAD mode.** We specify TW128, a nonce-based AEAD with associated data built as a two-level tree of strict [BDPVA11] duplexes over Keccak-p[1600, 12] (Section 3). A root duplex emits the authentication tag and acts as the digest point for the full encryption context, while indexed leaf duplexes provide scalable message processing. The mode fixes the root/leaf transcript but leaves the number of leaves evaluated simultaneously to the implementation, so scalar and vectorized implementations interoperate without changing ciphertexts or tags.

- **Root tag hash security.** We prove that the wrapper $\mathsf{H}_{\mathsf{TW128}}$ returning the root tag is a secure hash of the full (K, U, A, M) input in the sense of [RS04]: collision-, second-preimage-, and preimage-resistant (Theorems 4 to 6 and Corollary 2). Thus the authentication tag has the digest-like binding and preimage properties often expected of a compact AEAD tag.

- **Committing and RUP consequences.** We instantiate the same root tag analysis in the existing AEAD vocabulary. The all-bodies-equal structure of CMTDs-4 gives the full-input s-way committing bound without the leaf-collision slack of the general hash collision theorem [BH22] (Corollary 4), and the remaining CMT/CMTD notions for $\ell \in \{1, 3, 4\}$ follow from the committing hierarchy and direct source-game arguments (Corollary 5). We also prove INT-RUP integrity [ABL+14], with plaintext-computing oracle work charged through the same transcript resources (Theorem 11).

- **Baseline AEAD security.** We give a concrete multi-user indistinguishability bound [BT16] for D users in the public permutation model (Theorem 9). The $D = 1$ case yields a concrete all-in-one AE bound [RS06] (Corollary 6). The proof proceeds through a dedicated TW128 random oracle model and is transferred to the public permutation setting by the [BDPVA08] flat sponge lift.

- **Optimized implementations.** We describe and evaluate vectorized `amd64` (AVX-512 and AVX2) and `arm64` (NEON with the SHA-3 extension) implementations whose batched leaf cores realize the mode's scalable parallelism in practice (Section 8).

1.2 Related Work

Committing AEAD and tag commitment. The study of committing authenticated encryption was initiated by [GLR17], who introduced compactly committing AEAD for message franking, and by [DGRW18], who built it from a dedicated primitive (encryptment). [BH22] organized collision-style committing notions into the CMT-1/3/4 and CMTD hierarchy and gave the UtC, RtC, and HtE transforms; [CR22] gave the CTX transform, which commits a nonce-based AE scheme to its full context by adding a hash. These transforms are efficient and broadly applicable, but commitment remains an external layer or an AEAD game property. TW128 instead makes the authentication tag itself the hash-like commitment: the root tag is the compact object, and the root tag analysis gives the [BH22] full-input committing consequence. Closest in spirit is [DHM+24], which also builds nonce-based AE natively on SHA-3 functions and obtains CMT-4 from collision resistance; it targets session-based communication through a sequential duplex chain, whereas TW128 is a standalone tree AEAD with implementation-selected leaf parallelism (Section 9.3).

Sponge, duplex, and tree AEADs. Permutation-based authenticated encryption built on the sponge and duplex constructions of [BDPVA08, BDPVA11] is well established. Keyak [BDP+15] uses the Motorist mode over round-reduced Keccak-p and admits parallel duplex instances; Ascon [DEMS21] and Xoodyak [DHP+20] are serial duplex AEADs over smaller permutations. TW128 shares the single-permutation, constant-state duplex foundation but makes the root of a tree the authentication tag. Keyak fixes the degree of parallelism as a mode parameter, while TW128 fixes only the indexed leaf decomposition and lets the implementation choose how many leaves to evaluate at once (Section 9.2). Farfalle [BDH+17] reaches implementation-chosen, wire-invisible parallelism through a parallel PRF, instantiated over KECCAK-p as Kravatte, with AE modes layered on top. TW128 retains the strict [BDPVA11] duplex object and draws its parallelism from the tree topology; the committing behavior of the Farfalle modes has not been analyzed.

Release of unverified plaintext. Andreeva et al. [ABL+14] formalized authenticated encryption in the release-of-unverified-plaintext setting by separating plaintext computation from verification. They define INT-RUP for integrity and plaintext awareness notions for privacy, and show that ordinary ciphertext integrity does not imply INT-RUP. TW128 targets the integrity side of this model: the released stream plaintext is not claimed to be private, but it does not reveal the hidden tag values needed to make a fresh ciphertext verify.

Tree hashing. TW128's two-level shape is inspired by KangarooTwelve [BDP+16], a tree hash over the same permutation. TW128 adopts its choice of permutation and final-node/leaf split and adapts them to authenticated encryption: each leaf chunk is encrypted and tagged independently, leaf tags replace unkeyed chaining values, and the root is keyed by (K, U). The construction also inherits KangarooTwelve's short-message behavior: inputs fitting in one chunk use only the root duplex. Because TW128 has a fixed two-level topology with independently keyed, indexed leaves and phase-separated duplex calls, it does not need KangarooTwelve's Sakura tree encoding [BDPVA14] to make its transcripts unambiguous (Section 3).

1.3 Paper Organization

Section 2 recalls the notation, sponge and duplex constructions, nonce-based AEAD and RUP integrity notions, committing security [BH22], and the hash notions of [RS04]. Section 3 specifies the TW128 construction and parameters. Section 4 fixes the scope of the security claims, the public permutation games, intermediate random oracle model, resource accounting, and loss terms. Section 5 proves the root tag hash and committing security results that drive the tag-centric claims, and Section 6 proves multi-user indistinguishability, all-in-one AE security, and INT-RUP integrity. Section 7 summarizes the concrete bounds. Section 8 reports implementation performance, and Section 9 compares TW128 to prior AEAD and committing constructions and discusses limitations outside the formal scope. The appendices collect structural lemmas, the flat sponge lift, vectorized kernel details, and test vectors.

2 Preliminaries

2.1 Notation

Strings. Write $\{0,1\}$ for the binary alphabet. For an integer $n \geq 0$, $\{0,1\}^n$ denotes the set of n-bit strings, $\{0,1\}^* = \bigcup_{n\geq 0}\{0,1\}^n$ the set of all finite-length bit strings, and ε the empty string. For $x \in \{0,1\}^*$, $|x|$ is the bit length and $x \parallel y$ is the concatenation; the bitwise XOR of two equal-length strings $x, y \in \{0,1\}^n$ is $x \oplus y$. A *byte string* is an element of $(\{0,1\}^8)^*$; its byte length is denoted $\|x\| = |x|/8$. For an integer $0 \leq n < 2^{64}$, $\langle n \rangle_8 \in \{0,1\}^{64}$ is its eight-byte little-endian encoding. For a finite or infinite bit string x and $0 \leq \tau \leq |x|$ when x is finite, $\lfloor x \rfloor_\tau \in \{0,1\}^\tau$ denotes the leading τ bits of x.

Sampling and probability. We write $x \leftarrow\$ S$ for sampling x uniformly at random from a finite set S, with implicit coins independent across samples. For an event E in a probability experiment, $\Pr[E]$ denotes its probability.

Sets of maps. For a set S, $\mathsf{Perm}(S)$ denotes the set of permutations on S; for a positive integer b, $\mathsf{Perm}(\{0,1\}^b)$ is the set of b-bit permutations.

Games and advantages. We write $\mathcal{A}^{O_1,\dots,O_k}$ for a (possibly randomized) adversary with oracle access to $O_1,\dots,O_k$, and $\Pr[\mathsf{G}^{\mathcal{A}} \Rightarrow 1]$ for the probability that a game G—an experiment that runs $\mathcal{A}$ against specified oracles and returns a Boolean outcome—returns 1. For a distinguishing experiment between two games $\mathsf{G}_0, \mathsf{G}_1$, the advantage of $\mathcal{A}$ against a scheme Π in notion X is the absolute difference

$$\mathbf{Adv}^{\mathsf{X}}_{\Pi}(\mathcal{A}) = \left|\Pr[\mathsf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathsf{G}_1^{\mathcal{A}} \Rightarrow 1]\right|.$$

For a win-event experiment without a hidden challenge bit it is instead $\Pr[\mathsf{G}^{\mathcal{A}} \text{ wins}]$, the probability the game returns 1; a bit-guessing experiment with a hidden challenge bit uses the rescaled form $\left|2\Pr[\mathsf{G}^{\mathcal{A}} \text{ wins}] - 1\right|$, as in the multi-user notion of Section 2.7.

Concrete security. All bounds in this paper are concrete: each advantage is an explicit function of the resource caps introduced in Section 4.3. We make no use of asymptotic or negligibility conventions.

2.2 Sponges and Duplexes over Keccak-*p*

The Keccak-*p* permutation. The KECCAK-p family of permutations, defined in FIPS 202 [Nat15] as the generalization of the Keccak design's Keccak-f permutations to a free round count, is parameterized by a state width b and a round count n_r; we write $\text{KECCAK} - p[b, n_r] \in \mathsf{Perm}(\{0,1\}^b)$ for the corresponding permutation. $\mathsf{TW128}$ uses KECCAK-p[1600, 12], the same primitive as KangarooTwelve [BDP+16], fixed throughout. In the public permutation security model of Section 4.1 and the flat sponge lift of Section B, this permutation is modeled as a uniformly random element of $\mathsf{Perm}(\{0,1\}^{1600})$.

The sponge construction. Following [BDPVA08], fix a permutation $\pi \in \mathsf{Perm}(\{0,1\}^b)$, a *sponge-compliant* padding rule pad, and a rate $1 \le r < b$ with capacity $c = b - r$. A padding rule appends $\mathsf{pad}[r](|M|)$, a bit string determined by the input length and the rate, to an input M, extending it to a sequence of r-bit blocks; it is sponge-compliant [BDPVA11, Definition 1] if $M \,\|\, \mathsf{pad}[r](|M|)$ is never empty and, for all $M \neq M'$ and all $n \ge 0$, $M \,\|\, \mathsf{pad}[r](|M|) \neq M' \,\|\, \mathsf{pad}[r](|M'|) \,\|\, 0^{nr}$. The sponge function $\mathsf{Sponge}[\pi, \mathsf{pad}, r] : \{0,1\}^* \times \mathbb{N} \to \{0,1\}^*$ maps an input M and an output length ℓ to an ℓ-bit string by absorbing $M \,\|\, \mathsf{pad}[r](|M|)$ into the all-zero b-bit state in r-bit blocks (each separated by an application of π) and squeezing ℓ bits in r-bit blocks from the resulting state. [BDPVA08, Theorem 2] shows that $\mathsf{Sponge}[\pi, \mathsf{pad10}^*, r]$, where the simple rule $\mathsf{pad10}^*$ appends a single 1 bit followed by the minimum number of 0 bits reaching a multiple of r, is indifferentiable from a random oracle when π is uniformly random. We bound its distinguishing advantage by $\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)/2^{c+1}$, derived in Lemma 20 from the product upper bounds of [BDPVA08, Lemmas 4 and 5].

The duplex construction. The *duplex* construction of [BDPVA11] extends the sponge to a stateful interface. A duplex object $\mathsf{Duplex}[\pi, \mathsf{pad}, r]$ is initialized to the all-zero state. Each call $\mathsf{duplexing}(\sigma, \ell)$, with $\ell \le r$, absorbs an input string σ of at most $\rho_{\max}(\mathsf{pad}, r)$ bits via π and returns the first ℓ bits of the resulting state. Here $\rho_{\max}(\mathsf{pad}, r) = \min\{x :$

$x+|\mathsf{pad}[r](x)| > r\}-1$ is the largest input length for which the padded input fits in a single r-bit block; for the $\mathsf{pad10^*1}$ rule, $\rho_{\max}(\mathsf{pad10^*1}, r) = r-2$ [BDPVA11]. [BDPVA11, Lemma 3] establishes the duplex-to-sponge equality: for any call sequence $((\sigma_0, \ell_0), \ldots, (\sigma_i, \ell_i))$, the i-th output equals

$$\mathsf{Sponge}[\pi, \mathsf{pad}, r]\big(\sigma_0 \parallel \mathsf{pad}_0 \parallel \cdots \parallel \sigma_i,\ \ell_i\big),$$

where $\mathsf{pad}_j = \mathsf{pad}[r](|\sigma_j|)$ is the padding the duplex appends in its j-th call; the flattened input carries these interior paddings explicitly, and the sponge appends only the final pad_i. The flattening map is injective by [BDPVA11, Lemma 4]: the duplex input-string sequence is recoverable from the flattened sponge input.

TW128 instantiation. TW128 fixes the padding rule $\mathsf{pad10^*1}$ and the rate $r = 1344$, so the capacity is $c = 256$ bits. The largest input length per duplex call is therefore $\rho_{\max}(\mathsf{pad10^*1}, 1344) = 1342$ bits.

Bridge to the H-input domain. The indifferentiability theorem of [BDPVA08] is stated for the simpler $\mathsf{pad10^*}$ padding over its unpadded H-input domain, while TW128 uses $\mathsf{pad10^*1}$ internally. [BDPVA11, Theorem 3] supplies an injective preprocessing bridge $I[r, r_{\max}]$ between the two domains. For TW128, $r = r_{\max} = 1344$, so the bridge reduces to appending $1 \parallel 0^q$ with q determined by the input length modulo $r_{\max}$; on the bridge image, q is recovered as the trailing-zero count. Section 4.2 instantiates this bridge on typed TW128 duplex transcript prefixes as the map $\mathsf{flat}(\cdot)$ used by the random oracle proofs throughout the paper.

2.3 AEAD Syntax and Verification Interfaces

Syntax. A *nonce-based authenticated encryption scheme with associated data* (AEAD scheme) is a pair $\mathsf{SE} = (\mathsf{Enc}, \mathsf{Dec})$ of deterministic algorithms with associated key space $\mathcal{K} \subseteq (\{0,1\}^8)^*$, nonce space $\mathcal{U} \subseteq (\{0,1\}^8)^*$, associated data space $\mathcal{A} \subseteq (\{0,1\}^8)^*$, message space $\mathcal{M} \subseteq (\{0,1\}^8)^*$, and ciphertext space $\mathcal{C} \subseteq (\{0,1\}^8)^*$. The encryption algorithm $\mathsf{Enc} : \mathcal{K} \times \mathcal{U} \times \mathcal{A} \times \mathcal{M} \to \mathcal{C}$ produces a ciphertext $C = \mathsf{Enc}_K(U, A, M)$. The decryption algorithm $\mathsf{Dec} : \mathcal{K} \times \mathcal{U} \times \mathcal{A} \times \mathcal{C} \to \mathcal{M} \cup \{\perp\}$ returns either a plaintext or the rejection symbol $\perp$. The *tag length in bytes* is the constant $\|\mathsf{Enc}_K(U, A, M)\| - \|M\|$, fixed by the scheme. We write nonce values as U to keep N available for the construction-cost resource variable used in the concrete bounds. In this generic syntax, C denotes the full AEAD ciphertext, including any tag material. The TW128 algorithm section (Section 3) writes this same value as $C \parallel T$, with C the ciphertext body and T the root tag.

Correctness. For every $(K, U, A, M) \in \mathcal{K} \times \mathcal{U} \times \mathcal{A} \times \mathcal{M}$, $\mathsf{Dec}_K(U, A, \mathsf{Enc}_K(U, A, M)) = M$. An AEAD scheme is *tidy* if every accepting decryption is consistent with encryption: whenever $\mathsf{Dec}_K(U, A, C) = M \neq \perp$, one has $\mathsf{Enc}_K(U, A, M) = C$.

All-in-one AE. We use the all-in-one authenticated encryption notion of [RS06], instantiated for nonce-based AE following [BT16]. The experiment grants either the real pair $(\mathsf{Enc}_K, \mathsf{Ver}_K)$ or the ideal pair $(\mathsf{Rand}, \mathsf{Replay})$, where Rand returns a fresh uniform byte string of the ciphertext length on each encryption query and Replay accepts exactly the tuples Rand produced. The advantage $\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{SE}}(\mathcal{A})$ is the distinguishing advantage between these two worlds over nonce-respecting adversaries. This all-in-one notion is instantiated for TW128 in Section 6.1.

2.4 Root Tag Hash Security

Definition 1 (Root tag wrapper)**.** For every $\mathsf{TW128}$ encryption input $X = (K, U, A, M)$ in the scheme domain, define the root tag wrapper by

$$\mathsf{H}_{\mathsf{TW128}}(K, U, A, M) = T \quad \text{where} \quad \mathsf{Enc}_K(U, A, M) = C \parallel T.$$

We also write $\mathsf{H}_{\mathsf{TW128}}(X)$ for $\mathsf{H}_{\mathsf{TW128}}(K, U, A, M)$. Equivalently, $\mathsf{H}_{\mathsf{TW128}}$ runs $\mathsf{TW128}$ encryption and discards the ciphertext body.

The root tag as a hash. Many uses of commitment store, transmit, or compare a short tag rather than the full ciphertext, and ask that the short string alone be binding—the compactly committing AEAD goal introduced by [GLR17] for message franking. $\mathsf{TW128}$ ciphertexts are written $C \parallel T$ with T the τ_{root}-bit root tag, and we capture this goal directly by treating Definition 1 as a hash function. Keyed by the permutation, this is a hash family, and we ask that it satisfy the hash-function security notions of [RS04], over the $\mathsf{TW128}$ input domain, together with the local s-way multicollision extension used for the committing theorems. For fixed byte lengths a and μ, write

$$\mathcal{X}_{a,\mu} = \{0,1\}^k \times \{0,1\}^\nu \times (\{0,1\}^8)^a \times (\{0,1\}^8)^\mu$$

for the fixed-length slice of valid inputs, with bit length $m_{a,\mu} = k + \nu + 8a + 8\mu$.

Collision resistance (Coll). No adversary finds distinct inputs $X_1, \ldots, X_s$ (any $s \geq 2$) with $\mathsf{H}_{\mathsf{TW128}}(X_1) = \cdots = \mathsf{H}_{\mathsf{TW128}}(X_s)$. The case $s = 2$ is the RS04 Coll notion; larger s is the corresponding multicollision extension.

Preimage resistance (Pre). For fixed a, μ, given $\mathsf{H}_{\mathsf{TW128}}(X^*)$ for a uniform $X^* \leftarrow\$ \mathcal{X}_{a,\mu}$, no adversary finds any valid input X with $\mathsf{H}_{\mathsf{TW128}}(X) = \mathsf{H}_{\mathsf{TW128}}(X^*)$.

Second-preimage resistance (Sec/eSec). The standard and everywhere forms of second-preimage resistance of [RS04], on each fixed input slice $\mathcal{X}_{a,\mu}$, are reduced explicitly to two-way collision resistance in Corollary 2.

Everywhere preimage resistance (ePre). For a target tag T^* fixed in advance, no adversary finds X with $\mathsf{H}_{\mathsf{TW128}}(X) = T^*$.

The advantages $\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{TW128}}$, $\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{TW128}}$, and $\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{TW128}}$ are the respective success probabilities, instantiated for $\mathsf{TW128}$ in Section 5.1. The Pre bound is proved directly in Theorem 5, while the second-preimage bounds are inherited from Coll in Corollary 2. Coll is the binding property of the tag taken as a compact commitment to (K, U, A, M), Pre is the random-opening preimage property on fixed input slices, and ePre is the fixed-target preimage property for query-independent tags. The [BH22] committing notions are the collision-facing AEAD projection of the same tag analysis: a winning opening gives distinct inputs sharing a tag, while the dedicated CMTDs-4 proof uses the all-bodies-equal game structure to avoid the leaf-collision slack of the general hash collision theorem (Section 5).

2.5 AEAD Committing Notions

[BH22] formalize a family of committing security notions for nonce-based AEAD schemes, parameterized by how much of the encryption input is committed and by whether commitment is established through decryption (the D-notions) or through encryption agreement (the E-notions). We recall their definitions and the s-way generalization because the root tag hash bounds of Section 5 are later instantiated in this AEAD vocabulary.

What-is-committed functions. For $\ell \in \{1,3,4\}$, the *what-is-committed* function WiC_ℓ projects an encryption input $(K,U,A,M) \in \mathcal{K} \times \mathcal{U} \times \mathcal{A} \times \mathcal{M}$ to the portion it should commit:

$$\begin{aligned}\mathsf{WiC}_1(K,U,A,M) &= K,\\ \mathsf{WiC}_3(K,U,A,M) &= (K,U,A),\\ \mathsf{WiC}_4(K,U,A,M) &= (K,U,A,M).\end{aligned}$$

CMTDs-ℓ and CMTs-ℓ games. For $s \geq 2$, [BH22, Fig. 5] defines the s-way committing games on tuples $(K_i,U_i,A_i,M_i)_{i=1}^{s}$ with pairwise distinct WiC_ℓ-images, all entries in the scheme's syntactic domain and none equal to $\perp$. The s-way CMTDs-ℓ (decryption) game has the adversary output a ciphertext C alongside the tuples and returns 1 if $\mathsf{Dec}_{K_i}(U_i,A_i,C) = M_i$ for every i; the printed return line of [BH22, Fig. 5] writes C_i, but the game outputs a single ciphertext C. The s-way CMTs-ℓ (encryption) game has the adversary output the tuples alone and returns 1 if all encryptions $\mathsf{Enc}_{K_i}(U_i,A_i,M_i)$ are equal. Advantages $\mathbf{Adv}_{\mathsf{SE}}^{\mathsf{CMTDs}\text{-}\ell}(\mathcal{A})$ and $\mathbf{Adv}_{\mathsf{SE}}^{\mathsf{CMTs}\text{-}\ell}(\mathcal{A})$ are the probabilities that the adversary wins the respective game; the parameter s is suppressed from the notation when clear from context. The two-tuple case $s = 2$ is the original CMTD-ℓ/CMT-ℓ pair of [BH22, Fig. 4]. These definitions are instantiated for $\mathsf{TW128}$ in Section 5.1, with the public permutation lift supplied by Section B.

Relations. [BH22, Figs. 4 and 5] record the implications among the notions. For the D-notions we use:

$$\mathsf{CMTDs}\text{-}4 \ \Rightarrow\ \mathsf{CMTDs}\text{-}\ell \text{ for } \ell \in \{1,3\}$$

(both immediate: pairwise distinct keys ($\ell = 1$) or pairwise distinct (K,U,A) triples ($\ell = 3$) are *a fortiori* pairwise distinct full tuples, so every CMTDs-ℓ winner is a CMTDs-4 winner). The E-notions are bounded directly in Corollary 5 by the same final-root candidate-collision argument, rather than through an encryption reduction.

2.6 Integrity under Release of Unverified Plaintext

For the RUP setting of [ABL+14], a conventional decryption algorithm is separated into a plaintext-computing algorithm and a verification algorithm. The integrity notion $\mathsf{INT\text{-}RUP}$ gives the adversary access to encryption, plaintext computation, and verification, and asks whether verification accepts a fresh ciphertext and tag not returned by encryption. As in the nonce-IV setting of [ABL+14], encryption queries are nonce-respecting, while plaintext computation and verification may be queried on arbitrary nonces. This notion protects only ciphertext integrity; privacy under release of unverified plaintext requires plaintext awareness in addition to ordinary encryption privacy.

2.7 Multi-User Indistinguishability

The multi-user indistinguishability notion of [BT16, Fig. 3] extends the same real-vs-ideal oracle pair to an adversary-selected set of users. The number of users u is adversary-determined: it is the number of New queries the adversary makes, i.e., the final value of the user counter v in the game below. A challenge bit $b \leftarrow\$ \{0,1\}$ selects between the real construction interface (real ciphertexts, real verification) and an ideal interface (uniform strings of matching length, verification that accepts only previously-issued encryption tuples). The adversary also accesses the public primitive interface (π, π^{-1}), exposed identically in both worlds. This follows the same-oracle convention of [BT16], who work in the ideal-cipher model and give both worlds the same ideal-cipher oracles; here the shared primitive is the random permutation. The adversary outputs a guess b' and wins if $b' = b$.

Game and advantage. The game $\mathsf{G}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A})$ of [BT16, Fig. 3] samples $b \leftarrow\!\!\$\ \{0,1\}$ and maintains a user counter v, a set V of (d,U) pairs already used by encryption, and a set W of recorded encryption tuples; its New, Enc, and Vf oracles are instantiated for TW128 in Section 6.2, together with the public primitive interface (π,π^{-1}), which does not depend on the challenge bit. We write the user index as d and the associated data input as A, where [BT16] writes i and H, and take the absolute value of their signed advantage so that

$$\begin{aligned}\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A}) &= \left|2\Pr[\mathsf{G}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A}) \text{ returns } 1] - 1\right| \\ &= \left|\Pr[\mathcal{A}^{\mathsf{Enc},\mathsf{Vf},b=1} \Rightarrow 1] - \Pr[\mathcal{A}^{\mathsf{Enc},\mathsf{Vf},b=0} \Rightarrow 1]\right|\end{aligned}$$

the second equality holding because b is uniform. Thus $\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}$ is the two-game distinguishing advantage between the real $(\mathsf{Enc},\mathsf{Ver})$ pair and the ideal $(\mathsf{Rand},\mathsf{Replay})$ pair instantiated per user. The single-user case $u=1$ recovers the all-in-one AE notion of Section 2.3, augmented with the public primitive interface (the real permutation in both worlds).

Per-user nonce uniqueness. Nonce uniqueness is enforced per user via the set V: the same nonce may appear under distinct users but not twice under the same user. This is strictly weaker than the global nonce-uniqueness requirement that would force each nonce value U to appear under at most one user.

3 The TW128 Construction

3.1 Tree Structure

TW128 is a nonce-based AEAD built as a two-level tree of duplex objects over Keccak-p[1600, 12]. The *root* duplex is initialized with (K,U), absorbs the associated data and the first ciphertext chunk, and emits the final τ_{root}-bit root tag T. The remaining plaintext is partitioned into *leaf* chunks $M_1,\ldots,M_n$; leaf j is encrypted by an independent duplex initialized with (K,U,j) and returns a τ_{leaf}-bit leaf tag T_j. For multi-chunk messages, the root aggregates $T_1 \,\|\, \cdots \,\|\, T_n \,\|\, \langle n\rangle_8$ before emitting T. Thus the authentication tag is also the digest point for the full (K,U,A,M) context. The construction is depicted in Figure 1; the exact edge-case conventions are specified in Section 3.4.

The tree fixes the transcript but not the implementation's parallelism. Leaves operate on disjoint states after (K,U) is fixed, so scalar, SIMD, and multicore implementations can evaluate different numbers of leaves at once while producing identical ciphertexts and tags. The root state and each leaf state are bounded 1600-bit objects, and leaf tags can be aggregated as a stream, giving constant working memory at any fixed degree of parallelism. This is the AEAD analog of KangarooTwelve's final-node hashing of per-chunk chaining values, with leaf tags as the chaining values and the root tag as the final digest.

3.2 Parameters and Domain

Table 1 lists the user-visible and derived scheme parameters. The implementation uses the same byte length for the root tag and each leaf tag, but the proofs of Sections 5 and 6 keep τ_{root} and τ_{leaf} as separate symbols.

The bound $M_{\max} = 8183 \cdot 2^{64}$ bytes is the largest plaintext whose leaf index fits in $\langle\cdot\rangle_8$: writing $\mathrm{leaves}(M) = \max(0, \lceil \|M\|/\beta\rceil - 1)$, every supported plaintext satisfies $0 \le \mathrm{leaves}(M) \le 2^{64}-1$, so leaf indices lie in $\{1,\ldots,2^{64}-1\}$. The associated data bound $A_{\max}$ is a fixed scheme parameter, taken equal to $M_{\max}$. Inputs exceeding these bounds are outside the scheme domain.

The derived per-call absorption bound ρ follows the [BDPVA11] analysis: with $r = 1344$ and $\mathsf{pad10^*1}$ padding, $\rho_{\max}(\mathsf{pad10^*1}, r) = r - 2 = 1342$ bits. The byte-aligned data field

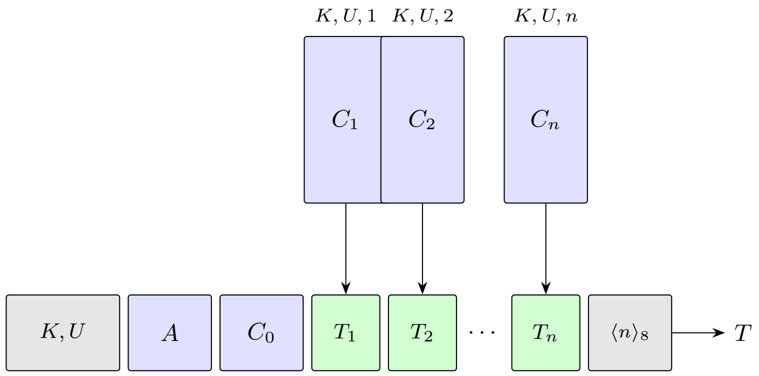

Figure 1: Schematic of TW128 for $n \geq 1$ leaves. The root duplex (bottom row) absorbs the associated data, root ciphertext chunk, leaf tags, and leaf count before emitting the root tag T. Each independent leaf duplex (top row) encrypts one leaf chunk and emits a leaf tag.

Table 1: TW128 parameters.

Parameter	Value	Description
k	256	Key length (bits)
ν	256	Nonce length (bits)
τ_{root}	256	Root tag length (bits)
τ_{leaf}	256	Leaf tag length (bits)
β	8183	Plaintext chunk size (bytes)
$M_{\max}$	$8183 \cdot 2^{64}$ bytes	Maximum plaintext length
$A_{\max}$	$8183 \cdot 2^{64}$ bytes	Maximum associated data length
b	1600	Permutation state width (bits)
n_r	12	Permutation rounds
r	1344	Sponge rate (bits)
c	256	Sponge capacity (bits)
ρ	167	Maximum byte-aligned absorption per duplex call (bytes)

shares this allotment with the four-bit phase suffix of Section 3.3, so ρ is the largest integer satisfying $8\rho + 4 \leq 1342$, giving $\rho = 167$ data bytes (and $8\rho + 4 = 1340 \leq 1342$).

Parameter rationale. The capacity, key length, and tag lengths are all set to 256 bits so the flat sponge loss, key-guessing terms, tag-guessing terms, and root/leaf collision terms meet a 128-bit target at the concrete work cap of Section 7.1. With $b = 1600$, this fixes the rate at $r = 1344$. The 12-round Keccak-p[1600, 12] permutation matches KangarooTwelve [BDP+16]; Section 9.4 discusses the cryptanalytic record for this round count. The chunk size is $\beta = 49\rho = 8183$ bytes, the largest multiple of the byte-aligned per-call absorption bound not exceeding 2^{13} bytes; this fully utilizes message duplex calls inside full leaves while keeping the leaf working set small. The 256-bit nonce is not required by the nonce-respecting bounds, but it is essentially free in the root initialization call and supports both random nonces and narrower counter nonces by zero-padding.

3.3 Domain Separation

Distinct duplex instances and phases are separated at the transcript level by initialization prefixes and phase suffixes. The initialization strings carry a 6-byte ASCII prefix and, for leaves, an eight-byte little-endian chunk index:

$$p_{\mathsf{root}} = \texttt{"TW128R"} \in \{0,1\}^{48},$$
$$p_{\mathsf{leaf}} = \texttt{"TW128L"} \in \{0,1\}^{48}.$$

The root's first absorbed string is $p_{\mathsf{root}} \parallel K \parallel U$, and leaf j's first absorbed string is $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j\rangle_8$. Every byte-aligned absorption also carries one of the four-bit phase suffixes in Table 2, written least significant bit first: absorbing a byte block σ under suffix σ_{ph} feeds $\sigma \parallel \sigma_{\mathsf{ph}}$ to the duplex interface of Section 2.2 before pad10*1 padding.

Table 2: Phase suffixes (4 bits each, appended LSB-first).

Name	Hex	Use
σ_{init}	0xC	End of initialization (the sole init call)
σ^{-}_{ad}	0x9	Non-final associated data block
σ^{+}_{ad}	0xD	Final associated data block
$\sigma^{-}_{\mathsf{msg}}$	0xA	Non-final message block
$\sigma^{+}_{\mathsf{msg}}$	0xE	Final message block
$\sigma^{-}_{\mathsf{agg}}$	0xB	Non-final aggregation block
$\sigma^{+}_{\mathsf{agg}}$	0xF	Final aggregation block (emits root tag)

Table 3 summarizes the root and leaf transcript schedule. The algorithms of Sections 3.4 and 3.5 give the normative byte-level specification; the table records which phase is present and what output the final call of that phase supplies.

Table 3: TW128 transcript schedule. "First M_i keystream" means $8\min(\|M_i\|, \rho)$ output bits for the first duplex block of that message chunk; subsequent message-phase outputs have the length of the next duplex block, as in Algorithm 1.

Instance	Phase	Suffixes	Absorbed string	Output supplied	When
Root	Init	σ_{init}	$p_{\mathsf{root}} \parallel K \parallel U$	0 if $A \neq \varepsilon$; otherwise first M_0 keystream	Always
Root	AD	σ^{-}_{ad}, σ^{+}_{ad}	ρ-byte block partition of A	Final call emits first M_0 keystream	$A \neq \varepsilon$
Root	Message	$\sigma^{-}_{\mathsf{msg}}$, $\sigma^{+}_{\mathsf{msg}}$	ρ-byte block partition of C_0	Non-final calls emit next keystream; final call emits T if $n = 0$, and 0 otherwise	Always
Leaf j	Init	σ_{init}	$p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j\rangle_8$	First M_j keystream	$1 \leq j \leq n$
Leaf j	Message	$\sigma^{-}_{\mathsf{msg}}$, $\sigma^{+}_{\mathsf{msg}}$	ρ-byte block partition of C_j	Non-final calls emit next keystream; final call emits T_j	$1 \leq j \leq n$
Root	Aggregation	$\sigma^{-}_{\mathsf{agg}}$, $\sigma^{+}_{\mathsf{agg}}$	$T_1 \parallel \cdots \parallel T_n \parallel \langle n\rangle_8$	Final call emits T	$n \geq 1$

The combined effect of the initialization prefixes and the phase suffixes is that the flattened sponge input of any TW128 duplex call uniquely determines the duplex instance, the leaf index when applicable, the phase, and the duplex-call position within that phase. Definition 2 (Well-formed TW128 transcript prefixes), Lemma 6 (Transcript Injectivity), and its structural corollaries formalize this parsing fact for the proofs of Sections 5 and 6 and Corollary 5.

3.4 Encryption

Algorithm 2 specifies $\mathsf{Enc}_K(U, A, M)$ using the helper procedures of Algorithm 1. The helpers sit below the tree layer: the encryption and decryption algorithms split bodies into β-byte chunks, while the helpers split a chosen byte string into ρ-byte duplex blocks. StreamEnc XORs each plaintext block with the current keystream, absorbs the resulting ciphertext block under the message suffix, and uses the duplex output as the next keystream.

Algorithm 1 TW128 helper procedures.

procedure $\textsc{Absorb}(\mathcal{D}, s, \sigma^-, \sigma^+, \ell)$
  Partition s into duplex blocks $s_1, \ldots, s_t$ of exactly ρ bytes each, except that the last block may be shorter.
  Require $t \geq 1$; the empty case yields the single empty duplex block $s_1 = \varepsilon$.
  for $i \leftarrow 1$ **to** $t - 1$ **do**
    $\mathcal{D}.\mathsf{duplexing}(s_i \parallel \sigma^-, 0)$
  end for
  return $\mathcal{D}.\mathsf{duplexing}(s_t \parallel \sigma^+, \ell)$
end procedure

procedure $\textsc{StreamEnc}(\mathcal{D}, m, Z, \sigma^-, \sigma^+, \ell)$
  Partition m into duplex blocks $m_1, \ldots, m_t$ of exactly ρ bytes each, except that the last block may be shorter.
  Require $t \geq 1$; if $m = \varepsilon$, set $t = 1$ and $m_1 = \varepsilon$.
  for $i \leftarrow 1$ **to** $t - 1$ **do**
    $c_i \leftarrow m_i \oplus Z$
    $Z \leftarrow \mathcal{D}.\mathsf{duplexing}(c_i \parallel \sigma^-, 8\|m_{i+1}\|)$
  end for
  $c_t \leftarrow m_t \oplus Z$
  $Z' \leftarrow \mathcal{D}.\mathsf{duplexing}(c_t \parallel \sigma^+, \ell)$
  return $(c_1 \parallel \cdots \parallel c_t, Z')$
end procedure

This is the duplex-level presentation of the overwrite mechanism of [BDPVA11, §6.2 and Algorithm 5]. The decryption helper $\textsc{StreamDec}$ uses the same schedule with ciphertext blocks as input: it recovers $m_i = c_i \oplus Z$, absorbs the same c_i, and returns the plaintext string and final output value.

Edge-case convention. The helper partition always has at least one block: an empty input is the single empty final block carrying the final phase suffix. The associated-data phase is present exactly when $A \neq \varepsilon$; when $A = \varepsilon$, the root σ_{init}call emits the first root-chunk keystream. The first keystream request for a message string X is $8\min(\|X\|, \rho)$ bits. If $n = 0$, the root message phase requests τ_{root} bits on its final σ^+_{msg}call and no aggregation phase occurs. If $n \geq 1$, that root message call requests zero bits, each leaf final message call requests τ_{leaf} bits, and the final σ^+_{agg}aggregation call requests τ_{root} bits. These conventions apply identically to decryption.

The leaf loop is annotated **in parallel** to make explicit that the n leaf computations are mutually independent; Section 8.1 discusses the SIMD realizations. The annotation is an implementation freedom, not a mode parameter: evaluating leaves serially, in SIMD batches, or across cores leaves the transcript and wire format unchanged.

Streaming aggregation. The string $G = T_1 \parallel \cdots \parallel T_n$ in Algorithms 2 and 3 names the leaf tags only for specification. The closing $\textsc{Absorb}$ consumes $G \parallel \langle n \rangle_8$ as ρ-byte duplex blocks in leaf order and advances the root state after each, so an implementation need not hold G in full. It can absorb leaf tags in order as they become available, or buffer completed out-of-order leaves until the next in-order tag or SIMD batch can be absorbed, keeping only a bounded number of tags at a time. With the bounded 1600-bit state of every duplex, this keeps the working memory constant in $\|M\|$ for a fixed degree of parallelism, growing only with the number of leaves evaluated at once rather than with the message length; Section 8.1 describes the batched realization.

Algorithm 2 $\mathsf{Enc}_K(U, A, M)$.

Require: $K \in \{0,1\}^k$, $U \in \{0,1\}^\nu$, A with $\|A\| \leq A_{\max}$, M with $\|M\| \leq M_{\max}$.
Ensure: $C \parallel T$ with $\|C\| = \|M\|$ and $T \in \{0,1\}^{\tau_{\mathsf{root}}}$.

$M_0 \leftarrow$ first $\min(\|M\|, \beta)$ bytes of M (possibly ε).
Partition the remaining suffix into chunks $M_1, \ldots, M_n$ of exactly β bytes each, except that the last chunk may be shorter but nonempty, so $n = \mathrm{leaves}(M)$.
$\mathcal{D}_{\mathrm{root}} \leftarrow$ new Duplex
if $A = \varepsilon$ **then**
  $Z \leftarrow \mathcal{D}_{\mathrm{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \parallel K \parallel U \parallel \sigma_{\mathsf{init}}, 8\min(\|M_0\|, \rho))$
else
  $\mathcal{D}_{\mathrm{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \parallel K \parallel U \parallel \sigma_{\mathsf{init}}, 0)$
  $Z \leftarrow \textsc{Absorb}(\mathcal{D}_{\mathrm{root}}, A, \sigma^-_{\mathsf{ad}}, \sigma^+_{\mathsf{ad}}, 8\min(\|M_0\|, \rho))$
end if
if $n = 0$ **then**
  $(C_0, T) \leftarrow \textsc{StreamEnc}(\mathcal{D}_{\mathrm{root}}, M_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{root}})$
  return $C_0 \parallel T$
end if
$(C_0, \_) \leftarrow \textsc{StreamEnc}(\mathcal{D}_{\mathrm{root}}, M_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, 0)$
for $j \leftarrow 1$ **to** n **in parallel do**
  $\mathcal{D}_j \leftarrow$ new Duplex
  $Z_j \leftarrow \mathcal{D}_j.\mathsf{duplexing}(p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8 \parallel \sigma_{\mathsf{init}}, 8\min(\|M_j\|, \rho))$
  $(C_j, T_j) \leftarrow \textsc{StreamEnc}(\mathcal{D}_j, M_j, Z_j, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{leaf}})$
end for
$G \leftarrow T_1 \parallel \cdots \parallel T_n$.
$T \leftarrow \textsc{Absorb}(\mathcal{D}_{\mathrm{root}}, G \parallel \langle n \rangle_8, \sigma^-_{\mathsf{agg}}, \sigma^+_{\mathsf{agg}}, \tau_{\mathsf{root}})$
return $C_0 \parallel C_1 \parallel \cdots \parallel C_n \parallel T$

3.5 Decryption and Correctness

Algorithm 3 specifies $\mathsf{Dec}_K(U, A, C \parallel T)$. Decryption mirrors encryption at the duplex block level: each ciphertext block is absorbed under the same phase suffix, and the plaintext block is recovered as $m_i = c_i \oplus Z^{(i)}$ from the keystream value that encryption would have used at that position. The edge cases are those of Section 3.4. The procedure absorbs all in-domain ciphertext bytes before comparing the final tag, so the verification transcript is determined by (K, U, A, C) whether or not the submitted tag accepts.

Correctness. For every $K \in \{0,1\}^k$, $U \in \{0,1\}^\nu$, A with $\|A\| \leq A_{\max}$, and M with $\|M\| \leq M_{\max}$, $\mathsf{Dec}_K(U, A, \mathsf{Enc}_K(U, A, M)) = M$. Encryption and decryption absorb the same ciphertext blocks under the same duplex instances and phase suffixes, so their duplex states and final root tags agree. Each recovered block satisfies $c_i \oplus Z^{(i)} = (m_i \oplus Z^{(i)}) \oplus Z^{(i)} = m_i$.

Lemma 1 (TW128 tidiness)**.** *For every valid TW128 decryption input, if* $\mathsf{Dec}_K(U, A, C \parallel T) = M \neq \perp$, *then* $\mathsf{Enc}_K(U, A, M) = C \parallel T$.

Proof. An accepting decryption has $\|M\| = \|C\|$ and therefore the same canonical chunking as re-encryption of M. The decryption transcript absorbs the same (K, U, A) values and ciphertext blocks that this re-encryption absorbs, under the same duplex instances and phase suffixes. Its stream outputs recover $m_i = c_i \oplus Z^{(i)}$, so re-encryption emits the submitted ciphertext blocks again. The recomputed leaf tags and final root candidate tag are therefore the same values; since decryption accepts only when the final candidate equals the submitted tag T, re-encryption returns $C \parallel T$. ∎

Algorithm 3 $\mathsf{Dec}_K(U, A, C \parallel T)$.

Require: K, U, A as in Algorithm 2, ciphertext C with $\|C\| \leq M_{\max}$, candidate tag $T \in \{0,1\}^{\tau_{\mathsf{root}}}$.

Ensure: M with $\|M\| = \|C\|$, or $\perp$.

$C_0 \leftarrow$ first $\min(\|C\|, \beta)$ bytes of C (possibly ε).
Partition the remaining suffix into chunks $C_1, \ldots, C_n$ of exactly β bytes each, except that the last chunk may be shorter but nonempty, so $n = \mathrm{leaves}(C)$.
$\mathcal{D}_{\mathrm{root}} \leftarrow$ new Duplex
if $A = \varepsilon$ **then**
  $Z \leftarrow \mathcal{D}_{\mathrm{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \parallel K \parallel U \parallel \sigma_{\mathsf{init}}, 8\min(\|C_0\|, \rho))$
else
  $\mathcal{D}_{\mathrm{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \parallel K \parallel U \parallel \sigma_{\mathsf{init}}, 0)$
  $Z \leftarrow \textsc{Absorb}(\mathcal{D}_{\mathrm{root}}, A, \sigma^-_{\mathsf{ad}}, \sigma^+_{\mathsf{ad}}, 8\min(\|C_0\|, \rho))$
end if
if $n = 0$ **then**
  $(M_0, T') \leftarrow \textsc{StreamDec}(\mathcal{D}_{\mathrm{root}}, C_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{root}})$
  return M_0 if $T' = T$ (compared in constant time), else $\perp$.
end if
$(M_0, \_) \leftarrow \textsc{StreamDec}(\mathcal{D}_{\mathrm{root}}, C_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, 0)$
for $j \leftarrow 1$ **to** n **in parallel do**
  $\mathcal{D}_j \leftarrow$ new Duplex
  $Z_j \leftarrow \mathcal{D}_j.\mathsf{duplexing}(p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8 \parallel \sigma_{\mathsf{init}}, 8\min(\|C_j\|, \rho))$
  $(M_j, T_j) \leftarrow \textsc{StreamDec}(\mathcal{D}_j, C_j, Z_j, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{leaf}})$
end for
$G \leftarrow T_1 \parallel \cdots \parallel T_n$.
$T' \leftarrow \textsc{Absorb}(\mathcal{D}_{\mathrm{root}}, G \parallel \langle n \rangle_8, \sigma^-_{\mathsf{agg}}, \sigma^+_{\mathsf{agg}}, \tau_{\mathsf{root}})$
if $T' = T$ (compared in constant time) **then**
  return $M_0 \parallel M_1 \parallel \cdots \parallel M_n$
else
  return $\perp$
end if

4 Security Model

This section fixes the shared proof framework for TW128: the public permutation convention, input-validity convention, intermediate random oracle model, resource accounting, and recurring loss terms. The formal game definitions live with the theorem sections that use them: tag hash and committing games in Section 5.1, and AEAD-facing games in Sections 6.1 and 6.2.

Table 4 summarizes the proof obligations before those formal games. The closed-form loss terms named there are defined in Section 4.4; the leaf tag and root tag guessing terms of the AE-facing rows are stated inline in the theorems of Section 6. The implemented-parameter instantiations appear in Section 7.

Proof worlds. All final statements are public-permutation claims for TW128 with $\textsc{Keccak}\text{-}p[1600, 12]$ modeled as a uniformly random permutation. The proofs first establish random oracle bounds for the flat duplex interface of Section 4.2; the flat sponge lift of Section B then transfers those bounds to the public-permutation games of Sections 5.1, 6.1 and 6.2.

Table 4: Proof obligations and the dominant loss shapes used for TW128. The table is only a roadmap; the formal games and resource accounting are defined in Sections 4.3, 5.1, 6.1 and 6.2.

Notion	Adversary goal	Main result	Loss shape
$\mathsf{H}_{\mathsf{TW128}}$ collision	Find distinct full inputs (K, U, A, M) with the same root tag.	Theorem 4	Capacity lift, root collision, leaf collision
$\mathsf{H}_{\mathsf{TW128}}$ preimage	Find an input hitting a random challenge tag (Pre) or a fixed target tag (ePre), in the [RS04] senses.	Theorems 5 and 6	Capacity lift, root preimage; leaf collision and a source-guess term for Pre
$\mathsf{H}_{\mathsf{TW128}}$ second preimage	Find a second preimage in the standard or everywhere [RS04] sense.	Corollary 2	Inherited from collision resistance
CMT / CMTD	Produce equal encryptions or one valid ciphertext opening under distinct committed inputs.	Corollaries 4 and 5	Capacity lift, root collision
INT-RUP	Produce a fresh accepting ciphertext after encryption and plaintext-computing queries.	Theorem 11	Capacity lift, key guess, leaf tag, root tag guess
AE / mu-ind	Distinguish real encryption and verification from random ciphertexts and replay-only verification, with nonce-respecting encryption queries.	Theorem 9 and Corollary 6	Capacity lift, key guess, leaf tag, root tag guess

Scope of the security claims. In the AE-style distinguishing games, encryption queries are nonce-respecting and the verification oracle returns only acceptance or rejection. The INT-RUP game gives integrity under a plaintext-computing oracle, but it does not give privacy under release of unverified plaintext or plaintext-awareness security. None of the AE-style games gives nonce-misuse resistance. The hash and committing games are adversarial-initialization games over submitted inputs (K, U, A, M) or ciphertext openings and impose no nonce uniqueness condition. Across all games, the stated bounds are birthday-type, single-stage, and leakage-free: beyond-birthday and side-channel security are outside the model. Section 9.5 discusses the consequences of these exclusions and the corresponding open problems, and Section 9.4 reviews the third-party cryptanalysis supporting the random-permutation modeling of KECCAK-p[1600, 12]. The random-permutation model also abstracts from dedicated cryptanalysis of TW128's tree topology, in particular the family of leaf initializations sharing (K, U) and differing only by leaf index; that mode-level question is identified as future work in Section 9.4.

The proof dependency spine is as follows. Section A establishes transcript decoding, bridge injectivity, output-point separation, and the local probabilistic accounting lemmas for key tests, leaf tag collisions, and root tag guesses. Section 5 uses those structural facts to build a chronological sequence of final root candidates and apply a single root tag hash bound; the hash and committing theorems differ only in which collision or preimage event is reduced to that candidate sequence. Section 6 uses the same facts for the random oracle mu-ind and INT-RUP bounds. Section B transfers both branches to the public permutation games defined in the theorem sections.

4.1 Public Permutation Framework

Primitive convention. In every public permutation game considered in this paper, the challenger samples $\pi \leftarrow\$ \mathsf{Perm}(\{0,1\}^{1600})$ and exposes the direct primitive interface (π, π^{-1}) to the adversary. The real-vs-ideal AE-style games compare the real construction interface with the ideal interface specified by the game over this shared primitive, following the same-oracle convention of [BT16]; the hash and committing games of Section 5.1 are one-world success probabilities over the same sampled permutation. The simulator $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ of Section B is a proof device, entering through the lift from the intermediate random oracle model.

Validity convention. All construction interface inputs are checked against TW128's public syntax and domain before any construction computation or construction-internal primitive lookup is performed. An encryption input is valid when $U \in \{0,1\}^\nu$, A is a byte string with $\|A\| \le A_{\max}$, and M is a byte string with $\|M\| \le M_{\max}$. A verification or decryption input is valid when $U \in \{0,1\}^\nu$, A is a byte string with $\|A\| \le A_{\max}$, and the full ciphertext parses as $C \,\|\, T$ with $\|C\| \le M_{\max}$ and $T \in \{0,1\}^{\tau_{\mathsf{root}}}$. In the hash and committing games, each submitted input (K, U, A, M) must be a valid encryption input including $K \in \{0,1\}^k$, and a submitted CMTDs ciphertext must be a valid verification/decryption ciphertext. Invalid encryption queries are rejected and not recorded, invalid plaintext-computing queries return $\perp$, invalid verification queries return 0, and invalid committing-game outputs make the game return 0. Such rejected inputs contribute no construction cost and create no construction transcripts. Throughout, an *accepted* encryption query is one that passes this validity check and any game-specific nonce or user check.

Construction oracles. Whenever a game uses a secret key K and permutation π, $\mathsf{Enc}_K[\pi]$ and $\mathsf{Dec}_K[\pi]$ are the algorithms of Algorithms 2 and 3 with that permutation. The verification oracle is $\mathsf{Ver}_K[\pi](U, A, C, T) = [\mathsf{Dec}_K[\pi](U, A, C \,\|\, T) \ne \perp]$. For INT-RUP, $\mathsf{PDec}_K[\pi](U, A, C, T)$ is the plaintext recovered by the decryption transcript before the final root-tag comparison; it returns neither the recomputed tag nor the verification bit.

4.2 The TW128 Random Oracle Model

The proofs of Sections 5 and 6 and Corollary 5 use an intermediate random oracle model. This model keeps the same construction-level games but replaces every duplex output by a lookup to a single random oracle through the [BDPVA11] bridge of Section 2.2. The flat sponge lift of Section B then converts the resulting bounds to the public-permutation form.

The model has a single random oracle $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ over the unpadded H-input domain of the [BDPVA08] simple-padded sponge interface, with lazy sampling: each distinct input receives an independent infinite bit string, and repeated queries return the same string. A direct adversary query is finite-output: on input (Y, ℓ) with finite ℓ, the oracle returns the first ℓ bits of the lazy-sampled value for Y.

Construction calls do not query $\mathsf{RO}_{\mathsf{flat}}$ on arbitrary strings. Every root or leaf duplex output is read from $\mathsf{RO}_{\mathsf{flat}}$ at the bridged flattened transcript prefix for that duplex call, and the caller takes only the requested output projection. Direct adversary queries to $\mathsf{RO}_{\mathsf{flat}}$ are the only random oracle queries counted as direct oracle queries; construction lookups are accounted for through the construction cost N of Section 4.3.

For a well-formed TW128 duplex transcript prefix X in the language of Definition 2, at one of the construction transcript lookup points formalized by Lemma 8 and Table 12, let $\mathsf{flat}(X) = I[1344, 1344](\mathsf{raw\_flat}(X))$ be the bridged image of the native flattened input under the [BDPVA11, Theorem 3] preprocessing map of Section 2.2, and define

$$\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X)).$$

The construction interfaces $\mathsf{Enc}^{\mathsf{RO}}_K$, $\mathsf{Dec}^{\mathsf{RO}}_K$, $\mathsf{PDec}^{\mathsf{RO}}_K$, and $\mathsf{Ver}^{\mathsf{RO}}_K$ are the TW128 algorithms of Section 3 with every root or leaf duplex output replaced by the corresponding requested projection of $\mathsf{RF}(X)$; the projection length is the duplexing call's output parameter ℓ, which may be zero. The bridge and the typed-restriction convention are formalized in

Lemma 8; the direct-query key-hit predicate ignores the adversary's requested output length and is formalized as KeyedTWOut in Lemma 8.

The random oracle games are obtained from the public-permutation games of Sections 5.1, 6.1 and 6.2 by this mechanical substitution: direct primitive access (π, π^{-1}) becomes finite-output access to $\mathsf{RO}_{\mathsf{flat}}$, real construction or deterministic challenger checks use the corresponding $^{\mathsf{RO}}$ algorithm, and the ideal oracles $(\mathsf{Rand}, \mathsf{Replay})$ are unchanged. We use the same advantage names with subscript RO: $\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{RO}}$, $\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{RO}}$, $\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{RO}}$, $\mathbf{Adv}^{\mathsf{CMTDs}\text{-}\ell}_{\mathsf{RO}}$, $\mathbf{Adv}^{\mathsf{CMTs}\text{-}\ell}_{\mathsf{RO}}$, and $\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{RO}}$ are the win probabilities of the corresponding random oracle games, with CMTs challengers comparing deterministic encryptions via $\mathsf{Enc}^{\mathsf{RO}}$. The all-in-one AE advantage $\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{RO}}$ is the corresponding real-vs-ideal distinguishing advantage with direct $\mathsf{RO}_{\mathsf{flat}}$ access in both worlds.

For mu-ind, $\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B})$ denotes the Section 6.2 game with the same New interface and per-user nonce discipline: user d uses $(\mathsf{Enc}^{\mathsf{RO}}_{K_d}, \mathsf{Ver}^{\mathsf{RO}}_{K_d})$ when $b = 1$ and $(\mathsf{Rand}, \mathsf{Replay})$ when $b = 0$. Equivalently, in the two-game notation of Section 2.7,

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}) = \left|\Pr[\mathcal{B}^{\mathsf{Real}^{\mathsf{mu}}_{\mathsf{RO}},\, \mathsf{RO}_{\mathsf{flat}}} \Rightarrow 1] - \Pr[\mathcal{B}^{\mathsf{Ideal}^{\mathsf{mu}},\, \mathsf{RO}_{\mathsf{flat}}} \Rightarrow 1]\right|,$$

where the superscripts name the entire multi-user construction interface in the corresponding world. Adversary queries to $\mathsf{RO}_{\mathsf{flat}}$ are finite-output and counted in $q_{\mathsf{RO}}(\mathcal{B})$ of Section 4.3. Construction-internal $\mathsf{RF}(\cdot)$ lookups (those made by $\mathsf{Enc}^{\mathsf{RO}}_K$, $\mathsf{Dec}^{\mathsf{RO}}_K$, $\mathsf{PDec}^{\mathsf{RO}}_K$, $\mathsf{Ver}^{\mathsf{RO}}_K$, or a hash or committing challenger) are not.

4.3 Resources

Adversary resources are measured by fixed, execution-uniform caps on the number and size of oracle queries. An execution-uniform cap is one that holds on every execution path, including over adversary randomness, oracle randomness, and adaptive query choices. Proofs may also introduce local execution-uniform caps, such as q_e for the number of accepted encryption queries, solely to index a finite hybrid sequence. Such local caps are part of the adversary's fixed query bounds, but they are not listed as headline resource parameters unless they affect the final bound.

Table 5 gives the compact notation summary used in later theorems. For the paired symbols, the lowercase form is the realized count in a random oracle experiment and the uppercase form is the execution-uniform cap used in the public-permutation bounds; the single-symbol quantities N, N_{prim}, and N_{tot} serve as both the realized count and its cap. The detailed definitions below fix exactly which computations contribute to each count, and Section 4.4 defines the closed-form loss terms.

***N*:** the construction interface cost, measured as the number of KECCAK-p[1600, 12] calls induced after the validity convention has been applied. It counts at per-duplex-call granularity, summing each root initialization, associated data duplex block, root message duplex block, leaf initialization, leaf message duplex block, and aggregation duplex block.

AE-style games. In real games, N counts the valid construction computations actually run by the adversary's oracle queries. In ideal games, the same accounting applies to the corresponding real $\mathsf{TW128}$ computation on the valid query, even when the ideal oracle answers by table lookup or without running the construction. Valid plaintext-computing computations and valid AE-style non-replay verification computations are counted whether they accept or reject; replay-table verification hits are transcript-consistency checks and are not charged to N or to the verification-tag terms.

Hash games. There is no adversary construction oracle. For the $\mathsf{H}_{\mathsf{TW128}}$ collision game, N counts the s deterministic encryptions on the submitted inputs. For the

Table 5: Resource caps and recurring loss terms. For paired resource symbols, the lowercase form is the realized count in random oracle executions or source games and the uppercase form is the cap appearing in public-permutation theorems; N, N_{prim}, and N_{tot} serve as both.

Symbol	Meaning	Primary use / definition
N	Construction-internal KECCAK-p[1600, 12] calls induced by valid construction oracle queries and deterministic challenger checks.	Public-permutation lift and concrete work cap.
N_{prim}	Direct adversary queries to the public primitive interface.	Key-guess, root-collision, root-preimage terms; direct-query part of the lift.
$N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$	Converted primitive-query cost used by the flat sponge replacement.	Argument of $\mathsf{Lift}_c(N_{\mathsf{tot}})$.
$q_{\mathsf{RO}}(\mathcal{B})$, Q_{RO}	Direct finite-output queries to $\mathsf{RO}_{\mathsf{flat}}$ in the intermediate random oracle model; construction lookups are excluded.	Random oracle bounds; after the lift $q_{\mathsf{RO}} \leq N_{\mathsf{prim}}$.
$q_{\mathsf{v}}(\mathcal{A})$, Q_{v}	Valid verification queries that are not replay-table hits.	Verification-tag terms in AE, mu-ind, and INT-RUP bounds.
$n_{\mathsf{leaf}}(\mathcal{B})$, N_{leaf}	Distinct construction-generated final leaf transcripts.	Leaf tag collision and tag-guessing terms; always at most N.
D	Execution-uniform cap on users activated in the mu-ind game.	Multi-user key-collision and key-guess terms.
$\mathsf{Lift}_c(N_{\mathsf{tot}})$	[BDPVA08] flat sponge differentiating loss.	Defined in § 4.4.
$\mathsf{KeyGuess}_k(Q)$	Ideal-system key-guessing term.	Defined in § 4.4.
$\mathsf{RootColl}_\tau(Q, s)$	s-way root-tag collision term.	Defined in § 4.4.
$\mathsf{RootPre}_\tau(Q)$	Root-tag preimage term.	Defined in § 4.4.
$\mathsf{LeafColl}_\tau(Q)$	Known-key leaf-tag collision term.	Defined in § 4.4.

standard preimage game it counts both challenger encryptions: the source encryption computing $T^* = \mathsf{H}_{\mathsf{TW128}}(X^*)$ and the deterministic encryption on the submitted input. For the everywhere-preimage game it counts the deterministic encryption on the submitted input. For the Sec and eSec games of Corollary 2, N and N_{prim} are those of the explicit induced two-way collision adversary in that corollary, whose source and candidate deterministic encryptions are counted exactly as in the $s = 2$ collision game.

Committing games. For CMTDs-ℓ, N uses conservative committing-game accounting: after the syntax, domain, and message-length checks, it charges the deterministic decryption checks on the common ciphertext whether or not the particular WiC_ℓ distinctness check would reject before those checks. Likewise, for the CMTs-ℓ source games of Corollary 5, after syntax and domain checks N charges the deterministic encryptions that would be used to compare the s ciphertexts whether or not the WiC_ℓ distinctness check would reject earlier.

$\boldsymbol{N_{\mathsf{prim}}}$**:** the number of direct adversary queries to the primitive interface, the pair (π, π^{-1}) in public permutation games.

$\boldsymbol{N_{\mathsf{tot}} = N + N_{\mathsf{prim}}}$**:** the total primitive-query cost used by the flat sponge lift. The lift of Section B is applied at N_{tot} because after the operational duplex-trace accounting of Lemma 19, the construction-internal duplex calls counted by N and the adversary's direct primitive-interface queries counted by N_{prim} form a primitive-query sequence of cost at most $N + N_{\mathsf{prim}}$.

$\boldsymbol{q_{\mathsf{v}}(\mathcal{A}), Q_{\mathsf{v}}}$**:** the number of verification queries with valid inputs that are not replay-table hits, and a cap on it. Used in the verification terms of the direct mu-ind, AE, and INT-RUP bounds of Section 6. Every counted verification query contributes at least one verification/decryption construction call in the N accounting, so $q_{\mathsf{v}}(\mathcal{A}) \leq N$.

$q_{\mathsf{RO}}(\mathcal{B})$, Q_{RO}: the number of direct finite-output queries to $\mathsf{RO}_{\mathsf{flat}}$ by a random oracle model adversary $\mathcal{B}$, and a cap on it. Repeated direct queries on the same Y are counted again. Construction-internal $\mathsf{RF}(\cdot)$ lookups are not counted. After the derived-adversary construction, q_{RO} counts only simulator-induced $\mathsf{RO}_{\mathsf{flat}}$ calls from the adversary's direct primitive-interface queries, not the construction calls already counted in N for the flat sponge lift.

$n_{\mathsf{leaf}}(\mathcal{B})$, N_{leaf}: the number of distinct construction-generated final leaf transcripts in the execution of $\mathcal{B}$, and a cap on it. Distinct final leaf transcripts are generated by valid encryption computations, by valid plaintext-computing computations, by valid AE-style verification computations (whether they accept or reject), and by hash and committing challenger construction computations that pass the early validity checks, including the preimage game's source encryption.

For a byte string X,

$$\mathrm{leaves}(X) = \max\bigl(0,\ \lceil \|X\|/\beta \rceil - 1\bigr)$$

counts the leaf chunks induced by a body byte string of length $\|X\|$. Thus n_{leaf} is bounded by the sum of leaves(X) over the valid construction computations that create final leaf transcripts, where X is the plaintext body for encryption computations and the submitted ciphertext body for plaintext-computing, verification, and committing decryption checks. Each distinct final leaf transcript contains a final leaf duplex call counted in N, so $n_{\mathsf{leaf}}(\mathcal{B}) \leq N$ unconditionally. The ratio n_{leaf}/N is largest when most processed bytes lie in leaf chunks, where each full leaf contributes $1 + \beta/\rho = 50$ duplex calls to N per final leaf transcript.

All theorems are stated for adversaries whose execution-uniform caps hold at the stated bounds.

4.4 Loss Terms

Five closed-form expressions recur across the security bounds: Lift_c throughout, with $\mathsf{KeyGuess}_k$, $\mathsf{RootColl}_\tau$, $\mathsf{RootPre}_\tau$, and $\mathsf{LeafColl}_\tau$ in some. We define them here, after the resource caps that parameterize them.

Flat sponge loss. The flat sponge differentiating loss is

$$\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}} + 1)/2^{c+1}.$$

For $N_{\mathsf{tot}} < 2^c$ it upper-bounds the [BDPVA08] sponge-differentiating advantage; Lemma 20 carries out the derivation from the [BDPVA08, Lemmas 4 and 5] product bounds through the Weierstrass product inequality. For $N_{\mathsf{tot}} \geq 2^c$, $\mathsf{Lift}_c(N_{\mathsf{tot}}) \geq 1$ and the advantage bound is trivial without any appeal to indifferentiability, and already at $N_{\mathsf{tot}} \geq 2^{c/2}$ the bound exceeds $1/2$ and provides no meaningful security.

Key-guessing term. The ideal system key-guessing term of [BDPVA11] is

$$\mathsf{KeyGuess}_k(Q) = Q/2^k.$$

It accounts for direct $\mathsf{RO}_{\mathsf{flat}}$ queries whose inputs satisfy the decoder $\mathsf{KeyedTWOut}$ of Table 12. Each such query is an adaptive test of the decoded candidate key against the hidden key K, independent of the query's requested output length. Lemma 10 bounds the probability that any of Q direct queries tests the right key by $Q/2^k$.

Root-collision term. The s-way root tag multi-collision term is

$$\mathsf{RootColl}_\tau(Q,s) = \binom{Q+s}{s} / 2^{\tau(s-1)},$$

with the abbreviation $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(Q,s) = \mathsf{RootColl}_\tau(Q,s)$ at $\tau = \tau_{\mathsf{root}}$. The $+s$ is candidate-sequence slack for the s final root transcript inputs generated by the challenger after the adversary outputs its tuples. These lookups are construction-internal and are not added to $q_{\mathsf{RO}}(\mathcal{B})$; after the flat sponge lift of Section B, the public-permutation bounds instantiate Q as N_{prim}.

Root-preimage term. The root tag preimage term is

$$\mathsf{RootPre}_\tau(Q) = (Q+1)/2^\tau,$$

with the abbreviation $\mathsf{RootPre}_{\tau_{\mathsf{root}}}(Q) = \mathsf{RootPre}_\tau(Q)$ at $\tau = \tau_{\mathsf{root}}$. It is the preimage analog of $\mathsf{RootColl}_\tau$: a single query-independent target tag replaces the s-way internal collision, one challenger lookup replaces the s final-root lookups, and the exponent drops from $\tau(s-1)$ to τ. As in the root-collision term, the flat sponge lift gives $Q = N_{\mathsf{prim}}$.

Leaf-collision term. The known-key leaf tag collision term is

$$\mathsf{LeafColl}_\tau(Q) = \binom{Q}{2} / 2^\tau.$$

It is the pairwise birthday bound on τ-bit leaf tag projections used by the collision-resistance theorem (Theorem 4). In this game the keys are adversary-supplied, so direct primitive queries can pre-read candidate leaf tag points; after the flat sponge lift this gives $Q = N_{\mathsf{prim}} + N_{\mathsf{leaf}}$. The hidden-key AE-style bounds instead carry the linear same-slot term $n_{\mathsf{leaf}}/2^{\tau_{\mathsf{leaf}}}$ of Lemma 11, written inline.

5 The Authentication Tag as a Hash Output

Externally, a TW128 encryption returns a ciphertext body and one authentication tag. Internally, that tag is the final root-duplex output: leaf tags summarize leaf chunks, the root aggregates those summaries with the context and root ciphertext chunk, and the wrapper $\mathsf{H}_{\mathsf{TW128}}$ returns the tag alone. This section proves that the visible tag is a hash output for the full opening (K, U, A, M), not merely an AEAD verification value.

The hash notions are the standard ones for $\mathsf{H}_{\mathsf{TW128}}$: collision, preimage, second-preimage, and everywhere preimage resistance. The committing statements are consequences of that hash security: a successful compact opening must force distinct openings to agree on the same τ_{root}-bit tag, the root-collision event bounded below.

The proof uses three ingredients. Transcript parsing separates final root inputs, except for the explicitly charged leaf-tag collision event. Lemma 3 orders every relevant final-root input into a chronological candidate sequence, and Lemma 4 applies the adaptive collision and preimage bound of Section A to that sequence. The random oracle theorems are short instantiations; the flat sponge lift gives the public-permutation bounds; the [BH22] committing notions reuse the same root-collision bound.

Table 6 maps the final claims to the proof ingredients and loss terms they use. It is only a dependency guide; the formal games and resource accounting remain those of Section 4.

5.1 Tag Hash and Committing Games

The public-permutation games in this section use the primitive, validity, and construction-oracle conventions of Section 4.1. The random oracle versions are obtained by the mechanical substitution of Section 4.2.

Table 6: Proof dependency map for the final TW128 bounds. "Leaf tag" denotes the linear hidden-key leaf term in the AE-facing bounds and LeafColl where shown explicitly; "root tag" denotes the verification or root-preimage guessing term for the relevant game.

Claim	Dependency summary
Coll, Theorem 4	Terms: Lift_c, LeafColl, RootColl. Ingredients: final-root input separation under no leaf collision; adaptive root candidate collision bound.
Pre, Theorem 5	Terms: Lift_c, LeafColl, RootPre, source-guess term. Ingredients: source/output separation under no leaf collision; sampled-source candidate hit bound.
ePre, Theorem 6	Terms: Lift_c, RootPre. Ingredient: fixed-target root candidate hit bound.
CMTDs-4, Corollary 4	Terms: Lift_c, RootColl. Ingredients: common ciphertext body; opening-triple separation.
CMT/CMTD, Corollary 5	Terms: Lift_c, RootColl. Ingredients: [BH22] hierarchy for D-notions; direct encryption-source root collision argument for CMTs.
mu-ind and AE, Theorem 9 and Corollary 6	Terms: Lift_c, $\mathsf{Lift}_c(N_{\mathsf{prim}})$, KeyGuess, leaf tag, root tag. Ingredients: adjacent-user hybrid; hidden-key leaf collision and verification-first root guessing.
INT-RUP, Theorem 11	Terms: Lift_c, KeyGuess, leaf tag, root tag. Ingredients: plaintext-computing work charged to transcript resources; hidden-key leaf collision and INT-RUP root guessing.

Hash games. The hash games for the wrapper $\mathsf{H}_{\mathsf{TW128}}$ (Definition 1) have no challenger-sampled key. In the s-way collision game, all inputs are adversary-supplied: the adversary outputs s inputs, and the challenger returns 1 iff they are pairwise distinct, in the TW128 domain, and share a root tag:

$$\pi \leftarrow\$ \mathsf{Perm}(\{0,1\}^{1600}), \quad (K_i, U_i, A_i, M_i)_{i=1}^{s} \leftarrow\$ \mathcal{A}^{\pi,\pi^{-1}},$$

$$\text{return 0 unless the inputs are pairwise distinct and in the TW128 domain,}$$

$$\text{return } \left[\mathsf{H}_{\mathsf{TW128}}(K_1, U_1, A_1, M_1) = \cdots = \mathsf{H}_{\mathsf{TW128}}(K_s, U_s, A_s, M_s)\right],$$

with advantage $\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{TW128}}(\mathcal{A}) = \Pr[\text{the game returns 1}]$; no nonce uniqueness is imposed. The [RS04] standard and everywhere second-preimage notions are used on fixed input slices; in Corollary 2 they are reduced explicitly to this two-way collision game. For fixed byte lengths a, μ, the standard preimage game samples $X^* \leftarrow\$ \mathcal{X}_{a,\mu}$ independently of the primitive, computes $T^* = \mathsf{H}_{\mathsf{TW128}}(X^*)$, gives T^* to $\mathcal{A}^{\pi,\pi^{-1}}$, and returns 1 iff $\mathcal{A}$ outputs a valid input X in the TW128 domain with $\mathsf{H}_{\mathsf{TW128}}(X) = T^*$; the advantage is $\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{TW128}}(\mathcal{A})$. The everywhere preimage game fixes a target tag $T^* \in \{0,1\}^{\tau_{\mathsf{root}}}$ before the primitive sampling and returns 1 iff $\mathcal{A}(T^*)$ outputs an input X in the domain with $\mathsf{H}_{\mathsf{TW128}}(X) = T^*$, the advantage $\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{TW128}}$ taken as the worst case over query-independent targets.

Committing games. The [BH22] committing games are instantiated as in Section 2.5, with no challenger-sampled key. The s-way CMTDs-ℓ (decryption) game has the adversary output a ciphertext $C \,\|\, T$ and s tuples (K_i, U_i, A_i, M_i) with pairwise distinct WiC_ℓ-images, and returns 1 iff $\mathsf{Dec}_{K_i}[\pi](U_i, A_i, C \,\|\, T) = M_i$ for every i, with the eager length rejection justified by Algorithm 3 ($\mathsf{Dec}_K(U, A, C \,\|\, T)$ returns $\perp$ or a message of length $\|C\|$); its advantage is $\mathbf{Adv}^{\mathsf{CMTDs}\text{-}\ell}_{\mathsf{TW128}}(\mathcal{A})$. The CMTs-$\ell$ (encryption) game has the adversary output the s tuples alone and returns 1 iff the encryptions $\mathsf{Enc}_{K_i}[\pi](U_i, A_i, M_i)$ are all equal; its advantage is $\mathbf{Adv}^{\mathsf{CMTs}\text{-}\ell}_{\mathsf{TW128}}(\mathcal{A})$.

5.2 Separating Final Tag Inputs

The tag value is read from a final root transcript input. The next two facts rule out structural collisions at that input, except for the explicitly charged possibility that two leaf tags collide before the root aggregates them.

Proposition 1 (Opening Triple Separation)**.** *If two TW128 root computations have distinct (K, U, A) triples, then their bridged final root transcript inputs under $\mathsf{flat}(\cdot)$ are distinct, regardless of whether each computation is an encryption or a decryption check, and whether or not they act on a common ciphertext.*

Proof. Consider two root computations i and j with contexts $(K_i, U_i, A_i) \neq (K_j, U_j, A_j)$, reaching final root transcripts R_i and R_j. Suppose for contradiction that $\mathsf{flat}(R_i) = \mathsf{flat}(R_j)$. Since the bridge $\mathsf{flat}(\cdot)$ is injective on well-formed transcript prefixes (Lemma 8), the native flattened inputs of the two final-root prefixes are equal. Then Lemma 6 recovers the full duplex-call sequence and reads off the pair $(\sigma, \sigma_{\mathsf{ph}})$ for each call. The seven 4-bit phase suffixes are pairwise distinct, so the σ_{init}, $\sigma^{\pm}_{\mathsf{ad}}$, $\sigma^{\pm}_{\mathsf{msg}}$, and (when present) $\sigma^{\pm}_{\mathsf{agg}}$ calls in the recovered sequence are unambiguously identified. The aggregation phase is present exactly when $n \geq 1$, and the two recovered sequences are equal, so they either both carry an aggregation phase or both omit it; either way the recovery of (K, U, A) below uses only the $\sigma_{\mathsf{init}}$and AD calls, which precede any message or aggregation block.

Equality of the recovered sequences forces equality of the σ_{init}call, whose σ is $p_{\mathsf{root}} \| K \| U$ with fixed-width K and U, hence $K_i = K_j$ and $U_i = U_j$. It also forces equality of the recovered AD subsequences. By the AD bullet of Lemma 6 the associated data phase contributes a σ^{+}_{ad}-tagged block exactly when the associated data is nonempty, so the recovered sequences agree on the presence of the AD phase. If exactly one of A_i, A_j were empty the recovered sequences would differ in the presence of a σ^{+}_{ad} call, contradicting their equality; hence either both are empty, giving $A_i = A_j = \varepsilon$, or both are nonempty, in which case the phase-recovery sub-claim of Lemma 6 inverts the ρ-byte-block partition map on the AD byte string and equal AD σ-block sequences imply $A_i = A_j$. In all cases $(K_i, U_i, A_i) = (K_j, U_j, A_j)$, contradicting the hypothesis of distinct triples. □

Triple Separation handles distinct contexts; when inputs share a context but differ in their messages, distinctness of the final root transcripts needs the absence of a leaf tag collision, recorded next.

Proposition 2 (Final-Root Input Separation)**.** *Let $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcripts receive the same τ_{leaf}-bit RF projection. On $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$, if two TW128 construction computations reach final root transcripts with $\mathsf{flat}(R) = \mathsf{flat}(R')$, then they share the same context (K, U, A) and the same full ciphertext C. Equivalently, on $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$ computations with distinct (K, U, A, C) reach distinct bridged final root transcripts.*

Proof. By bridge injectivity (Lemma 8), $\mathsf{flat}(R) = \mathsf{flat}(R')$ gives equal native flattened final-root inputs. Lemma 6 then recovers the same root initialization $p_{\mathsf{root}} \| K \| U$—hence the same K and U—the same associated data blocks—hence the same A—the same root ciphertext chunk, and the same leaf count and ordered leaf tag sequence. Under $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$, Leaf-Transcript Recovery (Lemma 7) identifies the equal leaf tag sequences with the same final leaf transcripts, hence the same leaf ciphertext bodies; with the common root ciphertext chunk this gives the same full C. □

5.3 Candidate Bound for Tag Inputs

After separation, the probabilistic question is local: how likely is a collision or target hit among the final root inputs whose tag values matter to a game? The candidate machinery rests on one engine, stated here and proved in full in Section A.5: adaptively chosen distinct random oracle inputs whose values are still unread when chosen behave, for collision and target-hitting purposes, as fresh uniform draws.

Lemma 2 (Adaptive Candidate Collision and Preimage)**.** *Consider any interaction with a lazy random oracle* $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ *that builds a sequence of distinct* candidate *inputs* $Y_1, \ldots, Y_m$*. The process may make arbitrary non-candidate oracle queries and may choose each candidate adaptively from the full prior transcript and private state sampled before the candidate's unread oracle value. Whenever a new candidate* Y_j *is appended, two premises hold:* predictability*—both the decision to append and the string* Y_j *are determined by the state preceding the append step—and* unrevealed value*—no oracle lookup before the append step has requested a positive number of bits of* $\mathsf{RO}_{\mathsf{flat}}(Y_j)$*; zero-length lookups at* Y_j *are permitted. Fix a projection length* τ*, and suppose the sequence has at most* L *candidates (*$m \leq L$*).*

1. Collision. *For every* $s \geq 2$*, the probability* $\Pr[\mathsf{coll}_{s,\tau}]$ *that some* s *distinct candidates have equal* τ*-bit projections,*

$$\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{i_1}) \right\rfloor_\tau = \cdots = \left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{i_s}) \right\rfloor_\tau \quad \text{for some } 1 \leq i_1 < \cdots < i_s \leq m,$$

satisfies $\Pr[\mathsf{coll}_{s,\tau}] \leq \binom{L}{s} \cdot 2^{-\tau(s-1)}$.

2. Preimage. *For every target* $t \in \{0,1\}^\tau$ *chosen independently of* $\mathsf{RO}_{\mathsf{flat}}$*, the probability* $\Pr[\mathsf{pre}_{\tau,t}]$ *that some candidate's* τ*-bit projection equals* t*, i.e.* $\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_j) \right\rfloor_\tau = t$ *for some* $1 \leq j \leq m$*, satisfies* $\Pr[\mathsf{pre}_{\tau,t}] \leq L \cdot 2^{-\tau}$.

The preimage case has a second-preimage *corollary: for a designated candidate position* j_0*, the probability* $\Pr[\mathsf{sec}_{\tau,j_0}]$ *that some candidate* $j \neq j_0$ *has* $\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_j) \right\rfloor_\tau = \left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{j_0}) \right\rfloor_\tau$*, i.e. collides with the designated candidate on its* τ*-bit projection, satisfies* $\Pr[\mathsf{sec}_{\tau,j_0}] \leq (L-1) \cdot 2^{-\tau}$.

Proof sketch. The premises license lazy sampling at the append step: predictability fixes the candidate string before its value is drawn, and the unrevealed-value premise states that no bit of that value has been exposed, so $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ is uniform and independent of the entire prior transcript. Conditioning position by position, each fixed position set then contributes at most $2^{-\tau(s-1)}$ (each fixed position at most $2^{-\tau}$), and a union bound over the $\binom{L}{s}$ position sets (the L positions) gives the bounds. The full argument, including the second-preimage corollary, appears in Section A.5. ☐

The remaining work is structural: exhibiting, in each game, a candidate sequence that satisfies these premises and contains the final root transcripts the win event constrains.

Lemma 3 (Root Tag Candidate Sequence)**.** *Consider one of the hash or committing random oracle games of Section 4.2. The adversary* $\mathcal{B}$ *makes direct* $\mathsf{RO}_{\mathsf{flat}}$ *queries and has no construction oracle. The only construction-internal positive-length lookups at root tag inputs are the root tag samplings of* $h \geq 1$ *deterministic challenger encryption or decryption checks at game-fixed points. Apart from the standard preimage game's designated sampled source check, all check inputs and keys are adversary supplied. In that preimage game the sampled source input, including its key, is private challenger state sampled before the source tag is revealed, and the source computation counts as one of the* h *checks. Let the* i*-th check reach a final root transcript* R_i *whose root tag is read by a single* $\mathsf{RO}_{\mathsf{flat}}$ *lookup at* $\mathsf{flat}(R_i)$*. Call an* $\mathsf{RO}_{\mathsf{flat}}$ *input a* root tag input *if it decodes under* $\mathsf{KeyedTWOut}$ *to a root tag output-point class—*$\mathcal{R}^+_{\mathsf{msg}}$ *(the final root message block, which carries the root tag only on the no-leaf path) or* $\mathcal{R}_{\mathsf{tag}}$ *(the aggregated root tag); multi-chunk computations also reach* $\mathcal{R}^+_{\mathsf{msg}}$ *with zero requested output before aggregation, so this predicate over-approximates the root tag candidate set, which only loosens the count. Every execution then admits a chronological candidate sequence in which each root tag input is appended exactly once, immediately before the first event that is either a direct* $\mathsf{RO}_{\mathsf{flat}}$ *query at it or a positive-length* $\mathsf{RO}_{\mathsf{flat}}$ *lookup reading it, and inputs never reached by such an event are never appended. This sequence is such that*

1. *the candidates are pairwise distinct, and if the challenger reaches its checks, each* $\mathsf{flat}(R_i)$ *is in the sequence no later than the sampling of the* i*-th root tag answer;*

2. *it satisfies the predictability and unrevealed-value premises of Lemma 2; and*

3. *its length is at most* $q_{\mathsf{RO}}(\mathcal{B}) + h$.

Proof. A direct query to a keyed transcript is one of $\mathcal{B}$'s visible $\mathsf{RO}_{\mathsf{flat}}$ queries counted in $q_{\mathsf{RO}}(\mathcal{B})$, not a separate construction query. The only construction computations in the game are the h deterministic challenger checks named in the statement.

Distinctness and coverage. Each root tag input is appended at its first triggering event and never again, so the candidates are pairwise distinct by construction, and the rule is well defined however direct queries and challenger checks interleave. In particular, in the standard preimage game the source check's root tag sampling may be the trigger for $\mathsf{flat}(R_1)$; a later direct query at the same address is not a first trigger and appends nothing. Each $\mathsf{flat}(R_i)$ is appended no later than the sampling of the i-th root tag answer, since that sampling is a positive-length lookup at $\mathsf{flat}(R_i)$, giving (1).

Predictability. The decoded root-tag-input predicate depends on the input string alone through the exact decoder of Definition 4, which ignores the requested output length; by Output-Point Separation (Corollary 8) each input decodes to at most one output-point class. The append rule also uses the already fixed next event: a direct query triggers at the address, while a construction lookup triggers only when it requests positive output length. The string itself is fixed in the state preceding the trigger: a direct query's input is chosen by $\mathcal{B}$ before its lookup, and a check's final root transcript is determined by the check's input and the prior oracle answers—including any private challenger state sampled before the candidate's unread value, which Lemma 2 expressly allows.

Unrevealed value. The trigger is the first direct query or positive-length lookup at the address, so no earlier direct query at it exists, and it suffices that no earlier construction lookup has read a positive number of its bits. An earlier positive-length construction lookup is either a stream output point, a leaf-tag point $\mathcal{L}_{\mathsf{tag}}(j)$, or a root-tag sampling. The first two are different output-point classes and hence, by Corollary 8, have different addresses from root tag inputs; the last would itself have been the first trigger at this address. Zero-requested-output lookups, including the $\mathcal{R}^{+}_{\mathsf{msg}}$ lookup a multi-chunk computation makes before aggregation, are permitted by the unrevealed-value premise of Lemma 2, including when such a zero-length lookup lands at another check's future no-leaf final-root address. Together with predictability this gives (2).

Length. Appends are triggered only by direct queries—at most $q_{\mathsf{RO}}(\mathcal{B})$ of them—or by the h checks' root tag samplings; zero-length construction lookups never trigger an append. Hence the sequence has length at most $q_{\mathsf{RO}}(\mathcal{B}) + h$, giving (3). ☐

Lemma 4 (Root Tag Hash and Target-Hit Bound)**.** *Consider one of the hash or committing* TW128 *random oracle games (Section 4.2) for an adversary* $\mathcal{B}$*, satisfying the hypotheses of Lemma 3, with win event* W*. By the Root Tag Candidate Sequence lemma (Lemma 3) the root tag inputs form a chronological candidate sequence meeting the premises of Lemma 2 with* $\tau = \tau_{\mathsf{root}}$*. Writing* $q_{\mathsf{RO}} = q_{\mathsf{RO}}(\mathcal{B})$*:*

1. Root-collision case. *If* W *forces* s *checks to reach pairwise distinct bridged final root transcripts carrying a common root tag, then, with* $h = s$ *checks,* $\Pr[\mathsf{W}] \leq \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}, s)$*;*

2. Fixed-target case. *If* W *forces a single check (*$h = 1$*) to carry a root tag equal to a target fixed independently of* $\mathsf{RO}_{\mathsf{flat}}$*, then* $\Pr[\mathsf{W}] \leq \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}})$*;*

3. Sampled-source case. *If the game first runs a designated source check, reveals its root tag, and* W *forces one later check to reach a distinct bridged final root transcript carrying that designated tag, then, with* $h = 2$ *checks,* $\Pr[\mathsf{W}] \leq \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}})$.

Proof. In each case the candidate sequence has the stated length and meets the premises of Lemma 2. *(1)* The s distinct candidates carrying a common τ_{root}-projection are an s-way collision; the collision case with $L = q_{\mathsf{RO}} + s$ gives $\binom{q_{\mathsf{RO}}+s}{s} \cdot 2^{-\tau_{\mathsf{root}}(s-1)} = \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}, s)$. *(2)* The candidate hitting the fixed target is a preimage; the preimage case with $L = q_{\mathsf{RO}} + 1$ gives $(q_{\mathsf{RO}} + 1) \cdot 2^{-\tau_{\mathsf{root}}} = \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}})$. *(3)* Let j_{src} be the position of the designated source check. The later check is required to be a distinct bridged final root transcript, so it is a candidate position $j \neq j_{\mathsf{src}}$ with the same τ_{root}-bit projection. This is an application of the second-preimage corollary, not the fixed-target case, because the designated source tag is sampled from $\mathsf{RO}_{\mathsf{flat}}$ rather than fixed independently of it. The candidate sequence has length at most $L = q_{\mathsf{RO}} + 2$, and the second-preimage corollary of Lemma 2 gives $(L-1)2^{-\tau_{\mathsf{root}}} = (q_{\mathsf{RO}} + 1)2^{-\tau_{\mathsf{root}}} = \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}})$. □

5.4 Random Oracle Tag-Hash Bounds

Each random oracle bound now matches a hash game to a case of Lemma 4. Collision and fixed-slice preimage resistance also carry a leaf-collision term, because distinct openings may have different ciphertext bodies before reaching the root.

Theorem 1 (Random Oracle Collision Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every $s \geq 2$ and every s-way collision adversary $\mathcal{B}$ in the TW128 random oracle model,*

$$\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s).$$

Proof. Run the random oracle collision game. Let $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcripts receive the same τ_{leaf}-bit RF projection; by the general count of Lemma 11, $\Pr[\mathsf{bad}^{\star}_{\mathsf{leaf}}] \leq \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B}))$, so it remains to bound the win under $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$. After rejecting unless the s output inputs are pairwise distinct and in the $\mathsf{TW128}$ domain, the challenger performs s deterministic encryptions $\mathsf{Enc}^{\mathsf{RO}}_{K_i}(U_i, A_i, M_i)$, reaching $R_1, \ldots, R_s$, all carrying a common root tag T on a win. Pairwise distinct tuples (K_i, U_i, A_i, M_i) have pairwise distinct (K_i, U_i, A_i, C_i): two tuples differing in their context already differ in (K, U, A), while two sharing a context but differing in the message yield distinct ciphertexts, since for a fixed context encryption maps the message to the ciphertext bijectively. So on $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$ Final-Root Input Separation (Proposition 2) makes $\mathsf{flat}(R_1), \ldots, \mathsf{flat}(R_s)$ pairwise distinct. The Root Tag Hash and Target-Hit Bound (Lemma 4, item 1) bounds the win under $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$ by $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s)$; adding $\Pr[\mathsf{bad}^{\star}_{\mathsf{leaf}}]$ gives the theorem. □

Theorem 2 (Random Oracle Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *Fix byte lengths $0 \leq a \leq A_{\max}$ and $0 \leq \mu \leq M_{\max}$, and let $m_{a,\mu} = k + \nu + 8a + 8\mu$. For every Pre adversary $\mathcal{B}$ against $\mathsf{H}_{\mathsf{TW128}}$ on the fixed input slice $\mathcal{X}_{a,\mu}$ in the TW128 random oracle model (Section 4.2),*

$$\begin{aligned}\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; &\mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})) \\ &+ \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B})) + 2^{\tau_{\mathsf{root}} - m_{a,\mu}}.\end{aligned}$$

Proof. Run the random oracle Pre game. The challenger samples $X^* \leftarrow\$ \mathcal{X}_{a,\mu}$, writes $X^* = (K^*, U^*, A^*, M^*)$, computes $T^* = \mathsf{H}_{\mathsf{TW128}}(X^*)$, gives T^* to $\mathcal{B}$, and later checks the output $X = (K, U, A, M)$. Let same be the event $X = X^*$. For a fixed random oracle table and fixed adversary coins, the adversary's output is a function of the τ_{root}-bit challenge tag T^*, so over the uniform source X^* it can equal X^* with probability at most $2^{\tau_{\mathsf{root}}}/2^{m_{a,\mu}} = 2^{\tau_{\mathsf{root}} - m_{a,\mu}}$. Hence $\Pr[\mathsf{same}] \leq 2^{\tau_{\mathsf{root}} - m_{a,\mu}}$.

It remains to bound winning executions with $X \neq X^*$. Let $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcripts in the source and output checks receive the same τ_{leaf}-bit RF projection. By the general count of Lemma 11—which charges every direct query and every construction-generated final leaf transcript regardless of key provenance, covering the challenger-sampled source key—

$$\Pr[\mathsf{bad}^{\star}_{\mathsf{leaf}}] \le \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})).$$

On $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$, the distinct tuples X^* and X have distinct final root transcripts: distinct contexts are separated by Proposition 1, while equal contexts and distinct messages produce distinct ciphertexts, and then Proposition 2 applies. A win with $X \neq X^*$ therefore forces the later output check to hit the designated source tag T^* at a distinct bridged final root transcript. The sampled-source case, item 3, of Lemma 4 bounds this probability by $\mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}))$. Adding the source-guess and leaf-collision events gives the theorem. ☐

Theorem 3 (Random Oracle Everywhere Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every query-independent target tag $T^* \in \{0,1\}^{\tau_{\mathsf{root}}}$ and every ePre adversary $\mathcal{B}$ in the $\mathsf{TW128}$ random oracle model (Section 4.2),*

$$\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{RO}}(\mathcal{B}) \;\le\; \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B})) = \frac{q_{\mathsf{RO}}(\mathcal{B}) + 1}{2^{\tau_{\mathsf{root}}}}.$$

Proof. Run the random oracle preimage game for the query-independent target T^*, which is independent of $\mathsf{RO}_{\mathsf{flat}}$. After rejecting unless the output input $X = (K, U, A, M)$ is in the $\mathsf{TW128}$ domain, the challenger performs one encryption $\mathsf{Enc}^{\mathsf{RO}}_K(U, A, M)$, reaching a final root transcript R. A win means $\mathsf{H}_{\mathsf{TW128}}(X) = T^*$, i.e. the root tag $\lfloor \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(R)) \rfloor_{\tau_{\mathsf{root}}}$ at R equals the fixed target T^*. The fixed-target case, item 2, of Lemma 4 gives $\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{RO}}(\mathcal{B}) \le \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}))$. ☐

5.5 Public Permutation Tag-Hash Bounds

These are the public-permutation hash claims for the authentication tag. The flat sponge lift transfers the preceding random oracle bounds to the public permutation model. For these one-world hash games, the lift contributes $\mathsf{Lift}_c(N_{\mathsf{tot}})$, and the derived random oracle adversary has $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$; challenger construction computations—including the preimage game's source encryption—are construction-internal work counted in N.

Theorem 4 (Collision Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every $s \ge 2$ and every adversary $\mathcal{A}$ against the s-way multi-collision resistance of $\mathsf{H}_{\mathsf{TW128}}$ in the sense of [BH22], with the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{Coll}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s).$$

Proof. By the Lift Application corollary (Corollary 9) for the collision game, the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$ and $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$. The random oracle bound of Theorem 1 is nondecreasing in both $q_{\mathsf{RO}}(\mathcal{B})$ and $n_{\mathsf{leaf}}(\mathcal{B})$, so the monotone form of Corollary 9 gives the stated bound. The s colliding inputs may differ in their messages, hence in their ciphertext bodies; the $\mathsf{LeafColl}$ term accounts for the only way two distinct inputs sharing a context (K, U, A) can reach the same final root transcript—a collision among the leaf tags it absorbs—while distinct contexts already reach distinct final root transcripts by Opening Triple Separation (Proposition 1). ☐

Corollary 1 (Two-Way Collision Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *Specializing Theorem 4 to $s = 2$ gives the classical two-way collision resistance of [RS04]: for every collision adversary $\mathcal{A}$ against $\mathsf{H}_{\mathsf{TW128}}$ with the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{Coll}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, 2),$$

with $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, 2) = \binom{N_{\mathsf{prim}}+2}{2}/2^{\tau_{\mathsf{root}}}$.

Proof. The $s = 2$ case of Theorem 4; [BH22] note that $s = 2$ recovers the classical notion of collision resistance, which under the keyed-by-the-permutation convention of Section 2.5 is the Coll notion of [RS04]. $\square$

Corollary 2 (Second-Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *Fix byte lengths* $0 \le a \le A_{\max}$ *and* $0 \le \mu \le M_{\max}$. *For every adversary against* $\mathsf{H}_{\mathsf{TW128}}$*, with the resources of Section 4.3, in either the standard second-preimage (Sec) or everywhere second-preimage (eSec) sense of [RS04] on the fixed input slice* $\mathcal{X}_{a,\mu}$*, the corresponding advantage is bounded, independently of* (a, μ)*, by the two-way collision bound of Corollary 1:*

$$\mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, 2).$$

Proof. We use [RS04] to identify the standard and everywhere second-preimage notions with the fixed-slice notions $\mathrm{Sec}[m_{a,\mu}]$ and $\mathrm{eSec}[m_{a,\mu}]$ under the canonical encoding of $\mathcal{X}_{a,\mu}$; the quantitative reduction is explicit.

For a standard Sec adversary $\mathcal{A}$, define a two-way collision adversary $\mathcal{C}$ as follows. It samples $X_0 \leftarrow\$\ \mathcal{X}_{a,\mu}$, runs $\mathcal{A}^{\pi,\pi^{-1}}(X_0)$, receives X_1, and outputs (X_0, X_1). Whenever $\mathcal{A}$ wins, X_1 is a valid input, $X_1 \neq X_0$, and $\mathsf{H}_{\mathsf{TW128}}(X_1) = \mathsf{H}_{\mathsf{TW128}}(X_0)$, so $\mathcal{C}$ wins the two-way collision game with exactly the same probability.

For eSec, fix a source $X_0 \in \mathcal{X}_{a,\mu}$ and define $\mathcal{C}_{X_0}$ to run the eSec adversary on that source and output (X_0, X_1). The same pointwise implication holds for every X_0; taking the maximum over sources gives the eSec bound.

The induced collision adversary makes exactly the direct primitive queries made by $\mathcal{A}$. Its two deterministic challenger checks are the source and candidate checks, counted exactly as in the $s = 2$ collision game. Applying Corollary 1 to the induced collision adversary gives the displayed bound. $\square$

Theorem 5 (Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *Fix byte lengths* $0 \le a \le A_{\max}$ *and* $0 \le \mu \le M_{\max}$*, and let* $m_{a,\mu} = k + \nu + 8a + 8\mu$. *For every adversary* $\mathcal{A}$ *against* $\mathsf{H}_{\mathsf{TW128}}$*, with the resources of Section 4.3, in the standard preimage (Pre) sense of [RS04] on the fixed input slice* $\mathcal{X}_{a,\mu}$*,*

$$\begin{aligned}\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathit{TW128}}(\mathcal{A}) \ &\le \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) \\ &\quad + \mathsf{RootPre}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}) + 2^{\tau_{\mathsf{root}} - m_{a,\mu}}.\end{aligned}$$

Proof. By the Lift Application corollary (Corollary 9) for the preimage game, the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$ and $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$; per Section 4.3, these caps cover both challenger encryptions' final leaf transcripts, the source encryption's included; the encryptions' duplex calls are counted in N and enter through $\mathsf{Lift}_c(N_{\mathsf{tot}})$. The random oracle bound of Theorem 2 is nondecreasing in both resources, so the monotone form of Corollary 9 gives the stated bound. $\square$

Theorem 6 (Everywhere Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every target tag* $T^* \in \{0,1\}^{\tau_{\mathsf{root}}}$ *fixed in advance and every ePre adversary* $\mathcal{A}$ *against* $\mathsf{H}_{\mathsf{TW128}}$ *with the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{ePre}}_{\mathit{TW128}}(\mathcal{A}) \ \le\ \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootPre}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}) \ =\ \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{N_{\mathsf{prim}} + 1}{2^{\tau_{\mathsf{root}}}}.$$

Proof. By the Lift Application corollary (Corollary 9) for the everywhere-preimage game—whose target T^* is fixed before the primitive sampling—the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$, and the random oracle bound of Theorem 3 is nondecreasing in $q_{\mathsf{RO}}(\mathcal{B})$, so the monotone form of Corollary 9 gives the stated bound. Compared with the s-way root-collision term $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(\cdot, s)$ of Section 4.4, the single fixed target replaces the exponent $\tau_{\mathsf{root}}(s-1)$ by τ_{root}. $\square$

Corollary 3 (Tag Hash Security and Compact Commitment)**.** *The TW128 authentication tag, represented by the wrapper $\mathsf{H}_{\mathsf{TW128}}$, is a secure hash of the full encryption context (K, U, A, M) in the sense of [RS04]: collision-resistant (Theorem 4), preimage-resistant on each fixed input slice (Theorem 5), second-preimage-resistant (Corollary 2), and everywhere preimage-resistant (Theorem 6). Under the implemented parameters and work cap $N_{\mathsf{tot}} \leq 2^{63}$, these advantages are at most 2^{-128} (Corollary 7). These hash bounds make the single τ_{root}-bit tag a binding compact commitment to (K, U, A, M), with standard and everywhere preimage resistance, native to the mode. The commitment is to an opening supplied for the tag: it includes the key and does not make the tag an independently verifiable public digest of (U, A, M).*

Proof. Collision resistance is Theorem 4; standard preimage resistance on fixed input slices is Theorem 5; second-preimage resistance follows from collision resistance by Corollary 2; and everywhere preimage resistance is Theorem 6. These bounds quantify the adversary's success at the same τ_{root}-bit projection of the final root transcript, and Corollary 7 evaluates them at the implemented parameters. Binding is the compact-commitment property supplied by collision and second-preimage resistance. Standard preimage resistance is the property that a tag produced from a uniformly sampled fixed-length opening does not reveal an efficiently findable opening; everywhere preimage resistance is the corresponding fixed-target property for tags chosen independently of the primitive. □

5.6 Committing Consequences of Tag Hash Security

The committing consequences use the same tag-collision analysis under the opening constraints of the [BH22] games. CMTDs-4 is slightly tighter than general collision resistance: all openings share one ciphertext body, so distinct openings separate through their opening triples and no leaf-collision term is needed.

Theorem 7 (Random Oracle CMTDs-4)**.** *For every $s \geq 2$ and every s-way CMTDs-4 adversary $\mathcal{B}$ in the TW128 random oracle model (Section 4.2),*

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s) = \frac{\binom{q_{\mathsf{RO}}(\mathcal{B})+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. Run the random oracle CMTDs-4 game. After rejecting on any failed syntax, message-length, or pairwise-full-input check, the challenger performs s deterministic decryption checks $\mathsf{Dec}^{\mathsf{RO}}_{K_i}(U_i, A_i, C \parallel T)$ on the one common ciphertext $C \parallel T$, reaching $R_1, \ldots, R_s$. On a winning execution the tuples (K_i, U_i, A_i, M_i) are pairwise distinct, and since all checks decrypt the common ciphertext, determinism of Dec makes the triples (K_i, U_i, A_i) pairwise distinct—equal triples would return equal messages, hence equal tuples. By Opening Triple Separation (Proposition 1) the candidates $\mathsf{flat}(R_1), \ldots, \mathsf{flat}(R_s)$ are pairwise distinct (leaf tag collisions cannot merge them), and every accepting check carries the common root tag T. The Root Tag Hash and Target-Hit Bound (Lemma 4, item 1) gives $\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B}) \leq \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s)$. □

Corollary 4 (CMTDs-4 Committing Security)**.** *For every $s \geq 2$ and every s-way CMTDs-4 adversary $\mathcal{A}$ against TW128 with the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s) \;=\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. By the Lift Application corollary (Corollary 9) for the CMTDs-4 game, the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$, and the random oracle bound of Theorem 7 is nondecreasing in $q_{\mathsf{RO}}(\mathcal{B})$, so the monotone form of Corollary 9 gives the stated bound. The challenger's s decryption checks are construction computations whose duplex calls are counted in N and enter through $\mathsf{Lift}_c(N_{\mathsf{tot}})$; their s final-root lookups are the candidate-sequence slack of $\mathsf{RootColl}_{\tau_{\mathsf{root}}}$ (Section 4.4). □

Remaining committing notions. The hierarchy and s-way extension of the [BH22] committing notions are given in [BH22, §3, Figs. 4–5, and App. A]. We use those relations for the D-notions and a direct source-game argument for the CMTs-ℓ notions to preserve the resource split of Section 4.3.

Corollary 5 (Committing-Notion Root-Collision Bound)**.** *For every $s \geq 2$, every*

$$X \in \{\mathsf{CMTDs\text{-}1},\ \mathsf{CMTDs\text{-}3},\ \mathsf{CMTDs\text{-}4},\ \mathsf{CMTs\text{-}1},\ \mathsf{CMTs\text{-}3},\ \mathsf{CMTs\text{-}4}\},$$

and every s-way X-adversary $\mathcal{A}$ against TW128 with the resources of Section 4.3,

$$\mathbf{Adv}^{X}_{\mathit{TW128}}(\mathcal{A}) \ \leq\ \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s) \ =\ \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. CMTDs-4 is bounded directly by Corollary 4. For the remaining D-notions, the s-way hierarchy of [BH22, §3, Fig. 5 and App. A] gives, for $\ell \in \{1, 3\}$, $\mathbf{Adv}^{\mathsf{CMTDs}\text{-}\ell}_{\mathsf{TW128}}(\mathcal{A}) \leq \mathbf{Adv}^{\mathsf{CMTDs}\text{-}4}_{\mathsf{TW128}}(\mathcal{A})$ for the same output distribution. By the conservative committing-game accounting of Section 4.3, the CMTDs-ℓ and CMTDs-4 resource caps charge the same deterministic decryption checks after syntax, domain, and message-length validation, independently of their distinctness gates.

For CMTs-ℓ, the challenger performs s deterministic encryption checks, which supply the candidate final-root transcripts (Lemma 3 builds the candidate sequence for encryption and decryption checks alike). On a winning execution all s encryptions are equal, hence of equal length and with one common root tag T. Pairwise WiC_1-distinctness gives pairwise distinct keys, pairwise WiC_3-distinctness gives pairwise distinct (K, U, A) triples, and pairwise WiC_4-distinctness gives pairwise distinct full inputs; in the last case, equal triples and equal ciphertexts would decrypt deterministically to equal messages, contradicting full-input distinctness. Thus each CMTs-ℓ winner reaches pairwise distinct opening triples. Opening Triple Separation (Proposition 1) makes the s final-root candidates distinct while their τ_{root}-bit projections all equal T. The CMTs-ℓ encryption checks are the construction-internal work counted in N by Section 4.3.

By the Lift Application corollary (Corollary 9) for the CMTs-ℓ game, the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$. The Root Tag Hash and Target-Hit Bound (Lemma 4, item 1) then bounds its win by $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s)$ in the random oracle model, which the lift transfers to the public permutation setting at N_{prim}; the encryption checks are construction-internal work counted in N, not in N_{prim}. □

6 Baseline Authenticated Encryption Bounds

Section 5 proves the tag-centric properties of TW128. Here we discharge the AEAD-facing obligations: multi-user indistinguishability, its single-user all-in-one AE corollary, and INT-RUP integrity. These bounds use the same hidden-key, leaf-collision, and tag-guessing accounting as the tag analysis, but their role is narrower: they show that the mode remains a conventional nonce-respecting AEAD.

We first prove mu-ind in the flat random oracle model, lift the result to the public permutation model using Section B, and derive all-in-one AE as the $D = 1$ case. We then prove INT-RUP in the separated-interface model of [ABL+14], where plaintext-computing queries increase the construction work counts but do not change the non-replay verification count. All concrete instantiations are summarized in Section 7.

6.1 AEAD Public Permutation Games

The public-permutation games in this section use the primitive, validity, and construction-oracle conventions of Section 4.1. The random oracle versions are obtained by the mechanical substitution of Section 4.2.

All-in-one AE. The all-in-one authenticated encryption game [RS06] samples $K \leftarrow\$ \{0,1\}^k$ and compares the real pair $(\mathsf{Enc}_K[\pi], \mathsf{Ver}_K[\pi])$ with the ideal pair $(\mathsf{Rand}, \mathsf{Replay})$. On each accepted encryption query (U, A, M), Rand samples a uniformly random byte string of length $\|M\| + \tau_{\mathsf{root}}/8$, parses it as $C \,\|\, T$ with $T \in \{0,1\}^{\tau_{\mathsf{root}}}$, records (U, A, C, T), and returns $C \,\|\, T$; Replay accepts exactly recorded tuples. Encryption queries are nonce-respecting, and

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}(\mathcal{A}) = \left|\Pr[\mathcal{A}^{\mathsf{Enc}_K[\pi],\, \mathsf{Ver}_K[\pi],\, \pi,\, \pi^{-1}} \Rightarrow 1] - \Pr[\mathcal{A}^{\mathsf{Rand},\, \mathsf{Replay},\, \pi,\, \pi^{-1}} \Rightarrow 1]\right|.$$

This all-in-one AE advantage is the single-user case of the multi-user real-vs-ideal notion of Section 2.7; Corollary 6 derives it from Theorem 9.

RUP integrity. The INT-RUP game samples $K \leftarrow\$ \{0,1\}^k$ and $\pi \leftarrow\$ \mathsf{Perm}(\{0,1\}^{1600})$, exposes $\mathsf{Enc}_K[\pi]$, $\mathsf{PDec}_K[\pi]$, $\mathsf{Ver}_K[\pi]$, and (π, π^{-1}), and maintains the replay table W of encryption outputs. Accepted encryption queries are nonce-respecting: an encryption query is rejected if its nonce has appeared in a previous accepted encryption query. Plaintext-computing and verification queries may use arbitrary nonces. On an accepted encryption query (U, A, M) the game returns $\mathsf{Enc}_K[\pi](U, A, M)$, parsed and recorded as (U, A, C, T) in W. On a valid plaintext-computing query (U, A, C, T) it returns $\mathsf{PDec}_K[\pi](U, A, C, T)$, and on an invalid query it returns $\perp$. On a valid verification query (U, A, C, T) it returns $\mathsf{Ver}_K[\pi](U, A, C, T)$; if this bit is 1 and $(U, A, C, T) \notin W$ when the query is asked, the game records a forgery. Invalid verification queries return 0. The game returns 1 iff a forgery is recorded, and

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) = \Pr[\mathsf{G}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) \Rightarrow 1].$$

6.2 Multi-User Setting

This subsection instantiates the mu-ind game of [BT16] (Section 2.7) for TW128 in the public permutation model, under the primitive convention of Section 4.1. The challenger samples a challenge bit $b \leftarrow\$ \{0,1\}$ and a permutation $\pi \leftarrow\$ \mathsf{Perm}(\{0,1\}^{1600})$, and exposes (π, π^{-1}) as the primitive interface in both worlds. The adversary $\mathcal{A}$ creates users on demand via New, and the game maintains an active-user counter v. We write D for an execution-uniform cap on the number of New calls. Equivalently, the game may sample independent keys $K_1, \ldots, K_D$ at the start of the experiment and let the d-th New call activate K_d; inactive indices are never valid users.

The mu-ind construction oracles inherit the validity convention of Section 4.1. The encryption oracle on input (d, U, A, M) rejects, without recording, if (U, A, M) is outside the TW128 encryption domain, if $d \notin \{1, \ldots, v\}$, or if $(d, U) \in V$; otherwise it sets $C = \mathsf{Enc}_{K_d}[\pi](U, A, M)$ when $b = 1$, and samples a uniformly random byte string C of length $\|M\| + \tau_{\mathsf{root}}/8$ when $b = 0$. The game records (d, U) in V and (d, U, A, C) in W, and returns C. Here C is the full ciphertext string: the message ciphertext concatenated with the root tag, written $C \,\|\, T$ in the all-in-one AE game of Section 6.1. The verification oracle on input (d, U, A, C) rejects if $d \notin \{1, \ldots, v\}$ or if (U, A, C) is not a valid TW128 verification/decryption input; if $(d, U, A, C) \in W$ it returns true; if $b = 0$ it returns false; otherwise it returns $[\mathsf{Dec}_{K_d}[\pi](U, A, C) \neq \perp]$. Nonce uniqueness is enforced per user by the set V.

After querying these oracles, $\mathcal{A}$ outputs $b' \in \{0,1\}$. The mu-ind advantage of $\mathcal{A}$ against TW128 is

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) = |2\Pr[b' = b] - 1|.$$

A valid verification query is *non-replay* if its tuple is not in W when the query is asked; replay hits are answered by the recorded transcript and are not counted in the verification-query resource below.

Resource caps split per potential user $d \le D$: $N^{(d)}$, $q_{\mathsf{v}}^{(d)}$, $n_{\mathsf{leaf}}^{(d)}$ are the per-user counts under Section 4.3, with inactive users contributing zero. The global totals $N = \sum_{d=1}^{D} N^{(d)}$, $q_{\mathsf{v}} = \sum_{d=1}^{D} q_{\mathsf{v}}^{(d)}$, $n_{\mathsf{leaf}} = \sum_{d=1}^{D} n_{\mathsf{leaf}}^{(d)}$ enter the final bound through their execution-uniform caps: N through the lift term $\mathsf{Lift}_c(N_{\mathsf{tot}})$, and n_{leaf} and q_{v} through the leaf-collision and verification-tag terms at N_{leaf} and Q_{v}. N_{prim} remains a single shared count over the primitive interface.

The mu-ind treatment applies only to indistinguishability. The committing security games already give the adversary control of all s keys, so the s-way CMTDs-ℓ statements of Section 5.1 are themselves the multi-key committing statement and need no separate $\mathsf{mu\text{-}cmt}$ notion.

All random oracle adversaries in this section are flat-RO adversaries (Section 4.2) with the displayed resources, and the flat sponge lift transfers each random oracle bound using the derived-adversary cap $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$.

Theorem 8 (Random Oracle Multi-User Indistinguishability)**.** *For every flat-RO mu-ind adversary $\mathcal{B}$ with execution-uniform user cap D and resource counts of Section 6.2,*

$$\mathbf{Adv}_{\mathsf{RO}}^{\mathsf{mu\text{-}ind}}(\mathcal{B}) \;\le\; \frac{D(D-1)}{2^{k+1}} + D \cdot \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}} + \frac{q_{\mathsf{v}}(\mathcal{B})}{2^{\tau_{\mathsf{root}}}}.$$

Here q_{v} is the non-replay verification-query count of Section 4.3.

Proof. Use the pre-sampled-key formulation of Section 6.2: sample keys $K_1, \ldots, K_D$ at the start of the experiment, and let the d-th New call activate K_d, unused keys being ignored. Let coll denote the event that two of the pre-sampled keys $K_1, \ldots, K_D$ collide; $\Pr[\mathsf{coll}] \le \binom{D}{2}/2^k = D(D-1)/2^{k+1}$ by union bound. In the random oracle model, use a standard hybrid over the D users: H_d answers users $1, \ldots, d$ with $(\mathsf{Rand}, \mathsf{Replay})$ and users $d+1, \ldots, D$ with the real construction interface, so H_0 is the real mu-ind game and H_D is the ideal one. Let $H_d^\star$ be H_d with a fixed output bit on coll. Since coll has the same probability in all hybrids,

$$\mathbf{Adv}_{\mathsf{RO}}^{\mathsf{mu\text{-}ind}}(\mathcal{B}) \le \Pr[\mathsf{coll}] + \left|\Pr[\mathcal{B}^{H_0^\star} \Rightarrow 1] - \Pr[\mathcal{B}^{H_D^\star} \Rightarrow 1]\right|.$$

By the triangle inequality, it remains to bound each adjacent stopped hybrid. For each d, Lemma 17 gives

$$\left|\Pr[\mathcal{B}^{H_{d-1}^\star} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^\star} \Rightarrow 1]\right| \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n_{\mathsf{leaf}}^{(d)}}{2^{\tau_{\mathsf{leaf}}}} + \frac{q_{\mathsf{v}}^{(d)}}{2^{\tau_{\mathsf{root}}}}.$$

Lemma 17 is the only construction-specific step in the baseline mu-ind proof. It couples the adjacent user-d hybrids through the all-reject continuation of Definition 5. The coupling agrees until a direct random-oracle query tests the hidden key K_d or a non-replay verification comparison succeeds. The resulting first-occurrence decomposition has exactly the three terms displayed above: the foreign-key test is bounded by $\mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B}))$; a same-slot collision between final leaf tags for user d is bounded by $n_{\mathsf{leaf}}^{(d)}/2^{\tau_{\mathsf{leaf}}}$ via Lemma 11; and a successful verification-first root comparison is bounded by $q_{\mathsf{v}}^{(d)}/2^{\tau_{\mathsf{root}}}$ via Lemma 15. Encryption transcript marginal uniformity and environment neutrality (Lemmas 12 and 13) are the facts that make this coupling independent of the other users.

Summing across $d = 1, \ldots, D$ gives $D\mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B}))$ for the key-test contribution. The leaf tag contribution sums exactly:

$$\sum_d \frac{n_{\mathsf{leaf}}^{(d)}}{2^{\tau_{\mathsf{leaf}}}} = \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}},$$

since $\sum_d n_{\mathsf{leaf}}^{(d)} = n_{\mathsf{leaf}}(\mathcal{B})$. The verification-tag contribution satisfies

$$\sum_d q_{\mathsf{v}}^{(d)}/2^{\tau_{\mathsf{root}}} = q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Combining these estimates with the coll penalty gives the stated RO bound. $\square$

Theorem 9 (Multi-User Indistinguishability)**.** *For every mu-ind adversary $\mathcal{A}$ against* TW128 *with execution-uniform user cap D and the global resources of Section 6.2,*

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{Lift}_c(N_{\mathsf{prim}}) + \frac{D(D-1)}{2^{k+1}} + D\cdot\mathsf{KeyGuess}_k(N_{\mathsf{prim}}) + \frac{N_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. By the Lift Application corollary (Corollary 9) for the mu-ind game—a real-vs-ideal game, so the ideal-side proximity term applies—there is a random oracle mu-ind adversary $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ with $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$, $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$, $q_{\mathsf{v}}(\mathcal{B}) \le Q_{\mathsf{v}}$, and the per-user construction-side caps of Section 6.2 such that

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}) + \mathsf{Lift}_c(N_{\mathsf{prim}}).$$

Theorem 8 bounds the random oracle term. Substituting $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$, $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$, and $q_{\mathsf{v}}(\mathcal{B}) \le Q_{\mathsf{v}}$ gives the stated bound. $\square$

Corollary 6 (All-in-One AE Security)**.** *For every nonce-respecting AE adversary $\mathcal{A}$ against* TW128 *with the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{Lift}_c(N_{\mathsf{prim}}) + \mathsf{KeyGuess}_k(N_{\mathsf{prim}}) + \frac{N_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. Build a mu-ind adversary $\mathcal{A}^{\mu}$ that makes one New query, runs $\mathcal{A}$, and forwards each encryption and verification query of $\mathcal{A}$ to user 1. The mu-ind oracles use the combined ciphertext $C \,\|\, T$ while the AE oracles split it into (C, T), so $\mathcal{A}^{\mu}$ concatenates on the way in and re-splits on the way out; this repackaging is a bijection and changes no probability. The primitive interface is forwarded unchanged. The real and ideal views of $\mathcal{A}$ in the all-in-one AE experiment are exactly the real and ideal views of $\mathcal{A}^{\mu}$ in the mu-ind experiment with $D = 1$, and the resource counts are unchanged. On a replayed encryption tuple the two real verification oracles agree: the mu-ind oracle short-circuits through its replay table, while the AE oracle re-runs decryption, which accepts by TW128 decryption correctness (Section 3.5); all other queries are answered by identical code. Applying Theorem 9 with $D = 1$ eliminates the key-collision term and gives the stated bound. $\square$

The $\mathsf{INT\text{-}RUP}$ bound is AEAD-facing, but it is closer to the tag-hash analysis than to the mu-ind baseline: plaintext-computing queries expose unverified stream plaintext, and freshness must still reduce to hidden key tests, leaf-tag collisions, and root-tag guessing. The separated interface changes the accounting by adding plaintext-computing work to N and n_{leaf}, while leaving the non-replay verification count q_{v} unchanged.

Theorem 10 (Random Oracle $\mathsf{INT\text{-}RUP}$ Integrity)**.** *For every flat-RO $\mathsf{INT\text{-}RUP}$ adversary $\mathcal{B}$ against* TW128 *with nonce-respecting encryption queries and the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{RO}}(\mathcal{B}) \;\le\; \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}} + \frac{q_{\mathsf{v}}(\mathcal{B})}{2^{\tau_{\mathsf{root}}}}.$$

Plaintext-computing queries contribute to N and n_{leaf}, but not to q_{v}.

Proof. The proof uses the same hidden-tag accounting as the adjacent-hybrid bound, with the plaintext-computing interface included in the construction transcript. Work in the flat-RO $\mathsf{INT\text{-}RUP}$ game of Section 4.2. Let forge be the event that a valid non-replay verification query accepts. Let $\mathsf{bad}_{\mathsf{key}}$ be the event that a direct $\mathsf{RO}_{\mathsf{flat}}$ query decodes under $\mathsf{KeyedTWOut}$ to the hidden key K and a counted output-point class, and let $\mathsf{bad}_{\mathsf{leaf}}$ be the event that some construction-generated final leaf transcript receives the same τ_{leaf}-bit leaf tag projection as a distinct final leaf transcript that shares its key, nonce, and leaf index and is reached by an encryption computation (the same-slot event of Lemma 11). Final leaf transcripts generated by encryption, plaintext-computing, and verification computations are all included in n_{leaf}.

The key-test term is unchanged by the plaintext-computing oracle. By Lemma 14, stated for the $\mathsf{INT\text{-}RUP}$ extension, the hidden key remains uniform before $\mathsf{bad}_{\mathsf{key}}$ outside the candidate keys already tested by direct queries, and Lemma 10 gives

$$\Pr[\mathsf{bad}_{\mathsf{key}}] \leq \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})).$$

For the leaf term, no direct query has pre-read a hidden-key final leaf tag point before $\mathsf{bad}_{\mathsf{key}}$: such a query would decode under $\mathsf{KeyedTWOut}$ to K and the corresponding $\mathcal{L}_{\mathsf{tag}}(j)$ class. Coupling the real $\mathsf{INT\text{-}RUP}$ execution to the all-reject continuation used in Lemma 16 preserves the construction computations and leaf-tag table accesses up to the first accepting non-replay submission. The same-slot case of Lemma 11, applied to the enlarged set of construction computations, gives

$$\Pr[\mathsf{bad}_{\mathsf{leaf}} \text{ before } \mathsf{bad}_{\mathsf{key}} \cup \mathsf{forge}] \leq \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}}.$$

It remains to bound accepting verification before $\mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}$. The $\mathsf{INT\text{-}RUP}$ root-guessing lemma (Lemma 16) gives

$$\Pr[\mathsf{forge} \text{ before } \mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}] \leq q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Combining the three bounds through the first-occurrence decomposition of $\Pr[\mathsf{forge}]$ proves the theorem. $\square$

Theorem 11 ($\mathsf{INT\text{-}RUP}$ Integrity)**.** *For every $\mathsf{INT\text{-}RUP}$ adversary $\mathcal{A}$ against $\mathsf{TW128}$ with nonce-respecting encryption queries and the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{KeyGuess}_k(N_{\mathsf{prim}}) + \frac{N_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. By the Lift Application corollary (Corollary 9) for the $\mathsf{INT\text{-}RUP}$ game, there is a random oracle $\mathsf{INT\text{-}RUP}$ adversary $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ with $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$, $n_{\mathsf{leaf}}(\mathcal{B}) \leq N_{\mathsf{leaf}}$, and $q_{\mathsf{v}}(\mathcal{B}) \leq Q_{\mathsf{v}}$ such that

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{RO}}(\mathcal{B}).$$

Theorem 10 bounds the random oracle term, and substituting the derived-adversary resource caps gives the stated bound. $\square$

7 Concrete Bounds and Work Cap

The preceding sections prove two families of bounds: the tag-centric hash and committing bounds of Section 5, and the AEAD-facing bounds of Section 6. This section instantiates all of them at the implemented $\mathsf{TW128}$ parameters and records the common work cap used for the paper's concrete security claim.

7.1 Concrete Security Claim

Corollary 7 (Concrete TW128 Security)**.** *Under the implemented parameters $k = c = \nu = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$, every adversary with work cap $N_{\mathsf{tot}} \le 2^{63}$—and, in the multi-user case, user cap $D \le 2^{63}$—has advantage at most 2^{-128} in each of TW128's notions: multi-user indistinguishability (Theorem 9), all-in-one AE (Corollary 6), INT-RUP integrity (Theorem 11), the hash security of $\mathsf{H}_{\mathsf{TW128}}$—$s$-way collision resistance for every $s \ge 2$ in the sense of [BH22], with the [RS04] notions of standard and everywhere second-preimage resistance inherited from collision resistance on every fixed input slice, standard preimage resistance on every fixed input slice, and everywhere preimage resistance (Theorems 4 to 6 and Corollary 2)—and the [BH22] CMTDs-4 committing bound it yields (Corollary 4).*

Proof. The resource consequences of Section 4.3 give $N_{\mathsf{prim}} \le N_{\mathsf{tot}}$, $Q_{\mathsf{v}} \le N_{\mathsf{tot}}$, $N_{\mathsf{leaf}} \le N_{\mathsf{tot}}$, and $N_{\mathsf{prim}} + N_{\mathsf{leaf}} \le N_{\mathsf{tot}}$. The user cap D remains independent, since created-but-idle users need not contribute to N.

At $N_{\mathsf{tot}} \le 2^{63}$, each flat-sponge term—at N_{tot} or at N_{prim}—and each birthday-scale root or leaf term at $s = 2$ is at most $2^{-131} + 2^{-192}$, while each linear root-preimage, key-guessing, verification, or leaf tag term with no factor D is at most a 2^{-193}-scale term. The fixed-slice Pre degradation is at most 2^{-256}, since $m_{a,\mu} \ge k + \nu = 512$. Thus the largest non-multi-user bound is the $s = 2$ collision bound, which is below $4 \cdot 2^{-131} = 2^{-129}$; larger s only decreases the root-collision term. The all-in-one AE bound carries its two flat-sponge terms as its only birthday-scale terms and lies below 2^{-129}, and the INT-RUP bound carries $\mathsf{Lift}_c(N_{\mathsf{tot}})$ alone. In the multi-user case, using $D \le 2^{63}$,

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) < \left(2^{-130} + 2^{-193}\right) + 2^{-130} + 2^{-192} + 2^{-131} < 2^{-128} :$$

the two flat-sponge terms sum to at most $2 \cdot (2^{-131} + 2^{-194}) = 2^{-130} + 2^{-193}$, the key-guessing term satisfies $DN_{\mathsf{prim}}/2^{256} \le 2^{126}/2^{256} = 2^{-130}$, the remaining linear terms satisfy $(Q_{\mathsf{v}} + N_{\mathsf{leaf}})/2^{256} \le 2^{64}/2^{256} = 2^{-192}$, and $D(D-1)/2^{257} < 2^{-131}$; the total is below $5 \cdot 2^{-131} + 2^{-192} + 2^{-193} < 2^{-128}$. These estimates instantiate every closed form in Section 7.2; Section 7.3 discusses the margin they leave. ☐

Reading the cap. The cap $N_{\mathsf{tot}} \le 2^{63}$ is a global work point for the whole experiment, not a per-user or per-theorem allowance. In a single-user execution with no direct primitive queries, it means up to 2^{63} construction-internal KECCAK-p[1600, 12] calls, on the order of 2^{71} processed bytes at the rate $\rho = 167$. In the multi-user theorem the separate cap $D \le 2^{63}$ bounds created users, while only construction computations and direct primitive queries consume N_{tot}; the extreme proof corner $D = N_{\mathsf{prim}} = 2^{63}$ therefore has no construction work. Beyond this cap the explicit formulas of Section 7.2 still apply, but Corollary 7 no longer asserts that every notion is simultaneously below the 2^{-128} target.

7.2 Instantiated Bounds

The implemented parameters of Section 3.2 fix

$$k = c = \nu = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256.$$

Substituting into Corollaries 2, 4 and 6 and Theorems 4 to 6, 9 and 11 gives the following closed-form bounds.

All-in-one AE.

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}(\mathcal{A}) \le \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}}(N_{\mathsf{prim}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}} + Q_{\mathsf{v}} + N_{\mathsf{leaf}}}{2^{256}}.$$

Table 7: Concrete $\mathsf{TW128}$ security at the work cap $N_{\mathsf{tot}} \le 2^{63}$ (with $D \le 2^{63}$ for mu-ind), under $k = c = \nu = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$ and the resource consequences $N_{\mathsf{prim}}, Q_{\mathsf{v}}, N_{\mathsf{leaf}} \le N_{\mathsf{tot}}$ and $N_{\mathsf{prim}} + N_{\mathsf{leaf}} \le N_{\mathsf{tot}}$ of Section 4.3. The Coll and CMTDs-4 rows hold for every $s \ge 2$; larger s only shrinks the root-collision term.

Notion	Dominant scale	At the cap
All-in-one AE	Lift_c	$\le 2^{-128}$
Multi-user ind.	$D\,\mathsf{KeyGuess}_k(N_{\mathsf{prim}})$	$\le 2^{-128}$
INT-RUP	Lift_c	$\le 2^{-128}$
$\mathsf{H}_{\mathsf{TW128}}$ collision (Coll)	Lift_c, $\mathsf{LeafColl}$, $\mathsf{RootColl}$	$\le 2^{-128}$
$\mathsf{H}_{\mathsf{TW128}}$ second-preimage (Sec/eSec)	inherited from Coll	$\le 2^{-128}$
$\mathsf{H}_{\mathsf{TW128}}$ preimage (Pre)	Lift_c, $\mathsf{LeafColl}$	$\le 2^{-128}$
$\mathsf{H}_{\mathsf{TW128}}$ everywhere preimage (ePre)	Lift_c	$\le 2^{-128}$
CMTDs-4 [BH22]	Lift_c and $\mathsf{RootColl}$	$\le 2^{-128}$

Multi-user indistinguishability.

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}}(N_{\mathsf{prim}}+1)}{2^{257}} + \frac{DN_{\mathsf{prim}} + Q_{\mathsf{v}} + N_{\mathsf{leaf}}}{2^{256}} + \frac{D(D-1)}{2^{257}}.$$

INT-RUP integrity.

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}} + Q_{\mathsf{v}} + N_{\mathsf{leaf}}}{2^{256}}.$$

s-way collision resistance of $\mathsf{H}_{\mathsf{TW128}}$. For every $s \ge 2$,

$$\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+N_{\mathsf{leaf}}}{2}}{2^{256}} + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{256(s-1)}}.$$

For $s = 2$, this is the ordinary two-input collision bound:

$$\begin{aligned}\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{TW128}}(\mathcal{A}) \le \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} &+ \frac{(N_{\mathsf{prim}}+N_{\mathsf{leaf}})(N_{\mathsf{prim}}+N_{\mathsf{leaf}}-1)}{2^{257}}\\ &+ \frac{(N_{\mathsf{prim}}+1)(N_{\mathsf{prim}}+2)}{2^{257}}.\end{aligned}$$

Second-preimage resistance of $\mathsf{H}_{\mathsf{TW128}}$. By Corollary 2, the standard and everywhere [RS04] second-preimage advantages of $\mathsf{H}_{\mathsf{TW128}}$ are bounded by the same $s = 2$ collision expression above.

Preimage resistance of $\mathsf{H}_{\mathsf{TW128}}$. For fixed byte lengths a, μ, let $m_{a,\mu} = k + \nu + 8a + 8\mu$. By Theorem 5,

$$\begin{aligned}\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{TW128}}(\mathcal{A}) \le \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} &+ \frac{(N_{\mathsf{prim}}+N_{\mathsf{leaf}})(N_{\mathsf{prim}}+N_{\mathsf{leaf}}-1)}{2^{257}}\\ &+ \frac{N_{\mathsf{prim}}+1}{2^{256}} + 2^{256-m_{a,\mu}}.\end{aligned}$$

Under the implemented parameters $k = \nu = 256$, the slice term is $2^{-256-8a-8\mu}$.

s-way CMTDs-4 committing security. For every $s \geq 2$, the all-bodies-equal special case drops the known-key leaf tag collision term:

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A}) \;\leq\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{256(s-1)}}.$$

Remark 1 (Two-Opening CMTDs-4 Specialization). Setting $s = 2$ recovers the ordinary two-opening CMTDs-4 statement of [BH22]:

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A}) \;\leq\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+2}{2}}{2^{256}} \;=\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1) + (N_{\mathsf{prim}}+1)(N_{\mathsf{prim}}+2)}{2^{257}}.$$

Everywhere preimage resistance of $\mathsf{H}_{\mathsf{TW128}}$.

$$\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{TW128}}(\mathcal{A}) \;\leq\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}}+1}{2^{256}}.$$

7.3 Interpreting the Work Cap

The bounds of Section 7.2 aggregate the loss terms of Section 4.4. The cap $N_{\mathsf{tot}} \leq 2^{63}$ is a simultaneous work point for all notions, not a separate parameter choice per theorem. Table 7 records the dominant terms and aggregate cap bounds; Corollary 7 supplies the individual term sizes at the cap.

Term tightness. The loss terms of Section 4.4 are mostly generic search or birthday terms, and their leading scales are tight in the usual random oracle sense. The key-guessing term is matched at constants by direct random guesses for a k-bit key. In the multi-user theorem's free-parameter setting, the term $D\,\mathsf{KeyGuess}_k(N_{\mathsf{prim}})$ is also tight at leading scale: using the per-user nonce discipline of Section 2.7, which permits one nonce value across users, an adversary obtains one known nonempty encryption from each targeted user and compares each direct candidate-key query against all collected root-initialization keystream blocks at once. Each guess then matches some targeted user with probability $D/2^k$. The concrete joint cap charges the target encryptions to N, lowering the feasible scale as discussed below.

The root terms are the standard random oracle search terms: an s-way birthday search matches the leading scale of $\mathsf{RootColl}_\tau(Q, s)$, and fixed-target preimage search matches the per-query $2^{-\tau}$ rate of $\mathsf{RootPre}_\tau(Q)$. The hidden-key same-slot leaf term $n_{\mathsf{leaf}}/2^{\tau_{\mathsf{leaf}}}$ is also tight at constants for nonce-respecting verification: after encrypting a two-chunk message under nonce U, an adversary can submit non-replay verification queries that replace the leaf ciphertext chunk by fresh strings of the same length while keeping the root tag. Each query recomputes the hidden leaf tag at the same slot $(K, U, 1)$, and acceptance occurs when the fresh leaf tag equals the original hidden leaf tag, with probability $2^{-\tau_{\mathsf{leaf}}}$ per verification-side final leaf transcript.

The flat sponge lift term $\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)/2^{c+1}$ dominates the BDPV differentiating bound $f_P(N_{\mathsf{tot}})$ of [BDPVA08, Lemma 5] via Lemma 20. That differentiating bound is matched at leading order by the standard inner-collision distinguisher; the remaining conservatism is in composing indifferentiability into the keyed games, not in an obviously improvable simulator bound. Real-vs-ideal games pay the same differentiating bound once more, at N_{prim}, to replace the ideal-side simulator by the real permutation (Lemma 21); since $N_{\mathsf{prim}} \leq N_{\mathsf{tot}}$, that term never dominates the N_{tot} term.

Dominant terms and margin. At $N_{\mathsf{tot}} \leq 2^{63}$, the hash bounds other than ePre, together with the CMTDs-4 bound, have flat-sponge and root or leaf birthday terms on the same 2^{-131} scale. The ePre and $\mathsf{INT\text{-}RUP}$ bounds retain the flat sponge loss as their only birthday-scale term, and the single-user AE bound carries its two flat-sponge terms, at N_{tot} and N_{prim}, as its only birthday-scale terms. Thus the flat sponge loss is not always the largest summand: under the loose per-term cap, the $s = 2$ root-collision term $\binom{N_{\mathsf{prim}}+2}{2}/2^{256}$ is on the same scale as $\mathsf{Lift}_c(N_{\mathsf{tot}})$.

The multi-user bound is the exception. Its hybrid-induced term $D\,\mathsf{KeyGuess}_k(N_{\mathsf{prim}})$ can reach 2^{-130} at the extreme cap $D = N_{\mathsf{prim}} = 2^{63}$, and the displayed estimate then has value $5 \cdot 2^{-131} + 2^{-194}$ because that corner forces $N = 0$ and hence $Q_{\mathsf{v}} = N_{\mathsf{leaf}} = 0$. The corner is a proof artifact rather than an attack point: an adversary without construction computations receives identical primitive answers in both worlds (Section 4.1). At feasible resource splits, the multi-user key-guessing witness needs one accepted nonempty encryption per targeted user; with t targeted users and $q = N_{\mathsf{prim}}$ direct guesses, the joint cap imposes $2t + q \leq 2^{63}$, so the feasible key-guessing scale is about 2^{-133}, reached near $t = 2^{61}$ and $q = 2^{62}$. The remaining distance to the proof bound is hybrid slack from charging created-but-idle users plus the two flat-sponge composition terms.

Expressed in bytes, $N_{\mathsf{tot}} \leq 2^{63}$ corresponds to at most $\rho \cdot 2^{63} = 167 \cdot 2^{63} < 2^{71}$ bytes of plaintext, associated data, ciphertext, and aggregation through the duplex (Section 3.2). The margin widens away from the multi-user corner: at $D \leq 2^{32}$, the D-indexed terms drop below 2^{-160} and the multi-user bound falls to its two flat-sponge terms, below $2^{-130} + 2^{-159}$.

Reading the work cap. Corollary 7 is a success-probability statement at a fixed cap, not a statement that security is exhausted there. The closed forms of Section 7.2 remain explicit functions of the resources beyond 2^{63}, and their dominant terms are birthday-type in N_{tot} on the 256-bit capacity, so the bounds degrade quadratically rather than abruptly: at $N_{\mathsf{tot}} = 2^{100}$, for example, the $s = 2$ collision bound is still below 2^{-55}. Once N_{tot} reaches $2^{c/2} = 2^{128}$, the flat sponge loss exceeds $1/2$ and no longer gives meaningful security (Section 4.4); the displayed closed forms themselves become trivial advantage bounds when they reach 1, which each form does within a factor of two of $N_{\mathsf{tot}} = 2^{c/2}$, the exact point depending on how many quadratic-scale terms it carries. The cap $N_{\mathsf{tot}} \leq 2^{63}$ is therefore a data-scale choice: a single work bound at which every notion's advantage lies below the 2^{-128} target simultaneously, not the largest workload the mode tolerates.

The residual conservatism in the cap comes from the indifferentiability route and, for mu-ind, the hybrid accounting above. A direct keyed-duplex analysis would be expected to replace the quadratic Lift_c terms by finer construction–primitive cross terms, but it would also need to handle the adversarial-initialization hash and committing games used here. Section 9.5 discusses that route and the gaps it would need to close.

Cross-scheme bound-level comparisons against prior sponge AEADs and the committing AEAD line of [BH22] are deferred to Sections 9.1 to 9.3.

8 Performance Evaluation

TW128 fixes the root/leaf transcript but leaves leaf scheduling to the implementation. This section evaluates that separation: the same ciphertext and tag are produced by scalar and vectorized implementations, while complete leaf chunks can be batched to recover practical long-message throughput. The comparison baseline is hardware-accelerated AES-128-GCM on the same hosts, so the measurements should be read as an implementation sanity check and same host reference point, not as a claim that TW128 is faster than AES-GCM.

8.1 Optimized Implementations

We analyze a Go implementation of TW128[1] with optimized assembly paths for the performance-critical permutation and absorb/squeeze loops. We compare three vectorized assembly paths: `amd64` AVX-512, `amd64` AVX2, and `arm64` NEON using the Armv8.2 SHA-3 extension. The framing, domain separation, padding, tree bookkeeping, and dispatch remain in shared Go code. A pure Go path uses a scalar permutation and a scalar eight-instance loop, and serves both as a reference path for cross-checking the assembly cores and as a fallback on other architectures.

The tree structure of TW128 exposes two distinct permutation workloads. The root duplex—which absorbs the associated data, processes chunk 0, and absorbs the leaf tag aggregation transcript—is inherently sequential, since each call consumes the previous call's output. It is driven by a *single-duplex* ($\times 1$) core, except for its chunk-0 message phase. The leaf duplexes operate on disjoint state and disjoint inputs (Section 3), so complete leaf chunks are batched and run through an *eight-way* ($\times 8$) core. Leftover runs of two to seven complete chunks use narrower remainder kernels, while a single leftover complete chunk, a partial trailing chunk, and the count-aggregation tail fall back to the single-duplex core.

Chunk 0 itself runs on the batched kernels rather than serially, a technique we refer to as *chunk* 0 *fusion*. The root duplex's chunk-0 message phase has the same kernel-visible schedule as a leaf chunk and differs only in its initial state. The implementation therefore seeds lane 0 of the first batched call with the root state, runs chunk 0 alongside the first leaf chunks, then writes lane 0's output state back to the root duplex before the root absorbs the leaf tags from that same call. This avoids a serial pass over chunk 0 and is the reason the cycles-per-byte curve drops as soon as complete leaves are present (Section 8.3). Section C gives the lane-major layout, remainder-kernel strategies, lane 0 writeback details, and instruction-level kernel structure.

Architecture-specific cores. On `amd64` the optimized implementation selects an AVX-512 or AVX2 core once at startup from the `AVX512VL` CPUID feature bit. On `arm64` the optimized implementation uses NEON with the Armv8.2 SHA-3 extension (`FEAT_SHA3`, available on Apple Silicon and Neoverse V1/V2 and N2). Both architecture-specific paths provide a single-duplex core and batched leaf kernels; the assembly stays below the mode boundary, so framing, domain separation, padding, and tree bookkeeping remain shared with the reference path.

Data-independent execution. The scalar reference and all three vectorized implementations avoid secret-dependent branches and secret-dependent memory or vector indices for the key, nonce, associated-data contents, and plaintext or ciphertext contents. The permutation and the XOR-based absorb/squeeze loops are straight-line with respect to data values. The only indexed table is the round-constant array, addressed by the public round number; dispatch, batched loads, stores, gathers, scatters, and inactive-lane handling are controlled by public lengths, chunk counts, and fixed chunk strides. Section C records the architecture-specific addressing details.

8.2 Benchmark Methodology

We measure on two hosts, one per architecture. The `amd64` host is a Google Compute Engine `c4-standard-4` instance with an Intel Xeon Platinum 8581C (Emerald Rapids, 2.30 GHz base, AVX-512 including `AVX512VL`, `AVX512_VBMI2`, `GFNI`, and `VAES`), two physical cores with simultaneous multithreading enabled, 48 KiB L1d and 32 KiB L1i per core, 2 MiB L2

[1] The implementation is available at https://github.com/codahale/treewrap.

per core, and a 260 MiB shared L3, running Debian 12 (kernel 6.1) under KVM. The guest does not expose a `cpufreq` interface, so the frequency governor and turbo state are not under our control; we mitigate the resulting variance statistically (below). The `arm64` host is an Apple M4 Pro (10 performance cores and 4 efficiency cores; per performance core, 128 KiB L1i, 64 KiB L1d, and a 16 MiB shared L2) running macOS 26.5.1 (Darwin 25.5), with `FEAT_SHA3` present. macOS exposes no userspace frequency control.

Measurements are collected by a standalone harness (`cmd/cpb`) that locks its goroutine to an OS thread and, for each algorithm, operation, and message length, first calibrates an iteration count that fills at least 100 ms, then records 100 samples. It reports the median cycles per byte and median wall-clock throughput together with the interquartile range as a dispersion measure (Table 10). Both metrics are read from the same timed loop, so the cycles-per-byte and throughput figures for a given measurement come from one interleaved run rather than two separately scheduled ones.

Each timed call uses a fixed all-zero key and nonce, empty associated data, and preallocated source and destination buffers. Open measurements decrypt and verify a ciphertext prepared before timing; seal and open reset the destination slice length inside the timed loop but do not allocate per call. The reported figures are therefore warm-cache, steady-state rates for message processing under a fixed AEAD object. The single-threaded measurement leaves the SMT sibling of the `amd64` host idle. All reported figures are single-threaded: they exercise the SIMD axis of TW128's parallelism, and multicore scaling remains unmeasured. We do not report multicore scaling separately: once leaf chunks are assigned to workers, the remaining scaling limit is scheduling, memory bandwidth, and ordered transport of 32-byte leaf tags back to the root, rather than a mode-level cryptographic effect. We report a sweep of seven message lengths from 1 B to 16 MiB.

The cycle counters differ by architecture, and this affects how the cycles-per-byte figures may be read. On `amd64` the harness reads `RDTSC`; on this part the time-stamp counter is invariant and ticks at the 2.30 GHz reference frequency, so the reported figures are *reference* cycles per byte and are unaffected by turbo (and convert directly to wall-clock time). On `arm64` the harness reads `CNTVCT_EL0` and scales it by the ratio of the peak core frequency to `CNTFRQ_EL0`; since Apple exposes no frequency API, the peak is auto-detected by timing a dependent integer-add chain and taking the top observed P-state, yielding *core* cycles per byte. Consequently the cycles-per-byte figures are comparable within an architecture and against the same-host AES-128-GCM baseline, but not directly across architectures; absolute throughput (Table 9) is the cross-platform metric.

The baseline is Go's AES-128-GCM implementation (`crypto/aes` with `crypto/cipher`). It uses AES-NI and `PCLMULQDQ` on `amd64`, and the AES and `PMULL` instructions on `arm64`. The AES block cipher, GCM wrapper, and TW128 AEAD object are constructed once before timing, so the measurements exclude key schedule and GHASH key-power precomputation. Each timed TW128 call still performs the root duplex initialization inside the call. Both directions are measured for each scheme; the cycles-per-byte table reports both, and the throughput table reports the encryption direction.

8.3 Throughput

Table 8 reports cycles per byte across the message-length sweep and Table 9 the corresponding wall-clock throughput in the bulk regime. The main pattern in the TW128 rows is the descent produced by the tree-parallel leaf core. A message of at most one chunk has no complete leaf chunk for chunk 0 to share a kernel with, so it runs entirely on the single-duplex core, and the 8 KiB column sits at that core's rate. Past one chunk, chunk 0 fusion (Section 8.1) keeps every complete chunk on the batched kernels, and the descent tracks lane occupancy.

On the `amd64` AVX-512 path, TW128 encryption falls $1.81 \to 0.73 \to 0.39$ cycles per byte across the 8, 32, and 64 KiB columns and then holds at 0.38–0.39 through 16 MiB.

Table 8: Median cycles per byte (100 samples). The `amd64` figures are `RDTSC` reference cycles at the 2.30 GHz invariant time-stamp counter; the `arm64` figures are core cycles at the auto-detected peak performance-core frequency. Cycle counts are not directly comparable across architectures (Section 8.2).

	Message length						
	1 B	64 B	8 KiB	32 KiB	64 KiB	1 MiB	16 MiB
Intel Xeon 8581C (Emerald Rapids), AVX-512							
TW128 encrypt	571	9.04	1.81	0.73	0.39	0.38	0.38
TW128 decrypt	605	9.55	1.82	0.73	0.39	0.38	0.38
AES-128-GCM seal	119	2.74	0.30	0.29	0.29	0.29	0.29
AES-128-GCM open	99	2.17	0.29	0.28	0.28	0.28	0.29
Intel Xeon 8581C (Emerald Rapids), AVX2							
TW128 encrypt	647	10.18	2.21	1.15	1.10	1.09	1.10
TW128 decrypt	682	10.70	2.21	1.06	1.10	1.08	1.06
Apple M4 Pro, NEON + `FEAT_SHA3`							
TW128 encrypt	620	9.88	1.97	0.92	0.89	0.89	0.91
TW128 decrypt	671	10.65	2.04	0.90	0.87	0.87	0.89
AES-128-GCM seal	113	2.84	0.44	0.43	0.43	0.44	0.46
AES-128-GCM open	123	2.56	0.42	0.41	0.41	0.42	0.43

Table 9: Measured wall-clock throughput (GB/s, GB = 10^9 bytes) from the same `cmd/cpb` run as Table 8, encryption direction. The `amd64` AES-128-GCM row is the AVX-512 host and is unaffected by the TW128 core selection.

	8 KiB	32 KiB	64 KiB	1 MiB	16 MiB
Intel Xeon 8581C (Emerald Rapids)					
TW128 (AVX-512)	1.27	3.17	5.93	6.10	6.00
TW128 (AVX2)	1.04	2.00	2.10	2.10	2.08
AES-128-GCM	7.75	8.00	8.05	8.00	7.87
Apple M4 Pro					
TW128 (NEON+SHA3)	2.27	4.84	5.04	5.01	4.93
AES-128-GCM	10.04	10.27	10.33	10.07	9.71

The AVX2 path has the same shape at lower throughput: it runs the eight instances as two groups of four rather than in a single 8-wide register, leaving the AVX-512 core about 2.9× faster in bulk. The `arm64` curve flattens earlier because the eight-way core processes its batch as four two-instance pairs; a 32 KiB message, chunk 0 and three leaves as two full pairs, is already close to the long-message rate. In wall-clock terms TW128 reaches 6.00 GB/s on the `amd64` AVX-512 host and 4.93 GB/s on the M4 Pro for 16 MiB messages.

Dispersion. Each cell of Tables 8 and 9 is the median of 100 samples; Table 10 summarizes their spread as the interquartile range (IQR) relative to the median. The medians are stable: across the sweep the relative IQR of the TW128 cycles-per-byte measurements is at most 0.7% on the `amd64` AVX-512 path, 0.9% on AVX2, and 1.9% on the M4 Pro, with per-path medians of 0.3%, 0.4%, and 1.2%. The full min–max range is wider at isolated points—up to 4.9%, 12.0%, and 16.7% on the three paths—because turbo, governor, and scheduler states are not under our control (Section 8.2); these are tail excursions that the IQR excludes. We therefore report medians throughout and treat the IQR as the

Table 10: Cycles-per-byte measurement dispersion for TW128 across the seven-length sweep (seal and open, 100 samples per point), as the interquartile range relative to the median. The final column is the widest single-point min–max range; it is excluded from the IQR as a tail effect.

	Relative IQR		Max
Path	median	max	range
Xeon 8581C, AVX-512	0.3%	0.7%	4.9%
Xeon 8581C, AVX2	0.4%	0.9%	12.0%
Apple M4 Pro	1.2%	1.9%	16.7%

dispersion measure.

8.4 Latency

For short messages in this benchmark, the cost is set by the serial root duplex and is independent of the parallel leaf machinery. With empty associated data, a message of at most $\rho = 167$ bytes occupies a single rate block and never enters leaf mode, so it costs exactly two permutations: one to initialize the root duplex and one to close the message block and extract the tag. The 1 B and 64 B columns of Table 8 bear this out: per-message latency is nearly flat across them at roughly 571 reference cycles on `amd64` (571 at 1 B versus $9.04 \times 64 \approx 578$ at 64 B) and 620 core cycles on `arm64` (620 versus $9.88 \times 64 \approx 633$), i.e. about 285 and 310 cycles per KECCAK-p[1600, 12] respectively. Because such messages bypass the leaf core entirely, leaf-level parallelism imposes no latency penalty on them.

The latency/throughput trade-off appears instead in the intermediate regime, which chunk 0 fusion has narrowed to roughly the one-to-eight-chunk band. The worst per-byte cost in the sweep is the single-chunk regime at 8 KiB (1.8–2.2 cycles per byte across the three paths), where the message is the chunk-0 phase alone on the single-duplex core. Once the message has complete leaf chunks for chunk 0 to share a kernel with, the cost falls quickly: by 32 KiB TW128 is within a factor of two of its AVX-512 floor and close to its `arm64` floor, and the floor itself is reached once the eight-way core fills at 64 KiB. The root duplex also bounds the tail of a long message: the final tag cannot be produced until chunk 0 has been processed and every leaf tag has been absorbed into the aggregation transcript. That absorption is cheap—each rate block of the root absorbs just over five 32-byte leaf tags, so a 16 MiB message ($\approx$ 2000 leaves) adds on the order of 400 root permutations against roughly 10^5 permutations of leaf work, under 0.5% overhead—so the serial root does not materially cap bulk throughput.

8.5 AES-128-GCM Baseline

We compare against the one baseline measured under the same harness on the same hosts, the Go standard library's hardware-accelerated AES-128-GCM (Tables 8 and 9). The harness times TW128 and AES-128-GCM back-to-back at each length and reads both metrics from the same loop, so the cycles-per-byte and throughput comparisons hold under identical conditions and agree. In the bulk regime AES-128-GCM is faster on both architectures, as expected for a primitive with dedicated AES and carryless-multiply instructions: at 16 MiB it runs at 0.29 cycles per byte on `amd64` and 0.46 on `arm64`, against 0.38 and 0.91 for TW128—about 1.3× and 2.0× faster. The wall-clock throughputs show the same margins: TW128 reaches 6.00 and 4.93 GB/s against AES-128-GCM's 7.87 and 9.71 GB/s. The wider `arm64` gap reflects the strength of Apple's AES and `PMULL` units. TW128 attains these rates with a permutation that has no dedicated hardware support,

Table 11: Structural comparison of TW128 with representative AEADs. Sizes are in bytes; AES-GCM-SIV also admits a 32-byte key, and Kravatte-SAE's key and nonce lengths are parameters (keys of up to 40 bytes, arbitrary-length nonces). "Misuse" is nonce-misuse resistance in the sense of [RS06]; "Commit" records a published committing result or attack in the relevant committing notion, with "broken" denoting a published low-cost committing attack, "hash CR" denoting the collision-resistance level of the hash used by the transform, and "open" marking a scheme with no published committing analysis. A dash means the entry depends on the base scheme. These entries are notion-specific summaries, not broad AE security claims. TW128's concrete bounds appear in Section 7.3.

Scheme	Primitive	Par.	K/U/T	AD	Misuse	Commit
TW128	Keccak-p[1600, 12]	yes	32/32/32	yes	no	CMTDs-4
Ascon-128a [DEMS21]	Ascon-p (320 b)	no	16/16/16	yes	no	$\approx$ 64 b
Xoodyak [DHP+20]	Xoodoo (384 b)	no	16/16/16	yes	no	2^{17} atk.
Kravatte-SAE [BDH+17]	Keccak-p[1600, 6]	yes	$\leq$ 40/var./16	yes	no	open
AES-GCM [Dwo07]	AES	yes	16/12/16	yes	no	broken
AES-GCM-SIV [GLL17]	AES	yes	16/12/16	yes	yes	broken
nAE + CTX [CR22]	any nAE	—	—	yes	—	hash CR

relying only on the SHA-3 extension on `arm64` and on general SIMD on `amd64`.

AES-128-GCM is also faster for short messages in this steady-state measurement: at 1 B it costs 119 and 113 cycles on the two hosts against TW128's 571 and 620. This is the expected result when AES-GCM key setup is amortized outside the timed call. In the sub-bulk regime AES-128-GCM is several times faster at one chunk (0.30 versus 1.81 cycles per byte at 8 KiB on `amd64`), but the gap narrows once the fused kernels engage: about 2.5× at 32 KiB and the bulk ratio from 64 KiB on. TW128's intended value relative to AES-128-GCM lies in the tag-centric hash and committing properties analyzed in Section 5, with the baseline AEAD bounds of Section 6, rather than in raw speed.

9 Discussion

9.1 Comparison Table

Table 11 situates TW128 against representative authenticated encryption schemes along the axes that distinguish it: the underlying primitive, whether the mode is parallelizable, the key, nonce, and tag sizes, associated data support, nonce-misuse resistance, and the published committing result or attack most relevant to the scheme. The rows are representative rather than exhaustive: sponge/duplex relatives, a Farfalle-based parallel design, AES baselines, a misuse-resistant AES baseline, and a generic committing transform. The comparison is structural, not a total security ranking; TW128's own concrete bounds and the regimes in which each term dominates are treated in Section 7.3, and its measured throughput against hardware-accelerated AES-128-GCM in Section 8.5. The committing column should be read within the named notions only, and a negative entry there does not assert a confidentiality or integrity break of the underlying AEAD. It records TW128's 128-bit CMTDs-4 bound derived from tag hash security, Ascon's roughly 64-bit committing bound [KSW23], published low-cost attacks where known [KSW23, CR22, ADG+22], and the hash-collision level inherited by CTX. The table does not separately track compact (tag-only) commitment, since that property has not been systematically analyzed for the comparison schemes.

9.2 Comparison with Prior Sponge AEADs

TW128 belongs to the single-permutation duplex AEAD family introduced in [BDPVA11]. Representative lightweight members of that family, Ascon [DEMS21] and Xoodyak [DHP+20], target 128-bit security on small permutations (320 and 384 bits) with a strictly serial duplex: each call depends on the previous one, so there is no mode-level parallelism to exploit. TW128 instead pays for a larger permutation, Keccak-p[1600, 12], and a 256-bit capacity—the same round count as KangarooTwelve [BDP+16]—in exchange for a tree that processes leaf chunks in parallel and for the headroom that meets the 2^{-128} target at the stated work cap. The cost of this choice is visible in Section 8: on short, single-leaf inputs TW128 is slower per byte than a compact serial duplex would be, and its advantage appears only once a message spans enough leaves to fill the vector units.

TW128 reuses KangarooTwelve's final-node/leaf split but not its Sakura coding [BDPVA14]. KangarooTwelve uses Sakura to make the final node parseable as a tree-hash transcript [BDP+16]. In TW128, the analogous chaining values are typed leaf tags: each leaf duplex is initialized with its leaf prefix and index, and the root absorbs leaf tags in index order followed by the leaf count (Sections 3 and 3.3). Sakura's structural role is therefore handled by mode initialization and domain separation rather than by a tree coding. The transcript separation lemmas make this explicit (Lemma 6 and Corollary 8); the hash collision bound sees the leaf encoding only through the explicit leaf tag collision term of Theorem 4.

Keyak [BDP+15], whose Motorist mode is already parallelizable, is the closest prior duplex AEAD in spirit; Section 1.2 contrasts its wire-visible, encryptor-and-decryptor-shared degree of parallelism with TW128's run-time leaf choice. Strobe [Ham17] is a serial sponge framework aimed at whole protocols rather than at a standalone AEAD, and like Ascon and Xoodyak it does not parallelize.

Farfalle [BDH+17] leaves the duplex setting altogether: it is a parallel pseudorandom function whose compression layer sums masked-and-permuted input blocks into an accumulator and whose expansion layer generates output from a rolling state, with authenticated encryption supplied by the Farfalle-SAE and Farfalle-SIV modes on top (Section 1.2). Its Kravatte instance runs Keccak-p[1600, 6], and the parallelism available to an implementation is arbitrary, as in TW128. The two designs price their permutations for different adversary models. Farfalle's six-round compression layer is analyzed in a PRF setting where inputs are randomized by secret masks and an adversary does not see both the input and output of the same permutation call [BDH+17]. TW128's tag-hash claims are adversarial-initialization claims: in the hash and committing games, the adversary may choose the key as part of the input being hashed. TW128 therefore keeps the strict duplex object, uses the hash-grade round count discussed in Section 9.4, and takes its parallelism from the tree topology rather than from a parallel PRF compression layer.

Committing security is not unique to TW128 in this family: a dedicated analysis of the sponge-based NIST lightweight finalists proves Ascon committing while breaking others [KSW23], and some constructions, such as Strobe and the Farfalle modes, have not been examined in that setting at all. What distinguishes TW128 is that the authentication tag is analyzed directly as a hash output for the full input. Collision, second-preimage, preimage, and everywhere preimage bounds for that tag then yield the compact commitment statements, including the 128-bit full-input CMTDs-4 bound, as mode-native consequences without leaving the single-permutation duplex setting.

9.3 Comparison with Committing AEAD Constructions

The dominant route to committing AEAD is a generic transform over a non-committing base scheme. [BH22] give the UtC, RtC, and HtE transforms, which add key- or context-commitment to an arbitrary AEAD, and [CR22] give CTX, which makes any nonce-based

AE scheme commit to its full context at the cost of a single hash over a short string and with no ciphertext expansion. These transforms are inexpensive and broadly applicable; CTX in particular is nearly free. It is nevertheless a committing-AE transform, not a tag-hash transform: it binds the transformed ciphertext, but does not make the tag a standalone hash output for the full input. TW128 differs not by undercutting their cost but by integration. The same permutation provides confidentiality, integrity, and the tag hash output; no separate commitment primitive, key-derivation step, or auxiliary hash is invoked. The multi-input full-commitment bound follows from the same root tag collision bound used for hash security (Section 5), and Theorem 4 applies even when the ciphertext bodies used as openings differ, up to the additional leaf tag birthday term. This places TW128 alongside the dedicated committing primitives—the compactly committing AEAD of [GLR17] and the encryptment of [DGRW18]—which likewise build commitment in rather than bolt it on, but which were designed for message franking rather than as general nonce-based AEADs. The contrast with transformed conventional schemes is sharpened by the attacks of Section 1.2: base schemes such as GCM and OCB are not committing on their own [CR22], and a transform is the only way to make them so.

Closest to TW128 here is the recent work of [DHM+24], which builds nonce-based AE directly on the SHA-3 functions SHAKE and TurboSHAKE—the latter using the same round-reduced KECCAK-p permutation as TW128—and likewise achieves CMT-4 natively, from the collision resistance of the underlying function rather than from a transform. The two designs make different structural choices: [DHM+24] targets session-based communication, chaining a stream of messages through intermediate tags and generalizing the keyed duplex into a *duplex cipher*, whereas TW128 is a single two-level tree whose independent leaves let the implementation choose parallelism at runtime with constant memory (Sections 1.2 and 8.1). Because a session chain processes each message serially, TW128's principal advantage is throughput on large messages (Section 8): its parallel leaves fill vector units a serial duplex leaves idle, at the cost of the session support [DHM+24] provides and TW128 does not. Both demonstrate that committing AE needs no add-on transform once it is built on a well-scrutinized KECCAK-p permutation.

9.4 Cryptanalysis of the Underlying Permutation

Every bound in this paper models KECCAK-p[1600, 12] as a uniformly random permutation (Section 4). That modeling choice is supported by cryptanalytic scrutiny rather than by proof, so this subsection summarizes the third-party record and the safety margin it indicates. The Keccak round function has been unchanged since 2008, and TW128 uses the permutation at the round count introduced by KangarooTwelve [BDP+16] and later exposed directly as TurboSHAKE [BDH+23], whose designers maintain a survey of third-party cryptanalysis. TurboSHAKE128 shares TW128's rate and capacity ($r = 1344$, $c = 256$) as well as its permutation, so attacks on it and on round-reduced SHAKE128 transfer to TW128's geometry directly.

In the unkeyed setting, the deepest collision attacks on any SHA-3 or SHAKE instance reach 6 of TW128's 12 rounds: a SAT-aided attack on 6-round SHAKE128 with time $2^{123.5}$ [GLST22], and internal-differential attacks reaching 6 rounds of SHAKE256 and 5 rounds of SHA3-384 [ZHL24]. Preimage attacks reach 5 rounds [ZHL25]. Against the collision- and preimage-style attacks most relevant to the root tag hash claims of Section 5, the permutation therefore retains a margin of 6 of 12 rounds—the assessment under which the KangarooTwelve round count was chosen and later reaffirmed [BDP+16, BDH+23]. The structural distinguisher reaching the most rounds of the permutation itself, the SymSum property on 9 rounds reported in the survey of [BDH+23], has, like the zero-sum distinguishers before it, not been extended to an attack on any sponge or duplex mode.

The keyed setting is closer to TW128's AE games: a hidden key, with adversary-controlled nonces and messages entering through the duplex rate. The attacks reaching the most rounds here are cube and conditional cube attacks. Dinur et al. break up to 9 rounds of Keccak-based stream ciphers and MACs faster than generic search, with keystream prediction on 8 rounds at time and data 2^{128} and on 9 rounds (of a 512-bit-key variant) at 2^{256} [DMP+15]. With MILP-optimized conditional cubes, nonce-respecting key recovery reaches 8 of Lake Keyak's 12 rounds at time 2^{71} for 128-bit keys, 9 rounds at 2^{137} for 256-bit keys, and 9 of KMAC256's 24 rounds at 2^{147} [SGSL18]. Lake Keyak is a keyed duplex over the same 1600-bit permutation whose initialization, like TW128's root and leaf initializations, absorbs the key and nonce ahead of nonce-varied cube queries, so these results are the closest published analog to TW128's keyed setting. They leave a margin of 3 to 4 of 12 rounds against attacks whose complexities start at 2^{71}.

In sum, the known attack frontier stands at 6 of 12 rounds for collisions and preimages and at 8 to 9 rounds for keyed-mode key recovery and structural distinguishers. TW128 uses Keccak-p[1600, 12] at the same round count and with the same rate/capacity split as KangarooTwelve and TurboSHAKE128, so its margin assessment tracks theirs, and any future advance against the permutation applies to all three alike. What TW128 adds at the mode level and these analyses do not cover is the tree structure itself—in particular the family of leaf initializations sharing (K, U) and differing only in the leaf index—and we consider dedicated cryptanalysis of that structure a natural target for future work.

9.5 Limitations and Open Problems

Section 4 states the formal scope of the security claims; this subsection discusses the consequences of the excluded settings and the corresponding open problems.

Nonce misuse. As scoped in Section 4, TW128's AE and mu-ind bounds require nonce-respecting encryption queries, and the mode is not misuse-resistant: a repeated nonce reuses leaf keystream and breaks confidentiality, as for any online duplex stream cipher. Schemes such as AES-GCM-SIV [GLL17], in the deterministic/misuse-resistant lineage of [RS06], tolerate nonce repetition at the price of a second pass. A misuse-resistant variant of TW128 is left open.

Unverified plaintext release. The INT-RUP theorem of Theorem 11 covers integrity when an adversary can obtain plaintext-computing oracle outputs before verification, in the release-of-unverified-plaintext setting of [ABL+14]. This is an integrity-only statement. It does not give plaintext awareness or privacy under release of unverified plaintext: TW128 is online and single-pass, and exposing unverified stream plaintext can reveal keystream information. Application behavior on unverified plaintext is likewise outside the provable model.

Conservative lift. Per-term tightness is itemized in Section 4.4; the flat-sponge terms are conservative because our proof follows the unkeyed sponge/hash line. This route matches the role of the authentication tag: TW128's committing statements are consequences of tag hash security, and sponge hash security is supported by a mature literature. The price is the indifferentiability lift of Section B: one $\mathsf{Lift}_c(N_{\mathsf{tot}})$ term on the real side, and in real-vs-ideal games a second proximity term at N_{prim} (Corollary 9).

A direct keyed-duplex treatment would have to close two gaps. First, existing keyed-duplex bounds are hidden-key AE bounds: the game samples the key, and the keyed initialization material is not part of the adversary's output. The root tag claims here are adversarial-initialization statements. In the hash and committing games, the adversary may choose the key, nonce, associated data, message, or competing opening. Thus the proof must handle adversarially chosen initialization transcripts, not only hidden-key encryption and verification queries.

Second, the available keyed-duplex AE interface is not a direct substitute for a byte-string AEAD with arbitrary final-block length. The modern bounds synthesized by [Men23] use a duplex call phased as permutation, squeeze, then absorb or overwrite ([Men23, Sec. 3.4, Table 1]). In the authenticated-encryption application, this interface is instantiated as MonkeySpongeWrap [Men23, Alg. 8]. As written, however, the overwrite interface does not reconstruct the missing padding bits of a truncated final ciphertext block on decryption: encryption absorbs the padded final plaintext block after squeezing and then truncates the resulting ciphertext to the message length, while decryption receives only this truncated ciphertext, pads it independently, and uses the overwrite branch to set the rate. The decryption transcript can therefore diverge before tag recomputation.

TW128 instead stays with the BDPV11 duplex phasing: a call absorbs the actual input string, padded by the duplex rule, and then returns the requested output. At the stream layer this supports arbitrary length messages by splitting them into bounded duplex blocks, with the final short ciphertext block absorbed as it is rather than reconstructed through an overwrite branch (Section 3). A direct keyed-duplex theorem for TW128 would therefore need both this BDPV11 phasing and adversarial-initialization hash and committing games. We retain the indifferentiability lift because it covers these claims uniformly and already meets the 128-bit target (Section 7.3).

Beyond-birthday security and leakage. All bounds are birthday-type on the 256-bit capacity, delivering the 128-bit target but no more; a beyond-birthday analysis is open. The model is also single-stage and leakage-free: side-channel resistance is addressed only at the implementation level, through the data-independent execution properties of Section 8.1, and is not part of the provable-security claims.

10 Conclusion

TW128 starts from the premise that an AEAD tag should be the compact digest applications often treat it as: a hash output for the full encryption context, with committing security following from that hash story rather than added as an external layer. The construction adapts the KangarooTwelve tree-hashing pattern into a two-level tree of keyed duplexes over a single Keccak-p[1600, 12] permutation. The chunk decomposition is canonical, while the number of leaf duplexes evaluated in parallel is an implementation choice rather than a mode parameter. This gives scalar, SIMD, and multicore implementations the same ciphertexts and tags, with constant working memory and scalable parallelism.

The main security result is tag hash security for the wrapper that returns the final authentication tag on input (K, U, A, M). In the public permutation model, via an intermediate random oracle and the flat sponge lift, we prove collision resistance, standard and everywhere second-preimage resistance, preimage resistance on fixed input slices, and everywhere preimage resistance. Those bounds yield the AEAD-facing committing statements, including full-input CMTDs-4. The same root structure gives INT-RUP integrity under release of unverified plaintext. As baseline AEAD properties, we prove concrete multi-user indistinguishability and derive all-in-one AE as its single-user corollary. At the implemented parameters, all stated notions meet a 128-bit target.

The implementation results show the cost and payoff of putting that tag hash inside a tree. Short inputs pay for the larger permutation and the root/leaf structure, but long inputs expose independent leaves while preserving one canonical wire format. Vectorized `amd64` and `arm64` implementations reach bulk throughputs of 48.0 Gbit/s on the AVX-512 path, 39.4 Gbit/s on the `arm64` path, and 16.6 Gbit/s on the AVX2 path, with the AVX-512 and `arm64` paths at 51–76% of hardware-accelerated AES-128-GCM throughput.

The remaining questions are those left open in Section 9.5: nonce misuse would require a different construction, privacy under release of unverified plaintext is not provided by the

INT-RUP theorem, and sharper bounds would need either a direct keyed-duplex treatment of the BDPV11-style phasing with adversarial initialization or a beyond-birthday argument. More broadly, TW128 supports the thesis that a single Keccak-p permutation with substantial cryptanalytic history can provide authenticated encryption whose tag has hash security strong enough to drive commitment, while retaining scalable hashing-style performance.

Acknowledgments

A hearty thank-you to my wife, who remains reasonably mystified at my choice of hobbies.

References

[ABL+14] Elena Andreeva, Andrey Bogdanov, Atul Luykx, Bart Mennink, Nicky Mouha, and Kan Yasuda. How to securely release unverified plaintext in authenticated encryption. In *Advances in Cryptology – ASIACRYPT 2014*, volume 8873 of *Lecture Notes in Computer Science*, pages 105–125. Springer, 2014.

[ADG+22] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. How to abuse and fix authenticated encryption without key commitment. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3291–3308. USENIX Association, 2022.

[BDH+17] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Farfalle: Parallel permutation-based cryptography. *IACR Transactions on Symmetric Cryptology*, 2017(4):1–38, 2017.

[BDH+23] Guido Bertoni, Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, and Benoît Viguier. TurboSHAKE. Cryptology ePrint Archive, Paper 2023/342, 2023.

[BDP+15] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. CAESAR submission: Keyak v2. CAESAR Competition, Round 3 Candidate, 2015.

[BDP+16] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, and Benoît Viguier. KangarooTwelve: Fast hashing based on Keccak-p. Cryptology ePrint Archive, Paper 2016/770, 2016.

[BDPVA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, 2008.

[BDPVA11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Duplexing the sponge: Single-pass authenticated encryption and other applications. In *Selected Areas in Cryptography – SAC 2011*, volume 7118 of *Lecture Notes in Computer Science*, pages 320–337. Springer, 2011.

[BDPVA14] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sakura: A flexible coding for tree hashing. In *Applied Cryptography and Network Security – ACNS 2014*, volume 8479 of *Lecture Notes in Computer Science*, pages 217–234. Springer, 2014.

[BH22] Mihir Bellare and Viet Tung Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, volume 13276 of *Lecture Notes in Computer Science*, pages 845–875. Springer, 2022. Citations refer to the full version of July 23, 2024.

[BT16] Mihir Bellare and Björn Tackmann. The multi-user security of authenticated encryption: AES-GCM in TLS 1.3. In *Advances in Cryptology – CRYPTO 2016*, volume 9814 of *Lecture Notes in Computer Science*, pages 247–276. Springer, 2016.

[CR22] John Chan and Phillip Rogaway. On committing authenticated encryption. In *Computer Security – ESORICS 2022*, Lecture Notes in Computer Science. Springer, 2022.

[DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, 2021.

[DGRW18] Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. Fast message franking: From invisible salamanders to encryptment. In *Advances in Cryptology – CRYPTO 2018*, volume 10991 of *Lecture Notes in Computer Science*, pages 155–186. Springer, 2018.

[DHM+24] Joan Daemen, Seth Hoffert, Silvia Mella, Gilles Van Assche, and Ronny Van Keer. Shaking up authenticated encryption. Cryptology ePrint Archive, Paper 2024/1618, 2024.

[DHP+20] Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Xoodyak, a lightweight cryptographic scheme. *IACR Transactions on Symmetric Cryptology*, 2020(S1):60–87, 2020.

[DMP+15] Itai Dinur, Paweł Morawiecki, Josef Pieprzyk, Marian Srebrny, and Michał Straus. Cube attacks and cube-attack-like cryptanalysis on the round-reduced Keccak sponge function. In *Advances in Cryptology – EUROCRYPT 2015*, volume 9056 of *Lecture Notes in Computer Science*, pages 733–761. Springer, 2015.

[Dwo07] Morris J. Dworkin. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. Technical Report Special Publication 800-38D, National Institute of Standards and Technology, 2007.

[GLL17] Shay Gueron, Adam Langley, and Yehuda Lindell. AES-GCM-SIV: Specification and analysis. Cryptology ePrint Archive, Paper 2017/168, 2017.

[GLR17] Paul Grubbs, Jiahui Lu, and Thomas Ristenpart. Message franking via committing authenticated encryption. In *Advances in Cryptology – CRYPTO 2017*, volume 10403 of *Lecture Notes in Computer Science*, pages 66–97. Springer, 2017.

[GLST22] Jian Guo, Guozhen Liu, Ling Song, and Yi Tu. Exploring SAT for cryptanalysis: (quantum) collision attacks against 6-round SHA-3. In *Advances in Cryptology – ASIACRYPT 2022*, volume 13793 of *Lecture Notes in Computer Science*, pages 645–674. Springer, 2022.

[Ham17] Mike Hamburg. The STROBE protocol framework. Cryptology ePrint Archive, Paper 2017/003, 2017.

[KSW23] Juliane Krämer, Patrick Struck, and Maximiliane Weishäupl. Committing AE from sponges: Security analysis of the NIST LWC finalists. Cryptology ePrint Archive, Paper 2023/1525, 2023.

[LGR21] Julia Len, Paul Grubbs, and Thomas Ristenpart. Partitioning oracle attacks. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 195–212. USENIX Association, 2021.

[Men23] Bart Mennink. Understanding the duplex and its security. *IACR Transactions on Symmetric Cryptology*, 2023(2):1–46, 2023.

[MLGR23] Sanketh Menda, Julia Len, Paul Grubbs, and Thomas Ristenpart. Context discovery and commitment attacks: How to break CCM, EAX, SIV, and more. In *Advances in Cryptology – EUROCRYPT 2023*, volume 14007 of *Lecture Notes in Computer Science*, pages 379–407. Springer, 2023.

[Nat15] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication FIPS PUB 202, U.S. Department of Commerce, August 2015.

[RS04] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In *Fast Software Encryption – FSE 2004*, volume 3017 of *Lecture Notes in Computer Science*, pages 371–388. Springer, 2004. Citations refer to the full version of July 16, 2009.

[RS06] Phillip Rogaway and Thomas Shrimpton. A provable-security treatment of the key-wrap problem. In *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 373–390. Springer, 2006.

[SGSL18] Ling Song, Jian Guo, Danping Shi, and San Ling. New MILP modeling: Improved conditional cube attacks on Keccak-based constructions. In *Advances in Cryptology – ASIACRYPT 2018*, volume 11273 of *Lecture Notes in Computer Science*, pages 65–95. Springer, 2018.

[ZHL24] Zhongyi Zhang, Chengan Hou, and Meicheng Liu. Probabilistic linearization: Internal differential collisions in up to 6 rounds of SHA-3. In *Advances in Cryptology – CRYPTO 2024*, volume 14923 of *Lecture Notes in Computer Science*, pages 241–272. Springer, 2024.

[ZHL25] Zhongyi Zhang, Chengan Hou, and Meicheng Liu. Preimage attacks on up to 5 rounds of SHA-3 using internal differentials. In *Advances in Cryptology – EUROCRYPT 2025*, volume 15601 of *Lecture Notes in Computer Science*, pages 333–363. Springer, 2025.

A Supporting Lemmas

The proof core separates structural decoding from probabilistic accounting. The well-formed transcript language of Definition 2 fixes which flattened queries are valid TW128 construction prefixes. Lemmas 5, 6 and 8 then show how such a query is flattened, parsed as a typed transcript, and assigned an output point; the exact keyed decoder of Definition 4 then decodes direct queries to candidate keys, when any. Lemma 10 turns those decoded keys into an adaptive key-test bound. The remaining interfaces expose the local events consumed by the main proofs: encryption-side freshness, verification hidden-key exposure and root tag guessing, leaf tag collisions, and final-root candidate sequencing. The tag hash proof applies the candidate bound, committing statements reuse the resulting root-collision bound, and the mu-ind and INT-RUP proofs derive the premises for their accounting lemmas.

How to read this appendix. Figure 2 records the proof dependency map. It groups local interfaces when the exact proof citations would obscure the spine. The Bridge & Decode node is Lemma 8 together with the exact keyed decoder of Definition 4. The Root/Output Separation node groups Output-Point Separation (Corollary 8), Opening Triple Separation (Proposition 1), and Final-Root Input Separation (Proposition 2); the use of Leaf-Transcript Recovery (Lemma 7) is folded into the final-root separation fact. The Key-Indexed Renaming node (Lemma 9) supplies the key-field disjointness and renaming used by hidden-key encryption and verification accounting. The Adaptive Candidate node (Lemma 2) is the generic probabilistic engine: it bounds the Leaf Tag Collision lemma (Lemma 11) and, through the Root Tag Candidate Sequence and Root Tag Hash and Target-Hit Bound (Lemmas 3 and 4) of Section 5, the tag hash proof; the committing consequences reuse the same root-collision bound under their opening constraints. The mu-ind proof reuses the same structural lemmas through the explicit hybrid game sequence of Definition 5, adding freshness, key-test, leaf-collision, and verification-first root guessing accounting.

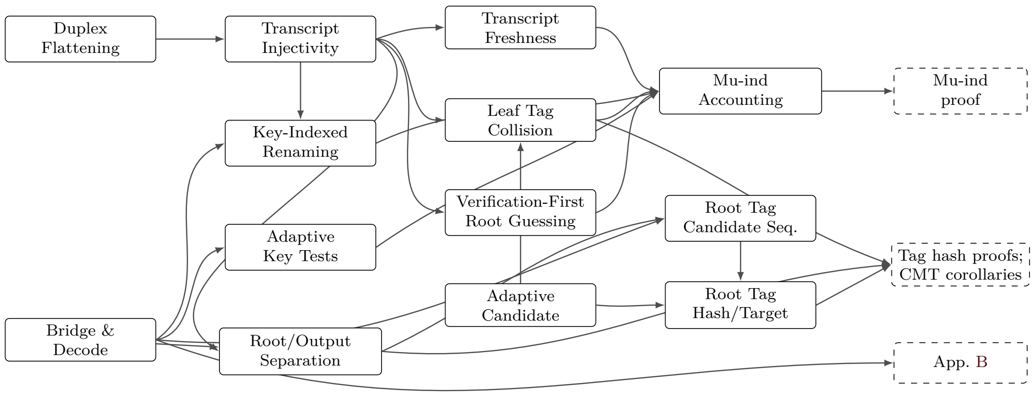

Figure 2: Proof dependency map for the supporting components in this appendix and their consumers. Solid boxes are lemmas or grouped local proof interfaces; dashed boxes are proof components that consume them.

A.1 Duplex Flattening

Lemma 5 (Duplex Flattening)**.** *Every TW128 root or leaf duplex output is equal to a [BDPVA11] sponge output on the corresponding flattened duplex-call transcript, and the ciphertext-absorbing message phase admits a flattened duplex transcript that is well defined for both encryption and decryption.*

Proof. Let D be a duplex object using permutation π, rate r, and padding $\mathsf{pad10^*1}$. For a sequence of duplex calls $Z_i = \mathsf{D.duplexing}(\sigma_i, \ell_i)$ with each σ_i short enough to fit in one duplex block after padding, [BDPVA11, Lemma 3] gives

$$Z_i = \mathsf{Sponge}[\pi, \mathsf{pad10^*1}, r]\big(\sigma_0 \parallel \mathsf{pad}_0 \parallel \sigma_1 \parallel \mathsf{pad}_1 \parallel \cdots \parallel \sigma_i,\ \ell_i\big),$$

where $\mathsf{pad}_j = \mathsf{pad10^*1}[r](|\sigma_j|)$, and Lemma 4 says that, for fixed $\mathsf{pad10^*1}$ and r, the map from the duplex input-string sequence to the flattened sponge input is injective.

In TW128, each duplex call absorbs a byte-aligned block σ followed by a 4-bit phase suffix $\sigma_{\mathsf{ph}} \in \{\sigma_{\mathsf{init}}, \sigma^-_{\mathsf{ad}}, \dots, \sigma^+_{\mathsf{agg}}\}$ (Section 3.3), so the duplex-call input is $\sigma \parallel \sigma_{\mathsf{ph}}$ followed by $\mathsf{pad10^*1}$. With $r = 1344$ and $\rho_{\max}(\mathsf{pad10^*1}, r) = r - 2 = 1342$ bits, the maximum byte-aligned input is $\rho = 167$ bytes (Section 3.2), giving a worst-case duplex-call input of $8{\cdot}167{+}4 = 1340 \le 1342$ bits. The two initialization calls are shorter still: $|p_{\mathsf{root}} \| K \| U|{+}4 = 8 \cdot (6 + 32 + 32) + 4 = 564$ bits and $|p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8| + 4 = 8 \cdot (6 + 32 + 32 + 8) + 4 = 628$ bits. [BDPVA11, Lemmas 3 and 4] therefore apply independently to every TW128 duplex object.

The root duplex is initialized by absorbing $p_{\mathsf{root}} \parallel K \parallel U$ under σ_{init}(Algorithm 2), then absorbs the associated data blocks under $\sigma^-_{\mathsf{ad}}/\ \sigma^+_{\mathsf{ad}}$, the root ciphertext blocks in the message phase under $\sigma^-_{\mathsf{msg}}/\ \sigma^+_{\mathsf{msg}}$, and the leaf tag aggregation blocks under $\sigma^-_{\mathsf{agg}}/\ \sigma^+_{\mathsf{agg}}$. By the duplex-to-sponge equality, every root duplex output is the sponge function evaluated on the root transcript prefix accumulated up to the relevant output point. The same holds for each leaf duplex, initialized by $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$ and absorbing only its own ciphertext blocks in the message phase. In decryption and verification, Algorithm 3 absorbs the submitted ciphertext blocks before the root tag comparison, so the same flattened message phase transcript is defined whether the comparison accepts or rejects. ☐

A.2 Well-formed Transcript Prefixes

For a duplex-call prefix $((\sigma_0, \sigma_{\mathsf{ph},0}), \dots, (\sigma_i, \sigma_{\mathsf{ph},i}))$, write

$$\mathsf{Flat}\big((\sigma_0, \sigma_{\mathsf{ph},0}), \dots, (\sigma_i, \sigma_{\mathsf{ph},i})\big) = \Big\Vert_{j=0}^{i-1} \big(\sigma_j \parallel \sigma_{\mathsf{ph},j} \parallel \mathsf{pad}_j\big) \parallel \sigma_i \parallel \sigma_{\mathsf{ph},i},$$

where $\mathsf{pad}_j = \mathsf{pad10^*1}[r](|\sigma_j \parallel \sigma_{\mathsf{ph},j}|)$, the padding of the j-th full duplex-call input. The current call's padding pad_i is not part of Flat; it is applied internally by the sponge function. Equivalently, after the i-th call the duplex state is the sponge state after absorbing $\mathsf{Flat}(\cdot) \parallel \mathsf{pad}_i$.

Definition 2 (Well-Formed TW128 Transcript Prefixes)**.** A typed TW128 transcript prefix is *well formed* if it is a prefix, ending at a construction transcript lookup point, of one of the following two call languages.

- A root transcript starts with $(p_{\mathsf{root}} \parallel K \parallel U, \sigma_{\mathsf{init}})$, with fixed-width fields $K \in \{0,1\}^k$ and $U \in \{0,1\}^\nu$. It then absorbs the associated data byte string (only when the associated data is nonempty; the phase is otherwise elided, Section 3.4), the root ciphertext byte string, and the aggregation byte string (only when $n \ge 1$; when $n = 0$ the aggregation phase is elided, Section 3.4, and the root tag is emitted by the final σ^+_{msg} message call), in that order. Each present phase byte string is partitioned by the block rule of Algorithm 1: the empty string gives one empty final block, and a nonempty string gives zero or more ρ-byte non-final blocks followed by one final block. The phases use respectively $\sigma^-_{\mathsf{ad}}/\sigma^+_{\mathsf{ad}}$, $\sigma^-_{\mathsf{msg}}/\sigma^+_{\mathsf{msg}}$, and $\sigma^-_{\mathsf{agg}}/\sigma^+_{\mathsf{agg}}$.

- A leaf-j transcript starts with $(p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8, \sigma_{\mathsf{init}})$, where $1 \le j < 2^{64}$ and the fields have fixed width. It then absorbs the leaf ciphertext byte string under $\sigma^-_{\mathsf{msg}}/\sigma^+_{\mathsf{msg}}$, using the same block rule.

The message byte string in this language is always the ciphertext byte string absorbed by the construction: encryption obtains it by XORing the plaintext with the current keystream, while decryption and verification use the submitted ciphertext. Thus a verification transcript is well formed for every in-domain ciphertext before the tag comparison is made; acceptance is not part of the transcript language.

The root aggregation byte string has the form $T_1 \parallel \cdots \parallel T_n \parallel \langle n \rangle_8$, where each T_j has fixed byte length $\tau_{\mathsf{leaf}}/8$. The trailing fixed-width field therefore gives a unique parse of the leaf tag sequence. The output-point class of a well-formed counted prefix is the class in Table 12 determined by its duplex instance and final-call suffix.

Remark 2 (Well-Formedness versus Realizability). The language of Definition 2 is deliberately larger than the set of transcript prefixes that TW128 computations realize. For example, it does not require an aggregation phase to follow a root message phase of exactly β bytes, although every real computation with $n \geq 1$ absorbs a full β-byte root chunk (Algorithm 2). The inclusion runs in the safe direction everywhere: every construction-generated prefix is well formed, so the structural lemmas of this appendix and the separation propositions of Section 5 apply to construction transcripts as restrictions of statements proved on the larger language, while the direct-query decoder of Definition 4 and the root tag candidate predicate of Lemma 3 accept the larger language and therefore only over-count adversary queries, which loosens no bound.

Name the tag-emitting transcript families:

$$\mathcal{R}_{\mathsf{tag}} = \text{root transcript prefix whose final call is a } \sigma^{+}_{\mathsf{agg}} \text{ aggregation call,}$$

$$\mathcal{L}_{\mathsf{tag}}(j) = \text{leaf-}j \text{ transcript prefix whose final call is a } \sigma^{+}_{\mathsf{msg}} \text{ message call.}$$

The final root tag is emitted by the $\sigma^{+}_{\mathsf{agg}}$ aggregation call (the $\mathcal{R}_{\mathsf{tag}}$ family) when $n \geq 1$, and by the closing $\sigma^{+}_{\mathsf{msg}}$ message call (an $\mathcal{R}^{+}_{\mathsf{msg}}$ prefix, in the notation of Corollary 8) when $n = 0$ and the aggregation phase is elided. Since $n = \mathrm{leaves}(C)$ is fixed by $\|C\|$, every computation on a given ciphertext emits its tag at the output point selected by $\|C\|$. Intermediate construction transcript lookup points (initialization, AD, non-final message, non-final aggregation, and the final root message call that every $n \geq 1$ computation reaches with zero requested output) are also typed prefixes covered by Lemma 8; the exact keyed direct-query decoder names them in Table 12.

Definition 3 (Root Tag Output Point and Final Root Transcript Class)**.** For any in-domain encryption or verification computation on ciphertext C, the *root tag output point* is the construction lookup that emits the root tag: $\mathcal{R}_{\mathsf{tag}}$ when $\mathrm{leaves}(C) \geq 1$, and the closing root message point $\mathcal{R}^{+}_{\mathsf{msg}}$ when $\mathrm{leaves}(C) = 0$ and aggregation is elided. The *final root transcript class* of such a computation is the set of construction computations that realize the same bridged final root input at this root tag output point. A class is *encryption-first* if its first construction generation is by an encryption query, and *verification-first* if its first construction generation is by a non-replay verification query.

A.3 Transcript Injectivity

Lemma 6 (Transcript Injectivity)**.** *Well-formed TW128 flattened duplex-call transcript prefixes are injectively encoded and separated by duplex instance (root vs leaf), leaf index j, phase suffix, and duplex-call order. For final-root prefixes, equality of flattened inputs recovers the same root initialization, AD blocks, and root ciphertext blocks, and, when $n \geq 1$, the same aggregation input and leaf tag sequence.*

Proof. By Lemma 5, every well-formed output point can be written as a [BDPVA11] flattened sponge input for the corresponding duplex-call prefix, namely Flat as defined above. The deterministic current-call $\mathsf{pad10^*1}$ is applied internally by the sponge function and is not part of the input to which [BDPVA11, Lemma 4] is applied. That lemma recovers the duplex-call sequence from the flattened sponge input; in TW128, each call input is $\sigma \parallel \sigma_{\mathsf{ph}}$ with byte-aligned σ and fixed 4-bit suffix σ_{ph}, so recovering the call input recovers the pair $(\sigma, \sigma_{\mathsf{ph}})$.

Definition 2 fixes the root and leaf initialization fields, the admissible phase order, and the final-call suffixes. The distinct prefixes p_{root} and p_{leaf} separate root from leaf transcripts; the fixed-width leaf field $\langle j \rangle_8$ separates leaves; and the seven suffix values of Table 2 separate phases and non-final/final calls. Thus equality of flattened inputs forces equality of the recovered instance type, key, nonce, leaf index when present, phase suffixes, and duplex-call order.

Phase-recovery sub-claim. For each present phase $P \in \{\mathsf{ad}, \mathsf{msg}, \mathsf{agg}\}$ (the ad phase is present exactly when the associated data is nonempty; see Section 3.4), TW128 absorbs the phase's input byte string s_P by partitioning into ρ-byte blocks (with the empty case yielding a single empty block, per Section 3.4) and feeding each block to the duplex with suffix σ_P^- (non-final) or σ_P^+ (final). Lemma 5 and [BDPVA11, Lemma 4] recover the corresponding $(\sigma, \sigma_{\mathsf{ph}})$ sequence from the flattened input. Because the partition map is deterministic and uniquely invertible on its image (the empty case yielding the single empty block carrying σ_P^+), concatenating the recovered σ blocks reconstructs s_P exactly. The map from s_P to the phase's $(\sigma, \sigma_{\mathsf{ph}})$ sequence is therefore injective.

Applied phase by phase to a well-formed final-root transcript:

- AD. The associated data phase contributes a σ_{ad}^+-tagged block exactly when the associated data is nonempty; an empty associated data elides the phase and contributes no block. When the phase is present the sub-claim recovers the AD byte string. Thus distinct associated data strings $A_i \neq A_j$ are separated either by the presence versus absence of the AD phase (when exactly one is empty) or, when both are nonempty, by their distinct AD σ-block sequences.

- MSG. Distinct absorbed root or leaf ciphertext byte strings produce distinct MSG σ-block sequences; the message language of Definition 2 is ciphertext-absorbing for encryption and decryption alike.

- AGG. The aggregation phase is present exactly when $n \geq 1$ (Section 3.4). When it is present, the sub-claim applied to the aggregation byte string $T_1 \parallel \cdots \parallel T_n \parallel \langle n \rangle_8$ recovers that byte string. The final eight bytes give $\langle n \rangle_8$, and each leaf tag has fixed byte length $\tau_{\mathsf{leaf}}/8$, so the byte string has a unique parse as a leaf tag sequence. Two root aggregation encodings are therefore equal exactly when their leaf tag sequences are equal. When $n = 0$ the aggregation phase is absent and the final-root transcript ends at the recovered σ_{msg}^+ root message call; the MSG bullet recovers the root ciphertext byte string, and the absence of any $\sigma_{\mathsf{agg}}^-/\sigma_{\mathsf{agg}}^+$ call distinguishes such a prefix from every $n \geq 1$ final-root prefix.

Therefore the recovered final-root transcript determines the root initialization, AD blocks, root ciphertext blocks, and, when $n \geq 1$, the aggregation byte string and leaf tag sequence. □

Lemma 7 (Leaf-Transcript Recovery)**.** *Let two construction-generated final-root prefixes have equal flattened inputs, and suppose that, for each position j of their common leaf tag sequence, the two computations' position-j final leaf transcripts are either equal or receive distinct leaf tags. Then equality of flattened inputs recovers the same leaf transcript sequence, and therefore the same full absorbed ciphertext. Both computations' position-j final leaf transcripts are leaf-j transcripts under the common recovered (K, U), so the hypothesis holds whenever no two distinct construction-generated final leaf transcripts with the same key, nonce, and leaf index receive the same leaf tag; when one of the two prefixes is reached by an encryption computation, it suffices that no construction-generated final leaf* *transcript receives the same leaf tag as a distinct encryption-reached final leaf transcript with the same key, nonce, and leaf index.*

Proof. By Transcript Injectivity (Lemma 6), the two equal flattened final-root inputs recover the same root initialization, AD blocks, root ciphertext blocks, and, when $n \geq 1$, the same aggregation byte string and leaf tag sequence; the trailing $\langle n \rangle_8$ field fixes the common leaf count n. When $n = 0$ no leaf is present and the recovered root ciphertext blocks already constitute the full absorbed ciphertext. When $n \geq 1$, the aggregation byte string parses uniquely as the leaf tag sequence $T_1 \parallel \cdots \parallel T_n$; the two leaf tag sequences are equal, so for each j the two computations' position-j final leaf transcripts—leaf-j transcripts carrying the common recovered (K, U) (Definition 2, Section 3.4)—receive the same leaf tag, and by hypothesis are equal, hence so are their absorbed leaf ciphertext bodies. Together with the common root ciphertext blocks this recovers the same full absorbed ciphertext. □

A.4 Bridge and Typed Decoding

Lemma 8 (Bridge Injectivity)**.** *The random oracle games used in Sections 5 and 6 and Corollary 5 are ordinary random oracle games on a single random oracle* $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ *over the unpadded [BDPVA08] H-input domain (Section 4.2), with* TW128 *root and leaf duplex outputs answered by the typed restriction*

$$\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X)), \qquad \mathsf{flat}(X) = I[r, r_{\max}]\big(\mathsf{raw\_flat}(X)\big),$$

for $r = r_{\max} = 1344$. Here $\mathsf{raw\_flat}(X)$ is the native flattened input $\mathsf{Flat}(\cdot)$ of Section A.2 applied to the duplex-call sequence of X, and $I[r, r_{\max}]$ is the [BDPVA11] Theorem 3 preprocessing map. The induced map $X \mapsto \mathsf{flat}(X)$ is injective on well-formed typed TW128 *duplex transcript prefixes.*

Proof. The bridge $I[r, r_{\max}]$ is described by the proof of [BDPVA11, Theorem 3] in three steps: (1) pad the input with $\mathsf{pad10^*1}[r]$; (2) for each r-bit block, append $r_{\max} - r$ zero bits; (3) strip the trailing $\mathsf{pad10^*}$ from the result. The $\mathsf{pad10^*1}$ padding in step (1) appends a 1, then $q = (-|\mathsf{raw\_flat}(X)| - 2) \bmod r$ zero bits, then a closing 1; the $\mathsf{pad10^*}$-unpadding in step (3) strips the $r_{\max} - r$ trailing zero bits introduced by step (2) together with the closing 1 of step (1). For TW128 with $r = r_{\max}$, step (2) appends an empty string and step (3) strips zero trailing zeros and the closing 1, so the net effect of steps (1) and (3) is to map a native flattened input $W = \mathsf{raw\_flat}(X)$ to $W \parallel 1 \parallel 0^q$. Any string in this image has a unique inverse: q is the number of trailing zero bits, the preceding bit is the inserted 1, and deleting this suffix recovers W.

Injectivity of flat on well-formed typed TW128 prefixes then follows from Lemma 6: distinct well-formed typed prefixes have distinct $\mathsf{raw\_flat}(\cdot)$ images, and the bridge is injective on that image. Direct adversary queries on inputs outside the bridged TW128 well-formed transcript language still count in $q_{\mathsf{RO}}(\mathcal{B})$ (Section 4.3) but cannot trigger $\mathsf{bad_{key}}$. □

The bridge injectivity established here underlies Opening Triple Separation (Proposition 1, Section 5): distinct (K, U, A) triples yield distinct bridged final root transcripts.

Corollary 8 (Output-Point Separation)**.** *Every well-formed* TW128 *typed transcript prefix at a counted construction transcript lookup point belongs to exactly one of the following output-point classes, identified by the recovered duplex instance and the 4-bit suffix of its final call:*

- $\mathcal{R}_{\mathsf{init}}$ *(root duplex, suffix σ_{init}),*
- $\mathcal{L}_{\mathsf{init}}(j)$ *(leaf-j duplex, suffix σ_{init}),*

- $\mathcal{R}^-_{\mathsf{ad}}$ / $\mathcal{R}^+_{\mathsf{ad}}$ *(root duplex, suffix* σ^-_{ad} *or* σ^+_{ad}*),*
- $\mathcal{R}^-_{\mathsf{msg}}$ / $\mathcal{R}^+_{\mathsf{msg}}$ *(root duplex, suffix* σ^-_{msg} *or* σ^+_{msg}*),*
- $\mathcal{L}^-_{\mathsf{msg}}(j)$ *(leaf-*j *duplex, suffix* σ^-_{msg}*),*
- $\mathcal{L}_{\mathsf{tag}}(j)$ *(leaf-*j *duplex, suffix* σ^+_{msg}*),*
- $\mathcal{R}_{\mathsf{agg}}$ *(root duplex, suffix* σ^-_{agg}*),*
- $\mathcal{R}_{\mathsf{tag}}$ *(root duplex, suffix* σ^+_{agg}*).*

For any two such prefixes whose bridged flattened inputs are equal, bridge decoding (Lemma 8) followed by the separation of Lemma 6 recovers the same duplex instance, leaf index j *when present,* (K, U) *pair, output-point class, and duplex-call sequence. Equivalently, any two distinct counted transcript prefixes already differ in one of these recovered fields, and hence in their flattened inputs.*

Proof. Lemma 8 recovers equality of native flattened inputs from equality of bridged inputs, after which Lemma 6 recovers the duplex-call sequence and its per-call $(\sigma, \sigma_{\mathsf{ph}})$ pairs. The first call's σ is $p_{\mathsf{root}} \parallel K \parallel U$ for root instances and $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$ for leaf instances, with fixed-width fields, so it fixes the instance type (by p_{root} vs p_{leaf}), the (K, U) pair, and the leaf index j; within each instance type the phase suffixes are pairwise distinct, so the recovered instance type together with the final-call suffix fixes the output-point class, and each prefix therefore lies in exactly one class. Conversely, two distinct well-formed prefixes in the same class on the same instance differ in some recovered call's $(\sigma, \sigma_{\mathsf{ph}})$ pair or in the sequence length, hence in their flattened inputs. $\square$

Definition 4 (Exact Keyed Decoder)**.** For a direct query to $\mathsf{RO}_{\mathsf{flat}}$ with input Y and finite length ℓ, define $\mathsf{KeyedTWOut}(Y)$ as a partial map from $\{0,1\}^*$ to $\{0,1\}^k \times \mathcal{C}_{\mathsf{out}}$, where $\mathcal{C}_{\mathsf{out}}$ is the finite set of output-point classes of Table 12. The map ignores ℓ. It returns $(K, \mathcal{C})$ iff there exists a typed $\mathsf{TW128}$ duplex-call prefix X such that $Y = \mathsf{flat}(X)$, the raw flattened prefix of X parses as a well-formed $\mathsf{TW128}$ transcript prefix in the class $\mathcal{C}$ listed in Table 12, and the first call of X contains the candidate key K in the fixed-width root field $p_{\mathsf{root}} \parallel K \parallel U$ or leaf field $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$. If no such prefix exists, $\mathsf{KeyedTWOut}(Y) = \perp$. In particular, inputs with the right final suffix but a malformed phase order, noncanonical block partition, malformed aggregation string, or out-of-range leaf index are outside the well-formed language and decode to $\perp$. A non-$\perp$ decoding always fixes the candidate key K and nonce U in the fixed-width initialization field. It fixes the associated data A only for root prefixes whose recovered phase order determines the complete AD string: a final AD prefix, or any later root message or aggregation prefix, with absence of AD calls determining $A = \varepsilon$. We use the key projection $(K, \mathcal{C})$ throughout.

By Lemma 8 and Corollary 8, $\mathsf{KeyedTWOut}$ is well defined: a bridged input cannot parse as two distinct counted well-formed prefixes, and an accepted input fixes a unique candidate key and output-point class. Inputs outside this well-formed transcript language decode to $\perp$.

The table is a transcript-lookup table, not an observation-length table. A class is counted even when the construction's requested output length at that call is zero, including root initialization, non-final AD and aggregation calls, a final AD call whose following root chunk requests no keystream bytes, and the final root message call. A direct query triggers the same decoder regardless of ℓ: short finite-output queries and zero-length queries $(Y, 0)$ still count as direct queries to the address Y. Leaves have no AD or aggregation classes; a leaf transcript enters the table only through leaf initialization, non-final leaf message blocks, or the final leaf tag point. When the associated data is empty the root transcript likewise has no $\mathcal{R}^-_{\mathsf{ad}}$ or $\mathcal{R}^+_{\mathsf{ad}}$ class (Section 3.4); the $\mathcal{R}_{\mathsf{init}}$class then supplies the first root

Table 12: Counted keyed transcript classes for $\mathsf{KeyedTWOut}$.

Class $\mathcal{C}$	Instance	Final call / role
$\mathcal{R}_{\mathsf{init}}$	root	σ_{init}, root initialization
$\mathcal{R}^{-}_{\mathsf{ad}}$	root	σ^{-}_{ad}, non-final AD block
$\mathcal{R}^{+}_{\mathsf{ad}}$	root	σ^{+}_{ad}, final AD block
$\mathcal{R}^{-}_{\mathsf{msg}}$	root	$\sigma^{-}_{\mathsf{msg}}$, non-final root message block
$\mathcal{R}^{+}_{\mathsf{msg}}$	root	$\sigma^{+}_{\mathsf{msg}}$, final root message block
$\mathcal{R}_{\mathsf{agg}}$	root	$\sigma^{-}_{\mathsf{agg}}$, non-final aggregation block
$\mathcal{R}_{\mathsf{tag}}$	root	$\sigma^{+}_{\mathsf{agg}}$, final aggregation / root tag
$\mathcal{L}_{\mathsf{init}}(j)$	leaf j	σ_{init}, leaf initialization
$\mathcal{L}^{-}_{\mathsf{msg}}(j)$	leaf j	$\sigma^{-}_{\mathsf{msg}}$, non-final leaf message block
$\mathcal{L}_{\mathsf{tag}}(j)$	leaf j	$\sigma^{+}_{\mathsf{msg}}$, final leaf message block / leaf tag

keystream block, exactly as $\mathcal{L}_{\mathsf{init}}(j)$ supplies the first leaf keystream block, and the decoder still ignores ℓ. Symmetrically, when $n = 0$ the root transcript has no $\mathcal{R}_{\mathsf{agg}}$or $\mathcal{R}_{\mathsf{tag}}$ class (Section 3.4); the final root message call $\mathcal{R}^{+}_{\mathsf{msg}}$ then supplies the root tag, exactly as $\mathcal{L}_{\mathsf{tag}}(j)$ supplies a leaf tag, and the decoder again ignores ℓ. An $n \geq 1$ computation also reaches an $\mathcal{R}^{+}_{\mathsf{msg}}$ point when closing chunk 0, but there the construction requests zero output and continues into the aggregation phase, so the $\mathcal{R}^{+}_{\mathsf{msg}}$ value is exposed as a tag only on the $n = 0$ path; since $n = \mathrm{leaves}(C)$ is fixed by $\|C\|$, the two uses never coincide on a single ciphertext. We call $\mathcal{R}_{\mathsf{init}}$, $\mathcal{R}^{+}_{\mathsf{ad}}$, $\mathcal{R}^{-}_{\mathsf{msg}}$, $\mathcal{L}_{\mathsf{init}}(j)$, and $\mathcal{L}^{-}_{\mathsf{msg}}(j)$ the *stream output points*: by the schedule just described, these are the only classes at which a construction computation requests keystream bytes, while tag values are exposed only at $\mathcal{L}_{\mathsf{tag}}(j)$ and $\mathcal{R}_{\mathsf{tag}}$ points and, on the $n = 0$ path, at $\mathcal{R}^{+}_{\mathsf{msg}}$.

The hidden-key encryption, verification, and mu-ind accounting all reindex transcript addresses by the key field. We record that reindexing once.

Lemma 9 (Key-Indexed Transcript Renaming)**.** *For a key $K \in \{0,1\}^k$, let $\mathcal{Y}_K$ be the set of bridged inputs $\mathsf{flat}(X)$ of well-formed TW128 transcript prefixes X whose first-call key field is K—the counted hidden-key transcript addresses of Table 12 for key K. Then:*

1. Disjointness. *For distinct keys $K_0 \neq K_1$, the sets $\mathcal{Y}_{K_0}$ and $\mathcal{Y}_{K_1}$ are disjoint.*

2. Renaming. *The map $\Phi_{K_0 \to K_1}$ that replaces the first-call key field K_0 by K_1 and leaves every other recovered field unchanged—nonce, leaf index, phase sequence, AD blocks, ciphertext blocks, aggregated leaf tag bytes, and root/leaf instance—induces through flat a bijection $\mathsf{flat} \circ \Phi_{K_0 \to K_1} \circ \mathsf{flat}^{-1} : \mathcal{Y}_{K_0} \to \mathcal{Y}_{K_1}$ preserving output-point classes and equality relations among addresses.*

3. Direct-query avoidance. *A direct query input $Y \in \mathcal{Y}_K$ has $\mathsf{KeyedTWOut}(Y) = (K, \mathcal{C})$ for a counted class $\mathcal{C}$; hence if no prior direct query decodes to K then $\mathcal{Y}_K$ is disjoint from all prior direct-query inputs.*

Proof. By bridge injectivity (Lemma 8) and Transcript Injectivity (Lemma 6), a bridged input $\mathsf{flat}(X)$ determines the native flattened prefix and hence the first duplex call $p_{\mathsf{root}} \| K \| U$ or $p_{\mathsf{leaf}} \| K \| U \| \langle j \rangle_8$, in particular its fixed-width key field K. *(1)* If $\mathsf{flat}(X) = \mathsf{flat}(X')$ with key fields K_0 and K_1 then $K_0 = K_1$, so distinct keys give disjoint address sets. *(2)* $\Phi_{K_0 \to K_1}$ rewrites only the key field, leaving the recovered duplex-call sequence otherwise intact, so it is a bijection on well-formed prefixes preserving the output-point class (Corollary 8) and the equality pattern among prefixes; flat transports it to the stated bijection on addresses. *(3)* By Lemma 8 a $Y = \mathsf{flat}(X) \in \mathcal{Y}_K$ decodes under $\mathsf{KeyedTWOut}$ to $(K, \mathcal{C})$; the contrapositive gives the avoidance claim. □

A.5 Adaptive Candidate Bounds

The Adaptive Candidate Collision and Preimage lemma is stated as Lemma 2 in Section 5, where a proof sketch also appears; this subsection supplies the full proof.

Proof of Lemma 2. Both parts expose the interaction in chronological order and sample each candidate's $\mathsf{RO}_{\mathsf{flat}}$ value lazily at its append step, which the premises license: when Y_j is appended its string is fixed by the prior state (predictability) and no bit of $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ has yet been revealed (unrevealed value), so $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ may be drawn then, uniform and independent of everything so far.

Collision. For a fixed position set $I = \{i_1 < \cdots < i_s\} \subseteq \{1, \ldots, L\}$, let E_I be the event that these candidate positions exist and their τ-bit projections are all equal. Condition on any positive-probability prefix immediately before position i_r is appended, for $r \geq 2$, and on the values sampled at the earlier positions in I. By the above, $\mathsf{RO}_{\mathsf{flat}}(Y_{i_r})$ is then uniform and independent of the conditioning, so its τ-bit projection equals the already fixed projection of Y_{i_1} with probability $2^{-\tau}$. Multiplying over $r = 2, \ldots, s$ gives $\Pr[E_I] \leq 2^{-\tau(s-1)}$; if some position in I is never appended then E_I is false and the bound still holds. A union bound over the $\binom{L}{s}$ position sets gives the claim.

Preimage. For a fixed position $j \in \{1, \ldots, L\}$, let E_j be the event that position j exists and $\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_j) \right\rfloor_\tau = t$. Condition on any positive-probability prefix immediately before Y_j is appended; the target t is a constant. By the above, $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ is uniform and independent of the conditioning, so its τ-bit projection equals t with probability $2^{-\tau}$. If position j is never appended, E_j is false. A union bound over the L positions gives the claim.

Second-preimage corollary. If the designated position j_0 is not appended, the event is false. Otherwise split the other candidates according to whether they occur before or after j_0. For earlier candidates, condition on any positive-probability prefix immediately before Y_{j_0} is appended, including the already sampled projections of candidates $1, \ldots, j_0 - 1$. The value $\mathsf{RO}_{\mathsf{flat}}(Y_{j_0})$ is then fresh and uniform, so its τ-bit projection hits one of those at most $j_0 - 1$ earlier projections with probability at most $(j_0 - 1)2^{-\tau}$. For a later candidate Y_j, $j > j_0$, condition on the full prefix immediately before it is appended, including the fixed projection $t_0 = \lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{j_0}) \rfloor_\tau$. The value $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ is fresh and uniform at its append step, so it hits t_0 with probability $2^{-\tau}$. A union bound over the at most $L - j_0$ later candidates gives an additional $(L - j_0)2^{-\tau}$. Adding the two sides gives $\Pr[\mathsf{sec}_{\tau,j_0}] \leq (L-1) \cdot 2^{-\tau}$; the designated candidate is excluded, so its trivially equal projection does not enter the count. $\square$

A.6 Adaptive Key Tests

Lemma 10 (Adaptive Key-Test Bound)**.** *Consider a* TW128 *random oracle model game with a hidden key K sampled uniformly from $\{0,1\}^k$, direct adversary access to* $\mathsf{RO}_{\mathsf{flat}}$*, and an immediate abort event* $\mathsf{bad}_{\mathsf{key}}$ *that fires on a direct query (Y, ℓ) exactly when Y decodes under* KeyedTWOut *to the candidate key K and a counted output-point class of Table 12. Suppose that, before the abort, the following invariant holds for every direct query position i: after conditioning on the full visible execution prefix and on the set S_i of distinct decoded candidate keys from earlier direct queries, the hidden K is uniform on $\{0,1\}^k \setminus S_i$. Then*

$$\Pr[\mathsf{bad}_{\mathsf{key}}] \leq \mathsf{KeyGuess}_k(q_{\mathsf{RO}}) = q_{\mathsf{RO}}/2^k,$$

where q_{RO} is the execution-uniform count of direct $\mathsf{RO}_{\mathsf{flat}}$ *queries.*

Proof. If $q_{\mathsf{RO}} \geq 2^k$ the right-hand side $q_{\mathsf{RO}}/2^k$ is at least one, so the claim is trivial. Assume $q_{\mathsf{RO}} < 2^k$.

Order the direct $\mathsf{RO}_{\mathsf{flat}}$ queries. By the definition of $\mathsf{KeyedTWOut}$ and Lemma 8, each query input either decodes to $\perp$ or decodes to a unique candidate key and counted output-point class. Queries that decode to $\perp$, and repeated queries whose decoded candidate key already lies in the current set S_i, do not change the set of possible first successful tests. Removing them can only shorten the sequence, so it suffices to analyze the subsequence of distinct fresh decoded candidate keys $K'_1, \ldots, K'_m$, where $m \le q_{\mathsf{RO}}$.

For a fixed position $i \in \{1, \ldots, q_{\mathsf{RO}}\}$, let E_i be the event that the i-th fresh key test occurs and its candidate key K'_i equals the hidden K. Because $\mathsf{bad}_{\mathsf{key}}$ aborts the game immediately, E_i implies that the first $i-1$ fresh tests all occurred and all missed; if some earlier fresh test never occurs then E_i is false and the bound below still holds, as in the proof of Lemma 2.

Fix $j \in \{0, \ldots, i-2\}$ and condition on any positive-probability visible prefix before the $(j+1)$-st fresh test on which the first j fresh tests occurred and missed. The invariant says that the hidden K is then uniform over the $2^k - j$ values not yet tested, and the next fresh candidate key K'_{j+1} is one of those remaining values and is determined by the adversary's next direct query, so the $(j+1)$-st fresh test misses with probability $1 - 1/(2^k - j)$. Chaining these conditional miss probabilities over the fixed steps $j = 0, \ldots, i-2$ telescopes to

$$\Pr[\text{the first } i-1 \text{ fresh tests occur and miss}] \le \prod_{j=0}^{i-2} \left(1 - \frac{1}{2^k - j}\right) = \prod_{j=0}^{i-2} \frac{2^k - j - 1}{2^k - j} = \frac{2^k - i + 1}{2^k}.$$

Conditioned on any such prefix, the i-th fresh test hits the hidden K with probability $1/(2^k - i + 1)$, so

$$\Pr[E_i] \le \frac{2^k - i + 1}{2^k} \cdot \frac{1}{2^k - i + 1} = \frac{1}{2^k}.$$

The first $\mathsf{bad}_{\mathsf{key}}$ abort occurs at a fresh key test, because $\mathsf{KeyedTWOut}$ returns either $\perp$ or a unique candidate key and a repeat of a previously tested candidate would already have fired the abort. Hence $\mathsf{bad}_{\mathsf{key}} \subseteq \bigcup_{i=1}^{q_{\mathsf{RO}}} E_i$, and a union bound over the q_{RO} positions gives $\Pr[\mathsf{bad}_{\mathsf{key}}] \le q_{\mathsf{RO}}/2^k$. □

A.7 Leaf Tag Collision Bound

Lemma 11 (Leaf Tag Collision Bound)**.** *Consider any TW128 random oracle model game with an adversary $\mathcal{B}$ that has direct access to $\mathsf{RO}_{\mathsf{flat}}$ and construction computations that may generate final leaf transcripts—by encryption, plaintext-computing, or verification computations, whether they accept or reject. Let $\mathsf{bad}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcript prefixes $X \ne X'$ in leaf tag output-point classes receive the same τ_{leaf}-bit projection from $\mathsf{RF}(\cdot)$, and let $n_{\mathsf{leaf}}(\mathcal{B})$ be the execution-uniform count of distinct construction-generated final leaf transcripts (Section 4.3). The first bound applies the collision case ($s = 2$, $\tau = \tau_{\mathsf{leaf}}$) of Lemma 2 to a chronological leaf tag candidate sequence; the second is a same-slot freshness argument in an all-reject game.*

1. General count. *Unconditionally—in particular when some or all keys are adversary-supplied, so direct queries may themselves land on leaf tag output points—*

$$\Pr[\mathsf{bad}_{\mathsf{leaf}}] \;\le\; \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}\big(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})\big) = \binom{q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})}{2} \Big/ 2^{\tau_{\mathsf{leaf}}}.$$

2. Hidden key, same slot. *Suppose the game has a hidden key, nonce-respecting encryption queries, and* all-reject *verification: every non-replay verification submission performs its construction computation and returns reject to the adversary. (The game $\mathsf{G}^{\mathsf{rj}}_d$ of Definition 5 and the INT-RUP continuation in the proof of Lemma 16 have this shape.) Say two final leaf transcripts* share a slot *if they recover the same*

key, nonce, and leaf index (K, U, j) (Corollary 8). Let $\mathsf{bad}_{\mathsf{leaf}}$ be the event that some construction-generated final leaf transcript receives the same τ_{leaf}-bit projection as a distinct final leaf transcript that shares its slot and is reached by an encryption computation, and let $\mathsf{hit}_{\mathsf{leaf}}$ be the event that a direct adversary query hits the bridged input of a construction-generated hidden-key final leaf transcript before that transcript is first created. Then, in such a game,

$$\Pr[\mathsf{bad}_{\mathsf{leaf}} \textit{ before } \mathsf{hit}_{\mathsf{leaf}}] \le \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}}.$$

Proof. Build a chronological leaf tag candidate sequence: a candidate is the bridged input of a leaf tag output point $(\mathcal{L}_{\mathsf{tag}}(j))$, appended the first time it is reached—by a direct query in $\mathcal{B}$'s oracle phase or by a construction computation generating a final leaf transcript—and not appended again. By Lemma 8, Corollary 8, and Lemma 6, two reaches of the same bridged input come from the same final leaf transcript, so repeated evaluations add no candidate—the lazy $\mathsf{RO}_{\mathsf{flat}}$ value is reused—and distinct final leaf transcripts give distinct candidates. No candidate's $\mathsf{RO}_{\mathsf{flat}}$ value is read before its append step, so the sequence meets the predictability and unrevealed-value premises of Lemma 2, whose collision case with $s = 2$ and $\tau = \tau_{\mathsf{leaf}}$ bounds the probability that two candidates share a τ_{leaf}-bit projection by $\binom{L}{2} \cdot 2^{-\tau_{\mathsf{leaf}}}$ for a length-L sequence. Every $\mathsf{bad}_{\mathsf{leaf}}$ pair is a pair of construction-generated candidates.

(1) General count. Direct queries may decode to a leaf tag class $\mathcal{L}_{\mathsf{tag}}(j)$ and are appended as candidates; together with the $n_{\mathsf{leaf}}(\mathcal{B})$ construction-generated transcripts the sequence has length at most $q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})$, so the case with $L = q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})$ gives $\Pr[\mathsf{bad}_{\mathsf{leaf}}] \le \binom{q_{\mathsf{RO}}(\mathcal{B})+n_{\mathsf{leaf}}(\mathcal{B})}{2} \cdot 2^{-\tau_{\mathsf{leaf}}} = \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B}))$. The count charges every direct query and every construction-generated final leaf transcript regardless of key provenance, so no assumption on the keys is used.

(2) Hidden key, same slot. Every $\mathsf{bad}_{\mathsf{leaf}}$ pair is a pair of construction-generated hidden-key transcripts sharing a slot, with at least one member reached by an encryption computation. Nonce-respecting encryption uses each nonce in at most one encryption query, and within one encryption computation the leaf index separates final leaf transcripts, so each slot contains at most one encryption-reached final leaf transcript. That transcript may have first been created by an earlier verification or plaintext-computing computation, when the encryption's absorbed leaf body reproduces an earlier absorbed body. Charge each possible $\mathsf{bad}_{\mathsf{leaf}}$ pair to its member that is not reached by an encryption computation. Each charged transcript has one slot, and that slot has at most one encryption-reached counterpart, so each charged transcript receives at most one charge. Thus at most $n_{\mathsf{leaf}}(\mathcal{B})$ charged pairs can occur.

Each pair's collision is decided at the first creation of its second-created member. At that moment the earlier member's τ_{leaf}-bit projection is a function of the prior execution, while the later member's projection is a fresh uniform lazy sample: by Transcript Injectivity (Lemma 6) two construction reaches of the same bridged input come from the same final leaf transcript, so no construction computation has read that bridged input before the creation, and a direct query reading it before the creation would decode under $\mathsf{KeyedTWOut}$ to the hidden key and its $\mathcal{L}_{\mathsf{tag}}(j)$ class (Lemma 8)—precisely $\mathsf{hit}_{\mathsf{leaf}}$. Hence before $\mathsf{hit}_{\mathsf{leaf}}$, conditioned on the entire prior execution, each pair collides on its τ_{leaf}-bit projections with probability $2^{-\tau_{\mathsf{leaf}}}$, and a union bound over the at most $n_{\mathsf{leaf}}(\mathcal{B})$ pairs gives the claim. No assumption on what the visible interface exposes is needed: a projection revealed after its sampling cannot alter a collision already decided. By Leaf-Transcript Recovery (Lemma 7), on $\neg\mathsf{bad}_{\mathsf{leaf}}$ equal flattened final-root inputs of an encryption computation and any construction computation identify the same leaf transcript sequence. $\square$

A.8 Encryption Transcript Marginal Uniformity

Lemma 12 (Encryption Transcript Marginal Uniformity)**.** *For any fixed nonce-respecting encryption query in the* TW128 *random oracle experiment, let* $\mathsf{hit}_{\mathsf{fresh}}$ *be the event that, before this query reaches some counted transcript point, a prior direct adversary query to* $\mathsf{RO}_{\mathsf{flat}}$ *has already hit the bridged input of that hidden-key point. On* $\neg\mathsf{hit}_{\mathsf{fresh}}$*, every transcript output of the query that contributes to a returned ciphertext block, to an internal leaf tag, or to the final root tag is either fresh when this query first generates its bridged input, or reuses the lazy-table value sampled uniformly when that input was first generated by an earlier construction computation. The counted output points first generated by the fixed query are pairwise distinct.*

Proof. Work on an execution outside $\mathsf{hit}_{\mathsf{fresh}}$ for the fixed query. This rules out prior adversary reads of the hidden-key construction inputs considered below. A previous construction computation may nevertheless have generated one of these inputs, for example through a non-replay verification query using the same nonce. Such a lookup samples the lazy-table value uniformly when it first occurs; if the fixed encryption query later reaches the same bridged input, it reuses that same value. Thus it remains only to show that the counted output points newly generated by the fixed encryption query are distinct from one another and from previous nonce-respecting encryption queries. Conditional information leaked by earlier verification comparisons of a final root tag is accounted for separately by Lemma 15, not by this lemma.

First root keystream block. The first root keystream block, when present, is produced by the root duplex after the initialization call $p_{\mathsf{root}} \parallel K \parallel U$ and any AD absorption (elided when $A = \varepsilon$, in which case the σ_{init}call itself emits the first root keystream block). Since the nonce-respecting adversary never repeats a nonce, no previous encryption query can have used the same root initialization transcript. Within the current query, Lemma 6 separates initialization, AD, message, and aggregation calls by their suffixes and by the duplex-call encoding, so the first root keystream transcript is distinct from the other counted output points generated by this query. If the requested keystream length is zero, no ciphertext block is returned at that point and there is nothing to prove for that output.

First leaf keystream block. For each leaf chunk $j \geq 1$, the first leaf keystream block is produced after the initialization call $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$. The prefix separates leaves from the root, the nonce separates this encryption query from all previous encryption queries, and $\langle j \rangle_8$ separates leaves within this query. Hence every first leaf keystream transcript newly generated by the query is distinct.

Later message outputs. Argue inductively within each root or leaf message stream. Suppose the current message-output transcript is either first generated by this query or reuses a value generated by an earlier construction computation. If its output is used as a keystream block, write its $\mathsf{RO}_{\mathsf{flat}}$ value as Z_i. Since the adaptive plaintext block M_i is a deterministic function of the prior view,

$$C_i = M_i \oplus Z_i$$

is now fixed. TW128 then absorbs C_i with the appropriate message phase suffix ($\sigma^{-}_{\mathsf{msg}}$ or $\sigma^{+}_{\mathsf{msg}}$) before requesting the next keystream block. Once C_i is fixed, the next transcript is fixed; Lemma 6's injectivity and domain separation imply it has not appeared earlier unless the same duplex-call prefix has appeared earlier. Nonce uniqueness excludes this across encryption queries. By Corollary 8, the next message-keystream output point can coincide with a within-query earlier output point only if both belong to the same output-point class on the same duplex instance (root or leaf-j with matching j) and have the same duplex-call sequence; the current message-stream prefix is strictly longer than every earlier prefix in that stream, so this is excluded. Across distinct instances within the same query (root vs leaf, or two leaves with distinct j), the prefixes already differ in the first call's σ by $p_{\mathsf{root}}/p_{\mathsf{leaf}}$ or $\langle j\rangle_8$ respectively. Each later message-output transcript that emits keystream is therefore distinct from the other points generated by this query, unless it is a reuse of a point generated by an earlier construction computation. The terminal message-output transcript of a leaf emits the leaf tag rather than a following keystream block; the same separation argument applies to that transcript, so each internal leaf tag newly generated by the query is sampled at a distinct point. Empty inputs still induce a duplex call, but with zero keystream output they contribute no ciphertext block.

Final root tag transcript. The final root tag is sampled at the root tag output point selected by $\|C\|$: the $\mathcal{R}_{\mathsf{tag}}$ point after the root duplex absorbs the aggregation transcript built from the leaf tags and $\langle n\rangle_8$ when $n \geq 1$, and the $\mathcal{R}^{+}_{\mathsf{msg}}$ point closing the root message phase when $n = 0$ (Section 3.4). When $n \geq 1$, the leaf tags are themselves $\mathsf{RO}_{\mathsf{flat}}$ outputs on final leaf transcripts; condition on their sampled values, on the sampled ciphertext, and on the adversary-visible prior view and $\mathsf{RO}_{\mathsf{flat}}$ table state, so that the root aggregation transcript is fixed. When $n = 0$ there is no aggregation transcript; condition on the sampled ciphertext and the adversary-visible prior view alone. By Corollary 8, the selected root tag output-point class is separated from every other output-point class in the same query — including root initialization (suffix σ_{init}), AD-phase outputs, the remaining message phase outputs, leaf-side output points (separated by p_{leaf}), and, in the $n \geq 1$ case, earlier $\mathcal{R}_{\mathsf{agg}}$-prefix output points (separated by the $\sigma^{-}_{\mathsf{agg}}$vs $\sigma^{+}_{\mathsf{agg}}$suffix). Even if two encryption queries realize the same aggregation transcript (or, for $n = 0$, the same root ciphertext), the full root transcript also contains the root initialization $p_{\mathsf{root}} \parallel K \parallel U$, and nonce uniqueness separates that initialization from all previous encryption queries. The final root tag transcript is therefore either newly generated at a distinct point of the fixed query, or is a reuse of a value first sampled by an earlier construction computation. In the newly generated case, its τ_{root}-bit $\mathsf{RO}_{\mathsf{flat}}$ output is a fresh lazy-table sample; in the reuse case, it is the same uniform sample drawn when the bridged input was first generated. □

A.9 Mu-Ind Hybrid Games and Verification Accounting

The direct mu-ind adjacent-hybrid proof bounds one user-replacement step of the hybrid through an explicit game sequence, defined next, using the generic key-test and leaf-collision accounting lemmas together with the verification-specific facts below. We use the root tag output point and final root transcript classes of Definition 3; in particular, computations on a common ciphertext share the same root tag output point because leaves(C) is fixed by $\|C\|$.

Definition 5 (Adjacent-Hybrid Games)**.** Fix a flat-RO mu-ind adversary $\mathcal{B}$, the pre-sampled keys $K_1, \ldots, K_D$ of Section 6.2, and a user index $d \in \{1, \ldots, D\}$. On the branch where two sampled keys collide, every game below outputs the same fixed bit; the remaining clauses describe the branch with pairwise distinct keys. Every game maintains one lazy table Θ implementing $\mathsf{RO}_{\mathsf{flat}}$: $\Theta[Y]$ samples a fresh uniform infinite string at the first access to Y and returns the stored string afterwards—equivalently, Θ is sampled in full at the outset and read on demand. Direct adversary queries (Y, ℓ) are answered by $\lfloor \Theta[Y] \rfloor_\ell$; the *environment* answers users $e < d$ by $(\mathsf{Rand}, \mathsf{Replay})$ and users $e > d$ by $(\mathsf{Enc}^{\mathsf{RO}}_{K_e}, \mathsf{Ver}^{\mathsf{RO}}_{K_e})$, reading construction outputs from Θ; and the New interface and the validity and nonce-respecting checks of Section 6.2 are common to every game. Every game maintains the flags $\mathsf{bad}_{\mathsf{key}}$, $\mathsf{bad}_{\mathsf{leaf}}$, and forge, initialized false; no oracle answer or branch depends on a flag, and $\mathsf{bad}_{\mathsf{key}}$ is set whenever a direct query input decodes under $\mathsf{KeyedTWOut}$ to K_d and a counted output-point class of Table 12.

Algorithm 4 Shared user-d oracle code of the games $\mathsf{G}_d^{\mathsf{re}}$ and $\mathsf{G}_d^{\mathsf{rj}}$ (Definition 5). Θ is the shared lazy $\mathsf{RO}_{\mathsf{flat}}$ table, read at bridged transcript prefixes, and W the user-d replay record. Inputs failing the validity or nonce-respecting checks of Section 6.2 are rejected beforehand, as in every game. The two games differ exactly in the marked lines 17 and 18.

1: **procedure** RO(Y, ℓ) ▷ direct adversary query
2: set $\mathsf{bad}_{\mathsf{key}}$ if $\mathsf{KeyedTWOut}(Y) = (K_d, \mathcal{C})$ for a counted class $\mathcal{C}$ (Table 12)
3: **return** $\lfloor \Theta[Y] \rfloor_\ell$
4: **end procedure**

5: **procedure** Enc(U, A, M)
6: run $\mathsf{Enc}_{K_d}^{\mathsf{RO}}(U, A, M)$, reading each requested duplex output as the corresponding projection of Θ at its bridged transcript prefix (Section 4.2); let the result be $C \parallel T$
7: set $\mathsf{bad}_{\mathsf{leaf}}$ if a final leaf transcript first created or first reached here completes a same-slot collision (Definition 5)
8: record $(U, A, C \parallel T)$ in W; **return** $C \parallel T$
9: **end procedure**

10: **procedure** Vf(U, A, $C \parallel T$)
11: **if** $(U, A, C \parallel T) \in W$ **then return** accept
12: **end if**
13: determine the verification transcript of (K_d, U, A, C); read from Θ the τ_{leaf}-bit leaf tags at its final leaf transcripts, setting $\mathsf{bad}_{\mathsf{leaf}}$ when a transcript first created here completes a same-slot collision, and nothing at its stream output points
14: $T' \leftarrow \lfloor \Theta[\mathsf{flat}(R)] \rfloor_{\tau_{\mathsf{root}}}$ at the root tag output point R of the computation (Definition 3)
15: **if** $T' = T$ **then**
16: $\mathsf{forge} \leftarrow \mathsf{true}$
17: **return** accept ▷ in $\mathsf{G}_d^{\mathsf{re}}$ only
18: **return** reject ▷ in $\mathsf{G}_d^{\mathsf{rj}}$ only
19: **end if**
20: **return** reject
21: **end procedure**

- Games $\mathsf{G}_d^{\mathsf{re}}$ and $\mathsf{G}_d^{\mathsf{rj}}$ answer user d by the shared oracle code of Algorithm 4 and differ exactly in the marked return statement: a non-replay verification whose comparison succeeds sets forge and returns accept in $\mathsf{G}_d^{\mathsf{re}}$ and reject in $\mathsf{G}_d^{\mathsf{rj}}$.

- Game $\mathsf{G}_d^{\mathsf{id}}$ answers user-d encryption queries with fresh uniform byte strings of length $\|M\| + \tau_{\mathsf{root}}/8$, recorded in W, and user-d verification by the replay check alone; it performs no user-d construction computation and sets neither $\mathsf{bad}_{\mathsf{leaf}}$ nor forge. $\mathsf{G}_d^{\mathsf{id}}$ is the hybrid $H_d^\star$ of Lemma 17, instrumented with $\mathsf{bad}_{\mathsf{key}}$.

The flag $\mathsf{bad}_{\mathsf{leaf}}$ is set, in $\mathsf{G}_d^{\mathsf{re}}$ and $\mathsf{G}_d^{\mathsf{rj}}$, when a user-$d$ construction computation first creates, or an encryption computation first reaches, a final leaf transcript whose τ_{leaf}-bit projection equals that of a distinct, already created final leaf transcript sharing its (K_d, U, j) slot, at least one of the two reached by an encryption computation—the same-slot event of Lemma 11, part 2, for user d. It guards no code difference and is used only in the accounting.

Lemma 13 (Environment Neutrality)**.** *In each game of Definition 5, on the branch with pairwise distinct keys, the environment's construction-internal* $\mathsf{RO}_{\mathsf{flat}}$ *lookups—made while* *simulating* $\mathsf{Enc}^{\mathsf{RO}}_{K_e}$ *and* $\mathsf{Ver}^{\mathsf{RO}}_{K_e}$ *for users* $e > d$*—neither set a user-*d *flag nor generate the bridged input of a user-*d *counted transcript point.*

Proof. Whenever such a lookup lies in a counted keyed transcript class, Lemma 8 decodes its candidate key as K_e, which on the distinct-keys branch differs from K_d; a lookup outside the counted keyed language is irrelevant to the flags. By the key-field disjointness of Lemma 9, the environment's addresses are disjoint from user d's. The flags are set only by direct adversary queries decoding to K_d ($\mathsf{bad}_{\mathsf{key}}$), by user-$d$ final leaf transcript creations ($\mathsf{bad}_{\mathsf{leaf}}$), and by user-$d$ verification comparisons (forge), none of which the environment performs. Ideal users $e < d$ perform no construction lookups at all. $\square$

Lemma 14 (Verification Visible-Prefix Key Uniformity)**.** *In the stopped single-key TW128 random oracle encryption/verification interaction, or its INT-RUP extension with a plaintext-computing oracle, with hidden key* K*, direct access to* $\mathsf{RO}_{\mathsf{flat}}$*, and immediate abort on* $\mathsf{bad}_{\mathsf{key}}$*, order the direct* $\mathsf{RO}_{\mathsf{flat}}$ *queries. For a direct query position* i*, let* S_i *be the set of distinct candidate keys decoded by* $\mathsf{KeyedTWOut}$ *from earlier direct-query inputs. Condition on any positive-probability visible execution prefix immediately before query* i*, on no earlier* $\mathsf{bad}_{\mathsf{key}}$*, and on* S_i*. Then the hidden key* K *is uniform on* $\{0,1\}^k \setminus S_i$*. Consequently the uniformity invariant required by Lemma 10 holds in the stopped interaction.*

Proof. The visible prefix contains the adversary's oracle queries and replies: direct $\mathsf{RO}_{\mathsf{flat}}$ outputs, encryption outputs and the resulting replay table, plaintext-computing replies when that oracle is present, verification bits, and the adversary state determined by these values. Fix such a prefix and two keys $K_0, K_1 \notin S_i$.

Let $\Phi_{0\to 1} = \Phi_{K_0 \to K_1}$ be the key-renaming map of Lemma 9 and $\Phi_{1\to 0}$ its inverse. By that lemma, $\mathsf{flat} \circ \Phi_{0\to 1} \circ \mathsf{flat}^{-1}$ is a bijection $\mathcal{Y}_{K_0} \to \mathcal{Y}_{K_1}$ between the bridged hidden-key construction addresses for K_0 and for K_1, preserving output-point classes and equality relations among addresses.

This bijection is applied online, not to a pre-existing fixed address set. Whenever a construction computation under K_0 would first query a hidden-key address Y, the coupled computation under K_1 queries $\Phi_{0\to 1}(Y)$ and receives the same lazy-table value; repeated queries are coupled consistently through the same map. If the queried address is a root aggregation address containing leaf tag bytes, those bytes are already coupled equal because the corresponding leaf tag lookups were either repeated or first sampled earlier under the same renaming rule. Thus adaptively generated message transcripts and root aggregation transcripts are renamed step by step with the same visible ciphertext, leaf tag, and root tag bytes.

Prior direct-query inputs are not renamed: they are adversary-chosen visible strings. Since $K_0, K_1 \notin S_i$, neither $\mathcal{Y}_{K_0}$ nor $\mathcal{Y}_{K_1}$ contains a prior direct-query input (Lemma 9, direct-query avoidance). Direct-query lazy-table values are therefore coupled identically and remain disjoint from the renamed hidden construction addresses.

Use the same adversary coins, the same direct-query lazy-table values, and the online renamed construction-internal lazy-table values. Encryption outputs have the same distribution under the coupling, because every construction lookup that determines ciphertext, leaf tags, or the final root tag is paired with a distinct renamed lookup carrying the same sampled value. Plaintext-computing computations are renamed in the same online way: a plaintext reply is the submitted ciphertext block XORed with a coupled keystream projection at a stream output point (Table 12), so it carries the same visible bytes under both keys, and the recomputed tag is not returned. Replay verification decisions depend only on the visible replay table. Non-replay verification computations are renamed in the same online way as encryption computations, and the final root tag comparison therefore gives the same accept/reject bit. Verification does not reveal plaintext on rejection or acceptance; it reveals only that bit.

Thus every visible prefix with hidden key $K_0 \notin S_i$ has the same conditional probability as the corresponding execution with hidden key $K_1 \notin S_i$. Since K is initially uniform and the condition "no earlier $\mathsf{bad}_{\mathsf{key}}$" removes exactly the keys in S_i, all keys in $\{0,1\}^k \setminus S_i$ remain equally likely. $\square$

Foreign-key key-test claim. In each game of Definition 5, work in the original experiment, where the key-collision branch outputs a fixed bit rather than conditioning on pairwise distinct keys. Then

$$\Pr[\mathsf{bad}_{\mathsf{key}}] \leq \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})).$$

Fix the foreign keys and leave their transcript addresses unchanged. Before the first user-d key hit, the visible prefix is invariant under renaming K_d among keys that have not already been tested and are not equal to a fixed foreign key, by the same online coupling as Lemma 14 and the key-field disjointness of Lemma 9. For each fresh candidate key tested by one of $\mathcal{B}$'s direct $\mathsf{RO}_{\mathsf{flat}}$ queries, the contributing event is contained in $\{K_d$ equals that candidate$\}$ and costs at most 2^{-k} in the original pre-sampled-key experiment; a candidate equal to a fixed foreign key contributes only on the fixed-output key-collision branch and therefore contributes nothing to $\mathsf{bad}_{\mathsf{key}}$. A union bound over at most $q_{\mathsf{RO}}(\mathcal{B})$ direct queries gives the claim.

Lemma 15 (Verification-First Root Guessing)**.** *In the game $\mathsf{G}_d^{\mathsf{rj}}$ of Definition 5, stopped at the first occurrence of $\mathsf{bad}_{\mathsf{key}} \vee \mathsf{bad}_{\mathsf{leaf}}$—with $\mathsf{bad}_{\mathsf{leaf}}$ the same-slot event of Lemma 11, part 2, for user d—let $\mathsf{win}_{\mathsf{vf}}$ be the event that some non-replay user-d verification comparison succeeds within a verification-first final root transcript class. Then*

$$\Pr[\mathsf{win}_{\mathsf{vf}}] \leq q_{\mathsf{v}}^{(d)}/2^{\tau_{\mathsf{root}}},$$

where $q_{\mathsf{v}}^{(d)}$ is the user-d non-replay verification cap of Section 6.2.

Proof. Throughout, a *non-replay verification submission* means a valid non-replay user-d verification query; by the validity convention, an invalid verification query returns 0 and performs no construction computation, so it realizes no bridged final root input, belongs to no final root transcript class, and contributes to no $G_{\mathcal{C}}$ below.

In $\mathsf{G}_d^{\mathsf{rj}}$, every non-replay submission performs its construction computation and returns reject (Algorithm 4), so no comparison outcome is visible to the adversary, and each class's root tag value is read from the lazy table only by the class's own comparisons and by any later encryption computation reaching the class.

Fix a verification-first final root transcript class $\mathcal{C}$ and follow the stopped execution until the first encryption query generating the same class, if any (the cutoff). Let $G_{\mathcal{C}}$ be the set of distinct candidate tags submitted by non-replay verification queries in class $\mathcal{C}$ before that cutoff; repeated tags only shrink this set. The class's correct root tag $U_{\mathcal{C}}$ is the τ_{root}-bit projection of $\mathsf{RO}_{\mathsf{flat}}$ at the class's bridged root tag input. Before the stop, no direct query has read this hidden-key input: such a query would be decoded by $\mathsf{KeyedTWOut}$ to the hidden key and would set $\mathsf{bad}_{\mathsf{key}}$; the environment reads no user-d address (Lemma 13). Before the cutoff, no encryption query has revealed $U_{\mathcal{C}}$ as an encryption tag. (When the class's root tag point is $\mathcal{R}^+_{\mathsf{msg}}$, i.e. $n = 0$, an encryption query on a longer message that shares chunk 0 may touch this same address while closing its chunk-0 message phase, but it requests zero output there and continues into its aggregation phase, so it does not reveal $U_{\mathcal{C}}$; its own tag is emitted at a later $\mathcal{R}_{\mathsf{tag}}$ point.) The comparisons against $U_{\mathcal{C}}$ return the constant reject, so the visible execution and the set $G_{\mathcal{C}}$ are functions only of the adversary's coins, visible oracle replies, and lazy-table values at inputs other than this class's root tag address. By the lazy-sampling order exchange, conditioned on those values, $U_{\mathcal{C}}$ may equivalently be sampled after $G_{\mathcal{C}}$ is fixed, and is uniform independently of $G_{\mathcal{C}}$. The probability that some pre-cutoff comparison in class $\mathcal{C}$ succeeds—that $U_{\mathcal{C}} \in G_{\mathcal{C}}$—is therefore at most $|G_{\mathcal{C}}|/2^{\tau_{\mathsf{root}}}$.

After the cutoff, if it exists, class $\mathcal{C}$ admits no successful non-replay comparison. Let the cutoff encryption query return (U_e, A_e, C_e, T_e). Any later verification submission in the same final root transcript class has, by the final-root determinacy of Lemma 6 and Leaf-Transcript Recovery (Lemma 7) under the absence of $\mathsf{bad}_{\mathsf{leaf}}$—which supplies the recovery hypothesis because the cutoff computation is an encryption computation—the same (U, A, C) as that encryption computation. If the submitted tag is T_e, the tuple is a recorded encryption tuple and the query is answered by the replay check before any comparison; if the submitted tag is not T_e, the comparison fails.

Each non-replay verification submission deterministically reconstructs one final root transcript at the root tag output point, so it contributes its submitted tag to at most one set $G_{\mathcal{C}}$, and only when its class is verification-first and the submission precedes that class's cutoff. Thus $\sum_{\mathcal{C}} |G_{\mathcal{C}}| \le q_{\mathsf{v}}^{(d)}$, and a union bound over the verification-first classes gives the claim. ☐

Lemma 16 (INT-RUP Root Guessing)**.** *Let $\mathcal{B}$ be a single-key INT-RUP adversary in the TW128 random oracle model, with nonce-respecting encryption queries and the resources of Section 4.3. In its encryption/plaintext-computing/verification interaction with hidden key K, let $\mathsf{bad}_{\mathsf{key}}$ be the event that some direct $\mathsf{RO}_{\mathsf{flat}}$ query input Y satisfies $\mathsf{KeyedTWOut}(Y) = (K, \mathcal{C})$ for a counted class $\mathcal{C}$ of Table 12, and let $\mathsf{bad}_{\mathsf{leaf}}$ be the same-slot event of Lemma 11, part 2, over the construction computations generated by encryption, plaintext-computing, and verification. Let forge be the event that some valid non-replay verification query accepts. Then*

$$\Pr[\mathsf{forge} \textit{ before } \mathsf{bad}_{\mathsf{key}} \vee \mathsf{bad}_{\mathsf{leaf}}] \le q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Proof. Throughout, a *non-replay verification submission* means a valid non-replay verification query; invalid verification queries return 0, perform no construction computation, realize no bridged final root input, and contribute to no set of guessed tags.

Work in the game stopped at the first occurrence of $\mathsf{bad}_{\mathsf{key}}$ or $\mathsf{bad}_{\mathsf{leaf}}$. A final root transcript class is *tag-revealed* once an encryption query has returned the root tag for that class. If a non-replay verification submission belongs to a tag-revealed class, compare it with an encryption computation that revealed the class. The two computations' final-root transcript prefixes are well-formed typed transcript prefixes with the same bridged final root input; by Lemma 8, they have the same native flattened input. Then final-root determinacy (Lemma 6) and Leaf-Transcript Recovery (Lemma 7) under $\neg\mathsf{bad}_{\mathsf{leaf}}$—which supplies the recovery hypothesis because the comparand is an encryption computation—recover the same (U, A, C). If the submitted tag is the revealed tag, the tuple is a replay-table hit; if it is any other tag, the final comparison rejects. Thus any accepting non-replay verification submission must lie in a class whose root tag has not yet been revealed by encryption.

For tag-unrevealed classes, define an all-reject continuation: encryption and plaintext-computing queries are answered normally, replay verification queries accept normally, and each non-replay verification submission performs the same construction computation but returns reject to the adversary. Plaintext-computing queries may generate the same stream and leaf tag transcripts as decryption, but they do not reveal the final root tag value. Equivalently, the final root tag sample for a tag-unrevealed class can be deferred until it is first needed for an encryption answer or a verification comparison; this does not change any plaintext reply. By Output-Point Separation (Corollary 8) and the keystream schedule recorded at Table 12, the keystream values used to compute returned plaintext blocks are projections at stream output points, not at the final root tag output point; in particular the chunk-0 $\mathcal{R}^{+}_{\mathsf{msg}}$ call of an $n \ge 1$ computation requests zero output, so no plaintext reply exposes an $\mathcal{R}^{+}_{\mathsf{msg}}$ tag value.

Fix a final root transcript class $\mathcal{C}$ whose first construction generation is not by an encryption query; for a class first generated by an encryption query the tag is revealed from that generation onward, so every verification submission in it is covered by the previous paragraph. Follow the all-reject path until the first encryption query that reveals the class, if any (the cutoff; any such query is later than the class's first generation). Let $G_{\mathcal{C}}$ be the set of distinct tags submitted by non-replay verification queries in class $\mathcal{C}$ before that cutoff. Before the cutoff, no encryption query has returned the class tag and no direct query has read the hidden-key root tag point without triggering $\mathsf{bad}_{\mathsf{key}}$. Conditioned on the all-reject path and on all lazy-table values except this final root tag projection, the correct tag for $\mathcal{C}$ is uniform in $\{0,1\}^{\tau_{\mathsf{root}}}$ and independent of $G_{\mathcal{C}}$. The probability that any pre-cutoff submitted tag in $G_{\mathcal{C}}$ equals the deferred class tag is therefore at most $|G_{\mathcal{C}}|/2^{\tau_{\mathsf{root}}}$.

Consider the event that the first accepting non-replay verification submission in the stopped game belongs to class $\mathcal{C}$. Up to that global first accept, the adversary's visible answers match the all-reject continuation: every earlier non-replay submission rejects, replay and plaintext-computing queries are answered identically in both executions, and encryption answers are unchanged. The construction computations also agree at every address except that the continuation defers the final root tag value for class $\mathcal{C}$ until the comparison is needed. Therefore the first accepting non-replay submission can belong to class $\mathcal{C}$ only before the cutoff, and only if its submitted tag equals the deferred class tag, which has probability at most $|G_{\mathcal{C}}|/2^{\tau_{\mathsf{root}}}$.

The execution-uniform count $q_{\mathsf{v}}(\mathcal{B})$ applies to the all-reject continuation. Each valid non-replay verification query contributes its submitted tag to at most one such set $G_{\mathcal{C}}$, and replay-table hits do not contribute. Thus $\sum_{\mathcal{C}} |G_{\mathcal{C}}| \le q_{\mathsf{v}}(\mathcal{B})$, and a union bound over the classes not first generated by an encryption query, each with its pre-cutoff guess set $G_{\mathcal{C}}$, gives the claim. □

A.10 Mu-Ind Adjacent-Hybrid Lemma

The multi-user indistinguishability theorem (Theorem 8) telescopes a hybrid over the D users, replacing one real user by its ideal $(\mathsf{Rand}, \mathsf{Replay})$ interface at each step. The following lemma bounds one such step through the game sequence of Definition 5: the instrumented real game $\mathsf{G}_d^{\mathsf{re}}$, the all-reject game $\mathsf{G}_d^{\mathsf{rj}}$ differing from it in a single return statement, and the ideal game $\mathsf{G}_d^{\mathsf{id}}$, with the divergence events set as the flags $\mathsf{bad}_{\mathsf{key}}$, $\mathsf{bad}_{\mathsf{leaf}}$, and forge by game code.

Lemma 17 (Mu-Ind Adjacent-Hybrid Bound)**.** *Let $\mathcal{B}$ be a flat-RO mu-ind adversary in the pre-sampled-key formulation of Section 6.2, with keys $K_1, \ldots, K_D$ sampled at the start of the experiment. Fix $d \in \{1, \ldots, D\}$. Let $H^\star_{d-1}$ and $H^\star_d$ be the adjacent stopped hybrids that output a fixed bit if any two sampled keys collide, and otherwise run $\mathcal{B}$ with one shared lazy $\mathsf{RO}_{\mathsf{flat}}$ table as follows: users $e < d$ are answered by $(\mathsf{Rand}, \mathsf{Replay})$, users $e > d$ are answered by $(\mathsf{Enc}^{\mathsf{RO}}_{K_e}, \mathsf{Ver}^{\mathsf{RO}}_{K_e})$, and user d is answered by $(\mathsf{Enc}^{\mathsf{RO}}_{K_d}, \mathsf{Ver}^{\mathsf{RO}}_{K_d})$ in $H^\star_{d-1}$ and by $(\mathsf{Rand}, \mathsf{Replay})$ in $H^\star_d$. If $n^{(d)}_{\mathsf{leaf}}$ and $q^{(d)}_{\mathsf{v}}$ are the user-d resource caps, then*

$$\left|\Pr[\mathcal{B}^{H^\star_{d-1}} \Rightarrow 1] - \Pr[\mathcal{B}^{H^\star_d} \Rightarrow 1]\right| \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n^{(d)}_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{q^{(d)}_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. The fixed-output branch on a key collision is identical in $H^\star_{d-1}$, $H^\star_d$, and the games of Definition 5, so only the branch with pairwise distinct keys contributes; work on that branch, with the keys fixed, throughout. The proof identifies $H^\star_{d-1}$ with $\mathsf{G}_d^{\mathsf{re}}$ (Claim 1), relates $\mathsf{G}_d^{\mathsf{re}}$ to $\mathsf{G}_d^{\mathsf{rj}}$ pointwise (Claim 2) and $\mathsf{G}_d^{\mathsf{rj}}$ to $H^\star_d = \mathsf{G}_d^{\mathsf{id}}$ in distribution (Claim 3), and assembles the bound by a single first-occurrence decomposition in $\mathsf{G}_d^{\mathsf{rj}}$ (Claim 4).

Claim 1: $\mathsf{G}_d^{\mathsf{re}}$ realizes $H^\star_{d-1}$. The user-d oracles of Algorithm 4 implement $(\mathsf{Enc}^{\mathsf{RO}}_{K_d}, \mathsf{Ver}^{\mathsf{RO}}_{K_d})$ with three distribution-preserving refactors. First, reading every construction output as a projection of the shared table Θ at its bridged transcript prefix is the lazy implementation of $\mathsf{RO}_{\mathsf{flat}}$ (Section 4.2); whether Θ is sampled in full at the outset or entry by entry on first access is immaterial. Second, VF reads Θ only at leaf tag and root tag output points. This is sound because decryption absorbs the submitted ciphertext, so the verification transcript is determined by (K_d, U, A, C) independently of any stream output (Lemma 5, Section 3.5), and verification discards the recovered plaintext, so the keystream projections a real $\mathsf{Ver}^{\mathsf{RO}}_{K_d}$ computation reads influence neither a visible answer nor the value of any other table entry; leaving those entries unread until some later computation first accesses them leaves the joint distribution of all oracle answers unchanged. Third, the flags are instrumentation: no oracle answer or branch depends on them. Hence $\Pr[\mathcal{B}^{\mathsf{G}^{\mathsf{re}}_d} \Rightarrow 1] = \Pr[\mathcal{B}^{H^\star_{d-1}} \Rightarrow 1]$.

Claim 2: $\mathsf{G}^{\mathsf{re}}_d$ and $\mathsf{G}^{\mathsf{rj}}_d$ agree until forge. Run the two games on the same adversary coins, environment coins, and table realization Θ. The games share all code, including the comparison that sets forge, and differ in exactly one statement, the marked return line of Algorithm 4, which executes only when forge has just been set. The two executions are therefore identical—every oracle answer, table access, record in W, and flag update—up to and including the query at which forge is first set, and the first differing value is that query's returned bit. In particular, on executions in which $\mathsf{G}^{\mathsf{rj}}_d$ never sets forge, the two runs coincide, outputs included, and the flags $\mathsf{bad}_{\mathsf{key}}$ and $\mathsf{bad}_{\mathsf{leaf}}$ are set at the same queries in both games up to the first forge.

Claim 3: $\mathsf{G}^{\mathsf{rj}}_d$ agrees with $H^\star_d$ until $\mathsf{bad}_{\mathsf{key}}$. Couple $H^\star_d = \mathsf{G}^{\mathsf{id}}_d$ to $\mathsf{G}^{\mathsf{rj}}_d$ as follows: run $\mathsf{G}^{\mathsf{rj}}_d$ on the same adversary and environment coins, give the $\mathsf{G}^{\mathsf{id}}_d$ coordinate the same visible oracle answers for every query answered while $\mathsf{bad}_{\mathsf{key}}$ is unset, and let the $\mathsf{G}^{\mathsf{id}}_d$ coordinate continue independently according to its own law from the query that sets $\mathsf{bad}_{\mathsf{key}}$ onward. We claim the resulting $\mathsf{G}^{\mathsf{id}}_d$ coordinate is distributed exactly as $\mathsf{G}^{\mathsf{id}}_d$. The answer rules coincide query by query except for user-d encryption: non-replay user-d verification returns reject in $\mathsf{G}^{\mathsf{rj}}_d$ and under Replay alike, and replay queries are answered from the record W, which the coupling keeps equal in both coordinates; the environment runs identical code in both games; and direct queries read table entries that agree in both games before $\mathsf{bad}_{\mathsf{key}}$—an entry written by a user-d construction computation is read by a direct query only if that query decodes under $\mathsf{KeyedTWOut}$ to K_d and a counted class (Lemma 9), which sets $\mathsf{bad}_{\mathsf{key}}$, while entries written by the environment are produced identically in both games (Lemma 13).

It remains to show that, before $\mathsf{bad}_{\mathsf{key}}$, each user-$d$ encryption answer of $\mathsf{G}^{\mathsf{rj}}_d$ is, conditioned on the entire prior visible transcript, a fresh uniform byte string of length $\|M\| + \tau_{\mathsf{root}}/8$—the law of Rand. Fix such a query (U, A, M). Before $\mathsf{bad}_{\mathsf{key}}$, no direct query has read the bridged input of a user-d hidden-key counted point, so the hypothesis $\neg\mathsf{hit}_{\mathsf{fresh}}$ of Encryption Transcript Marginal Uniformity (Lemma 12) holds, and every counted output point first generated by this query is pairwise distinct and receives a fresh uniform sample. The stream output points are always first generated by this query: user-d verification computations read no stream points (Algorithm 4), nonce-respecting encryption and Transcript Injectivity (Lemma 6) separate them from every earlier user-d encryption computation, and the environment writes only foreign-key addresses (Lemma 13). The ciphertext body C is therefore a fresh uniform string. The internal leaf tags are either fresh samples or reuse entries written by user-d verification computations; they are never returned and influence the answer only through the root tag address. The root tag T is read at the computation's root tag output point (Definition 3), an address distinct from every stream point of the same query by Output-Point Separation (Corollary 8). If that entry is first generated here, T is a fresh uniform sample. Otherwise the entry was written by an earlier user-d verification computation—before $\mathsf{bad}_{\mathsf{key}}$, the only other writer of user-d addresses—whose reads of it produced only the constant reject answers of $\mathsf{G}^{\mathsf{rj}}_d$; since no oracle answer or branch depends on the flags, the visible transcript is a function of the coins and of table entries other than this one, and conditioned on the visible transcript the entry remains uniform. In either case (C, T) has the Rand law, and at most one user-d encryption reaches any root tag address, since two would share the nonce U, contradicting nonce-respecting encryption. By induction over the query sequence, the coupled $\mathsf{G}_d^{\mathsf{id}}$ coordinate has exactly the law of $H_d^\star$, and its visible transcript agrees with $\mathsf{G}_d^{\mathsf{rj}}$'s while $\mathsf{bad}_{\mathsf{key}}$ is unset.

Claim 4: assembly. Place the three games on one probability space: shared adversary and environment coins and one table realization Θ drive $\mathsf{G}_d^{\mathsf{re}}$ and $\mathsf{G}_d^{\mathsf{rj}}$ pointwise, and the $H_d^\star$ coordinate is coupled to $\mathsf{G}_d^{\mathsf{rj}}$ as in Claim 3. Until the first occurrence of $\mathsf{forge} \vee \mathsf{bad}_{\mathsf{key}}$ in $\mathsf{G}_d^{\mathsf{rj}}$, the three visible transcripts coincide: $\mathsf{G}_d^{\mathsf{re}}$'s by Claim 2, since forge has not yet been set, and $H_d^\star$'s by Claim 3, since $\mathsf{bad}_{\mathsf{key}}$ has not. When neither flag is ever set, the final outputs of $\mathcal{B}$ coincide as well, so the outputs can differ only on the event $\mathsf{forge} \vee \mathsf{bad}_{\mathsf{key}}$. With Claim 1 identifying $\Pr[\mathcal{B}^{H_{d-1}^\star} \Rightarrow 1] = \Pr[\mathcal{B}^{\mathsf{G}_d^{\mathsf{re}}} \Rightarrow 1]$ and Claim 3 identifying $\Pr[\mathcal{B}^{H_d^\star} \Rightarrow 1]$ with the coupled coordinate's output probability,

$$\left|\Pr[\mathcal{B}^{H_{d-1}^\star} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^\star} \Rightarrow 1]\right| \;\le\; \Pr_{\mathsf{G}_d^{\mathsf{rj}}}[\mathsf{forge} \vee \mathsf{bad}_{\mathsf{key}}].$$

Decompose by first occurrence, inserting $\mathsf{bad}_{\mathsf{leaf}}$—which guards no code difference and enters only this accounting:

$$\begin{aligned}\Pr_{\mathsf{G}_d^{\mathsf{rj}}}[\mathsf{forge} \vee \mathsf{bad}_{\mathsf{key}}] \;\le\; & \Pr[\mathsf{bad}_{\mathsf{key}}] + \Pr[\mathsf{bad}_{\mathsf{leaf}} \text{ before } \mathsf{bad}_{\mathsf{key}} \vee \mathsf{forge}] \\ & + \Pr[\mathsf{forge} \text{ before } \mathsf{bad}_{\mathsf{key}} \vee \mathsf{bad}_{\mathsf{leaf}}].\end{aligned}$$

The three terms are bounded in $\mathsf{G}_d^{\mathsf{rj}}$. For the first, the foreign-key key-test claim above gives $\Pr[\mathsf{bad}_{\mathsf{key}}] \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B}))$. For the second, $\mathsf{G}_d^{\mathsf{rj}}$ restricted to user d is a hidden-key game with nonce-respecting encryption and all-reject verification; before $\mathsf{bad}_{\mathsf{key}}$ no direct query has read a user-d hidden final leaf point—such a query would decode to K_d and its $\mathcal{L}_{\mathsf{tag}}(j)$ class and set $\mathsf{bad}_{\mathsf{key}}$—and the environment creates no user-d leaf transcript (Lemma 13), so the same-slot case of Lemma 11 gives $\Pr[\mathsf{bad}_{\mathsf{leaf}} \text{ before } \mathsf{bad}_{\mathsf{key}} \vee \mathsf{forge}] \le n_{\mathsf{leaf}}^{(d)}/2^{\tau_{\mathsf{leaf}}}$. For the third, before $\mathsf{bad}_{\mathsf{key}} \vee \mathsf{bad}_{\mathsf{leaf}}$ every successful comparison lies in a verification-first class: a class first generated by a user-d encryption computation admits none, since by the final-root determinacy of Lemma 6 and Leaf-Transcript Recovery (Lemma 7)—whose hypothesis $\neg\mathsf{bad}_{\mathsf{leaf}}$ supplies because the comparand is the class's encryption computation—any non-replay submission in the class carries the same (U, A, C) as that encryption and either matches the recorded tuple, in which case the replay check answers it before any comparison, or fails the comparison. Lemma 15 then gives $\Pr[\mathsf{forge} \text{ before } \mathsf{bad}_{\mathsf{key}} \vee \mathsf{bad}_{\mathsf{leaf}}] \le q_{\mathsf{v}}^{(d)}/2^{\tau_{\mathsf{root}}}$.

Summing the three bounds gives

$$\left|\Pr[\mathcal{B}^{H_{d-1}^\star} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^\star} \Rightarrow 1]\right| \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n_{\mathsf{leaf}}^{(d)}}{2^{\tau_{\mathsf{leaf}}}} + \frac{q_{\mathsf{v}}^{(d)}}{2^{\tau_{\mathsf{root}}}},$$

with the user-d key-test event $\mathsf{bad}_{\mathsf{key}}$ charged a single time for the adjacent hybrid. □

B The Flat Sponge Lift

This appendix supplies the flat sponge lift machinery (Lemmas 20 and 22 and Corollary 9) invoked by the tag hash bounds of Section 5, the committing consequences of Corollary 5, and the baseline AEAD bounds of Section 6. The machinery applies uniformly to the games named once here.

The lift has two steps in the win-event games and three in the real-vs-ideal games. First, the [BDPVA08] indifferentiability theorem replaces the real permutation used by the TW128 construction and by direct primitive queries with the simulator over a flat random oracle, at cost $\mathsf{Lift}_c(N_{\mathsf{tot}})$, where $N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$ bounds the operational primitive trace: construction-internal duplex calls induced by construction queries and deterministic challenger computations, together with direct primitive queries. The simulator-side construction trace is then identified with the flat-RO construction interface by the operational trace lemma below. Second, the simulator-world adversary is converted into a flat-RO adversary whose direct random oracle query count is at most N_{prim}. Thus the random oracle bounds are instantiated at $q_{\mathsf{RO}} \le N_{\mathsf{prim}}$, while the public permutation lift pays $\mathsf{Lift}_c(N + N_{\mathsf{prim}})$. Third, in the real-vs-ideal games, whose ideal world exposes the real permutation rather than the simulator (Section 4.1), the ideal-side simulator introduced by the first step is exchanged back for (π, π^{-1}) at cost $\mathsf{Lift}_c(N_{\mathsf{prim}})$ (Lemma 21); the exchange is possible because the ideal construction oracles are independent of the primitive.

Definition 6 (Single-Stage TW128 Games)**.** The *single-stage TW128 games* are the public-permutation games of Sections 5.1, 6.1 and 6.2 in which one adversary interacts with the challenger in a single stage, so that a [BDPVA08] replacement applies to an entire world's public-permutation transcript at once, including deterministic challenger checks after the adversary's final output. They are the $\mathsf{H}_{\mathsf{TW128}}$ tag hash games (collision and preimage games, standard and everywhere), CMTDs-4, CMTs-ℓ for $\ell \in \{1, 3, 4\}$, INT-RUP, all-in-one AE, and mu-ind. The win-event games among these have one world; the real-vs-ideal games—all-in-one AE and mu-ind—have two, and the lift below treats their real world by the full replacement and their ideal world by the simulator–permutation exchange of Lemma 21.

The derived adversary forwards the following game data to the corresponding flat-RO experiment.

Game family	Forwarded data
$\mathsf{H}_{\mathsf{TW128}}$ tag hash games	final structured hash game output
CMTDs / CMTs	final openings and deterministic challenger checks
AE / mu-ind / INT-RUP	construction queries, including New where present and replay state

B.1 Simulator Convention

Throughout this appendix, $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ denotes the public primitive simulator supplied by [BDPVA08, Theorem 2] — the indifferentiability theorem for the simple padded sponge over a random permutation — after applying the [BDPVA11, Theorem 3] bridge of Lemma 8 to TW128's native pad10*1 transcript inputs. The superscript records that both simulator directions share the same internal lazy $\mathsf{RO}_{\mathsf{flat}}$ table.

The simulator is a proof device: each public-permutation game of Sections 5.1, 6.1 and 6.2 states its advantage over the real permutation, and the reduction to the corresponding random oracle model adversary passes through intermediate simulator worlds. In those worlds, the construction interface is the one specified by the game ($(\mathsf{Rand}, \mathsf{Replay})$, instantiated per user in mu-ind, on the ideal side) and the public primitive interface is $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$. The $\mathsf{RO}_{\mathsf{flat}}$ table in this notation is internal to that system and is not an extra interface exposed to the application adversary.

Lemma 18 (At Most One Simulator RO Call)**.** *For the simulator $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$, every primitive-interface query induces at most one simulator-side $\mathsf{RO}_{\mathsf{flat}}$ call. More precisely, every inverse primitive-interface query and every forward primitive-interface query violating one of the following conditions induces no simulator $\mathsf{RO}_{\mathsf{flat}}$ call. The symbols s, p, $\mathcal{R}$, $\mathcal{O}$,* *and $\mathcal{C}$ are the local internal notation of the BDPV simple-padded sponge simulator, not* TW128 *resource notation:*

(C1) New edge. *The simulator's internal graph has no outgoing edge from the input node* s*.*

(C2) Unsaturated. $\mathcal{R} \cup \mathcal{O} \neq \mathcal{C}$*.*

(C3) Rooted input. *The input node is rooted.*

(C4) Paddable path. *The constructed path* p *decomposes uniquely as* $p = p' \parallel 0^{rj}$ *with* p' *not ending in* 0^r*, and the prefix* p' *is a valid padded sponge input: equivalently,* $p' = x \parallel \mathsf{pad10}^*[r](|x|)$ *for an unpadded BDPV08 random oracle input* x*.*

When all four conditions hold, the forward primitive-interface query induces one simulator $\mathsf{RO_{flat}}$ *call. Consequently, each primitive-interface query induces at most one simulator-side* $\mathsf{RO_{flat}}$ *call.*

Proof. This is the branch structure of the [BDPVA08] simple-padded permutation simulator. The simulator invokes the random oracle only on the forward-query branch where the input node is rooted, unsaturated, paddable to an unpadded random oracle input x, and not already extended by an outgoing edge in the simulator graph. Forward queries violating any of these conditions and all inverse queries return outputs sampled or read from the simulator's internal graph without invoking the random oracle. Under the [BDPVA11, Theorem 3] bridge fixed in Section B.1, that simulator random oracle invocation is the simulator-side $\mathsf{RO_{flat}}$ call used throughout this appendix. ☐

Lemma 19 (Operational Duplex Trace)**.** *In any execution of a single-stage* TW128 *game (Definition 6), the construction computations counted by* N *admit an operational primitive trace of cost at most* N*: each root or leaf duplex call contributes one forward primitive query, and the requested output bits are the corresponding projection of the state returned by that query. When this trace is run against the [BDPVA08] simulator* $(\mathcal{S}^{\mathsf{RO_{flat}}}, \mathcal{S}^{-1,\mathsf{RO_{flat}}})$ *in an execution whose combined primitive trace has cost less than* 2^c*, the construction outputs are the flat-RO values* $\mathsf{RF}(X) = \mathsf{RO_{flat}}(\mathsf{flat}(X))$ *at the same well-formed* TW128 *transcript prefixes* X*.*

Proof. A TW128 duplex call absorbs a single suffixed input block, padded by $\mathsf{pad10}^*1$, applies the permutation once, and returns at most r rate bits from the resulting state. Thus the public-permutation implementation of any root or leaf construction computation is already a primitive trace with one forward query per duplex call. The definition of N in Section 4.3 counts exactly these root initialization, associated data, message, leaf initialization, leaf message, and aggregation duplex calls, including the deterministic challenger checks of the tag hash and committing games.

Lemma 5 is the [BDPVA11, Lemma 3] duplex-to-sponge identity specialized to TW128: the output of each duplex call is the output of the sponge after absorbing the accumulated flattened duplex-call transcript, and no additional permutation call is made after the current absorption block when the requested output length is at most r. The bridge of Lemma 8, fixed in Section B.1, maps these $\mathsf{pad10}^*1$ transcript prefixes to the [BDPVA08] padded-sponge input domain without changing the output bits at $r = r_{\max}$.

When the same operational trace is run against $(\mathcal{S}^{\mathsf{RO_{flat}}}, \mathcal{S}^{-1,\mathsf{RO_{flat}}})$, every construction-internal forward query lies on a rooted, paddable path determined by a well-formed TW128 prefix. In an execution whose combined primitive trace has cost less than 2^c, the simulator cannot have saturated: each primitive query adds at most one capacity value to the simulator's internal $\mathcal{R} \cup \mathcal{O}$ set, so fewer than 2^c queries cannot fill the 2^c-element capacity space. The simulator's sponge-consistency invariant [BDPVA08, Lemma 2] therefore makes the rate part returned at that node agree with the corresponding lazy value $\mathsf{RO_{flat}}(\mathsf{flat}(X))$.

Thus the simulator-side operational construction has the same construction-output law as the flat-RO construction interface used in Section 4.2. □

Lemma 20 (Flat Sponge Lift Bound)**.** *Let* G *be a single-stage* $\mathsf{TW128}$ *game (Definition 6). Let* $\mathcal{A}$ *be a public permutation* G *adversary with construction interface cost at most* N *and at most* N_{prim} *direct primitive-interface queries. In the [BDPVA08] single-stage replacement, implement the* $\mathsf{TW128}$ *construction computations by the operational duplex trace of Lemma 19 and keep the adversary's direct primitive-interface queries as primitive queries. This primitive transcript has [BDPVA08] query cost at most*

$$N_{\mathsf{tot}} = N + N_{\mathsf{prim}}.$$

Consequently replacing the real public permutation and $\mathsf{TW128}$ *construction by the simulator-world primitive interface* $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ *and the corresponding flat-RO construction interface costs at most* $\mathsf{Lift}_c(N_{\mathsf{tot}})$.

Proof. The [BDPVA08] replacement is applied before the derived-adversary construction. View the entire single-stage game, including its construction interface and deterministic challenger checks, as a primitive-only distinguisher that runs the $\mathsf{TW128}$ algorithms internally. By Lemma 19, the construction side contributes at most N forward primitive queries: one per duplex call. This includes plaintext-computing computations, verification computations whether they accept or reject, the per-user sum of construction calls in mu-ind, the deterministic challenger decryption or encryption checks charged in the committing games, and the tag hash games' challenger encryptions—the collision game's s submitted-input encryptions, the standard preimage game's source encryption and post-output check, and the everywhere-preimage game's post-output check.

Each direct adversary query to (π, π^{-1}) contributes one primitive-interface query and is counted in N_{prim}. These two counts are disjoint in the public permutation experiment: N counts construction-internal use of the primitive, while N_{prim} counts only the adversary's public primitive interface queries.

Construction work is modeled by the operational duplex trace, rather than by a collection of standalone [BDPVA08] construction queries at every flattened duplex prefix. The standalone-query model would assign a nested-prefix transcript the sum of the individual sponge-evaluation costs, while the operational trace is the same stateful computation the construction actually performs, with the output of each call read from the state just returned by that call. Hence the execution-uniform caps bound the primitive-only query cost by $N + N_{\mathsf{prim}} = N_{\mathsf{tot}}$. If $N_{\mathsf{tot}} \geq 2^c$ then $\mathsf{Lift}_c(N_{\mathsf{tot}}) \geq 1$ and the claimed bound holds trivially, so assume $N_{\mathsf{tot}} < 2^c$, satisfying the hypothesis of [BDPVA08, Theorem 2]. Applying that theorem to this primitive-only distinguisher gives the real-to-simulator replacement, and Lemma 19 identifies the simulator-side construction outputs with the flat-RO construction interface in this nontrivial case. The theorem gives distinguishing or success-probability cost at most the random-permutation differentiating bound $f_P(N_{\mathsf{tot}})$ of [BDPVA08, Lemma 5]. The [BDPVA08, Lemmas 4 and 5] product bounds give the random-transformation bound $f_T(N_{\mathsf{tot}}) = 1 - \prod_{i=1}^{N_{\mathsf{tot}}}(1 - i/2^c)$, and the factor of $f_P(N_{\mathsf{tot}})$ corresponding to the f_T factor at index i is that factor divided by a quantity $1 - (i-1)/2^{r+c} \in (0, 1]$, hence at least as large, so $f_P(N_{\mathsf{tot}}) \leq f_T(N_{\mathsf{tot}})$. With $N_{\mathsf{tot}} < 2^c$ every factor lies in $[0, 1]$ and

$$f_P(N_{\mathsf{tot}}) \leq f_T(N_{\mathsf{tot}}) = 1 - \prod_{i=1}^{N_{\mathsf{tot}}}\Bigl(1 - \frac{i}{2^c}\Bigr) \leq \sum_{i=1}^{N_{\mathsf{tot}}} \frac{i}{2^c} = \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{c+1}} = \mathsf{Lift}_c(N_{\mathsf{tot}}),$$

the second inequality by the Weierstrass product inequality $\prod_i(1 - x_i) \geq 1 - \sum_i x_i$ for $x_i \in [0, 1]$. □

Lemma 21 (Simulator–Permutation Proximity)**.** *Let $\mathcal{D}$ be a distinguisher with access only to a two-directional primitive interface, making at most q queries. Then*

$$\left|\Pr[\mathcal{D}^{\mathcal{S}^{\mathsf{RO_{flat}}},\, \mathcal{S}^{-1,\mathsf{RO_{flat}}}} \Rightarrow 1] - \Pr[\mathcal{D}^{\pi,\, \pi^{-1}} \Rightarrow 1]\right| \;\le\; \mathsf{Lift}_c(q),$$

where $\pi \leftarrow\$\ \mathsf{Perm}(\{0,1\}^{1600})$ and $\mathsf{RO_{flat}}$ is the lazily sampled random oracle internal to the simulator.

Proof. Consider the indifferentiability distinguisher $\mathcal{D}'$ that runs $\mathcal{D}$, forwards its primitive-interface queries, and never queries the construction interface. The [BDPVA08] query cost of $\mathcal{D}'$ is at most q: each forwarded query costs one, and no construction queries arise. Because $\mathcal{D}'$ ignores the construction interface, its two views are exactly $\mathcal{D}^{\pi,\pi^{-1}}$ in the real pair and $\mathcal{D}^{\mathcal{S}^{\mathsf{RO_{flat}}},\mathcal{S}^{-1,\mathsf{RO_{flat}}}}$ in the ideal pair of [BDPVA08, Theorem 2], applied under the [BDPVA11, Theorem 3] bridge fixed in Section B.1. If $q \ge 2^c$ then $\mathsf{Lift}_c(q) \ge 1$ and the claim is trivial; otherwise that theorem bounds the displayed difference by the random-permutation differentiating bound $f_P(q)$ of [BDPVA08, Lemma 5], and the derivation in the proof of Lemma 20 gives $f_P(q) \le \mathsf{Lift}_c(q)$. ◻

Lemma 22 (Derived-Adversary Execution Contract)**.** *Let G be a single-stage TW128 game (Definition 6). Let $\mathcal{A}$ be a simulator-world G adversary whose public primitive interface is answered by $(\mathcal{S}^{\mathsf{RO_{flat}}}, \mathcal{S}^{-1,\mathsf{RO_{flat}}})$, with at most N_{prim} direct primitive-interface queries. Define $\mathcal{B}_{\text{derived}}(\mathcal{A})$ to run $\mathcal{A}$ internally, answer each primitive-interface query by running the same simulator against direct finite-output queries to its own $\mathsf{RO_{flat}}$ oracle, forward construction interface queries unchanged in games with construction oracles, and forward the adversary's final structured output unchanged in the output-checked $\mathsf{H_{TW128}}$ tag hash and committing games. Then the simulator-world execution of $\mathcal{A}$ and the random oracle execution of $\mathcal{B}_{\text{derived}}(\mathcal{A})$ have identical joint distributions, and*

$$q_{\mathsf{RO}}(\mathcal{B}_{\text{derived}}) \le N_{\mathsf{prim}}.$$

In this coupling, simulator-induced $\mathsf{RO_{flat}}$ calls and construction-internal RF lookups use the same lazy table. Therefore, if a simulator-induced query is the bridged input $\mathsf{flat}(X)$ of a well-formed TW128 transcript prefix X and a forwarded construction computation later reaches the same prefix X, both interfaces read the same $\mathsf{RO_{flat}}(\mathsf{flat}(X))$ value, projected to the requested output length. Moreover, $\mathcal{B}_{\text{derived}}$ inherits the construction-side caps of $\mathcal{A}$:

$$\begin{aligned} q_{\mathsf{v}}(\mathcal{B}_{\text{derived}}) &= q_{\mathsf{v}}(\mathcal{A}) \quad \textit{(AE, mu-ind, INT-RUP)},\\ n_{\mathsf{leaf}}(\mathcal{B}_{\text{derived}}) &\le n_{\mathsf{leaf}}(\mathcal{A}) \quad \textit{(all games above)}. \end{aligned}$$

In the mu-ind game these resource inheritances hold per user, and the New-query transcript is unchanged.

Proof. Couple the two executions by using the same lazy $\mathsf{RO_{flat}}$ table for the simulator and for all construction-internal $\mathsf{RF}(\cdot)$ lookups. In the simulator-world execution, $\mathcal{A}$ sees construction answers from the selected game interface and public primitive answers from $(\mathcal{S}^{\mathsf{RO_{flat}}}, \mathcal{S}^{-1,\mathsf{RO_{flat}}})$. In the derived random oracle execution, $\mathcal{B}_{\text{derived}}$ runs the same $\mathcal{A}$, forwards construction interface queries or the final structured output required by the selected game unchanged, and answers primitive-interface queries by invoking the same simulator code with the same lazy $\mathsf{RO_{flat}}$ table. The construction interface, tag hash challenger, or committing challenger therefore receives the same forwarded queries or final output, the simulator receives the same primitive queries in the same order, and the lazy table supplies the same values. The full views of $\mathcal{A}$ in the simulator-world execution and of $\mathcal{B}_{\text{derived}}$ in the random oracle execution are identically distributed.

The simulator's random oracle calls may be on arbitrary [BDPVA08] H-input strings, not only bridged TW128 transcript inputs. These calls are the direct $\mathsf{RO_{flat}}$ queries of

$\mathcal{B}_{\text{derived}}$ and are counted in $q_{\mathsf{RO}}(\mathcal{B}_{\text{derived}})$. Construction-internal $\mathsf{RF}(\cdot)$ lookups made by $\mathsf{Enc}_K^{\mathsf{RO}}$, $\mathsf{Dec}_K^{\mathsf{RO}}$, $\mathsf{PDec}_K^{\mathsf{RO}}$, $\mathsf{Ver}_K^{\mathsf{RO}}$, or a committing challenger are forwarded to the same $\mathsf{RO}_{\mathsf{flat}}$ table but are not counted in q_{RO}, exactly as in Section 4.3.

By Lemma 18, each primitive-interface query induces at most one direct $\mathsf{RO}_{\mathsf{flat}}$ query by $\mathcal{B}_{\text{derived}}$, and inverse queries or non-RO-touching forward queries induce none. Since $\mathcal{A}$ makes at most N_{prim} direct primitive-interface queries, this gives $q_{\mathsf{RO}}(\mathcal{B}_{\text{derived}}) \le N_{\mathsf{prim}}$.

Lemma 8's bridge places TW128 construction restrictions and simulator-induced $\mathsf{RO}_{\mathsf{flat}}$ calls in a single lazy table. If the simulator invokes $\mathsf{RO}_{\mathsf{flat}}$ on $\mathsf{flat}(X)$ for a well-formed TW128 transcript prefix X, and a forwarded construction computation later reaches X, the construction lookup $\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X))$ reads the same table entry and returns the requested projection. Since TW128 uses $r = r_{\max}$, the [BDPVA11, Theorem 3] output post-processing does not alter TW128 output bits. Thus simulator-induced direct queries and construction-internal lookups at the same bridged input are transcript-consistent. In the AE-style games, a simulator call whose input is accepted by KeyedTWOut for the hidden key is still a direct $\mathsf{RO}_{\mathsf{flat}}$ query and is covered by the same $\mathsf{bad}_{\mathsf{key}}$ accounting used by the direct random oracle mu-ind proof. The derivation step itself does not prove a key-guessing statement; it only supplies the derived adversary and the query bound at which the random oracle bounds of Sections 5 and 6 are instantiated.

Finally, the execution-uniform caps of Section 4.3 hold along every oracle-answer trace of $\mathcal{A}$, including traces generated by the simulator on the ideal public primitive interface. The derived-adversary construction does not add construction interface encryption, plaintext-computing, or verification queries and forwards $\mathcal{A}$'s construction interface queries unchanged along each trace. $\mathcal{B}_{\text{derived}}$ therefore inherits the relevant construction resources: $q_{\mathsf{v}}(\mathcal{B}_{\text{derived}}) = q_{\mathsf{v}}(\mathcal{A})$ in the AE, mu-ind, and INT-RUP games, and $n_{\mathsf{leaf}}(\mathcal{B}_{\text{derived}}) \le n_{\mathsf{leaf}}(\mathcal{A})$ for the distinct construction-generated final-leaf-transcript cap. In mu-ind, construction interface forwarding preserves the New calls and these caps user by user. □

B.2 Lift Application

The lift bound of Lemma 20, the simulator–permutation proximity bound of Lemma 21, and the derived-adversary construction of Lemma 22 combine into a single template, stated once and then instantiated by the tag hash bounds of Section 5, the committing consequences of Corollary 5, and the baseline AEAD bounds of Section 6. Every public-permutation bound is obtained by adding $\mathsf{Lift}_c(N_{\mathsf{tot}})$—plus $\mathsf{Lift}_c(N_{\mathsf{prim}})$ in the real-vs-ideal games—and substituting $q_{\mathsf{RO}} \le N_{\mathsf{prim}}$ in the corresponding random oracle theorem.

Corollary 9 (Lift Application)**.** *Let* G *be a single-stage TW128 game (Definition 6). For every public permutation* G*-adversary* $\mathcal{A}$ *against TW128 with the resources of Section 4.3, the derived adversary* $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ *of Lemma 22 is a random oracle* G*-adversary with* $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$ *and the construction-side caps of Lemma 22—in particular* $q_{\mathsf{v}}(\mathcal{B}) \le Q_{\mathsf{v}}$ *and* $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$*, where these apply—such that, when* G *is a win-event game,*

$$\mathbf{Adv}^{\mathsf{G}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B}),$$

and, when G *is a real-vs-ideal game (all-in-one AE or mu-ind),*

$$\mathbf{Adv}^{\mathsf{G}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B}) + \mathsf{Lift}_c(N_{\mathsf{prim}}).$$

If moreover $\mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B}) \le \varepsilon_{\mathsf{G}}(q_{\mathsf{RO}}(\mathcal{B}), n_{\mathsf{leaf}}(\mathcal{B}))$ *for some* ε_{G} *nondecreasing in each argument, then* $\varepsilon_{\mathsf{G}}(N_{\mathsf{prim}}, N_{\mathsf{leaf}})$ *may replace* $\mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B})$ *in the corresponding bound.*

Proof. By the flat sponge lift bound (Lemma 20), replacing the real public permutation and TW128 construction by $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ and the flat-RO construction interface costs at most $\mathsf{Lift}_c(N_{\mathsf{tot}})$, where $N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$ counts both the construction calls of N—including any construction oracle exposed during $\mathcal{A}$'s run and, in the win-event games, the challenger's construction computations, pre- and post-output (in the standard preimage game, the source encryption computing T^* as well as the post-output check)—and the N_{prim} direct primitive queries. The derived-adversary contract (Lemma 22) expresses the resulting simulator-world execution as the identically distributed random oracle execution of $\mathcal{B} = \mathcal{B}_{\mathrm{derived}}(\mathcal{A})$, with $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$ and the stated construction-side inheritances. For a win-event game this gives the first bound.

For a real-vs-ideal game, chain four worlds: W_1, the real construction interface with (π, π^{-1}); W_2, the flat-RO real construction interface with the simulator; W_3, the ideal construction interface with the simulator; and W_4, the ideal construction interface with (π, π^{-1}). The real-vs-ideal games of Sections 6.1 and 6.2 compares W_1 with W_4. The step from W_1 to W_2 costs at most $\mathsf{Lift}_c(N_{\mathsf{tot}})$ as above, and under the coupling of Lemma 22 the pair $(\mathsf{W}_2, \mathsf{W}_3)$ is precisely the real/ideal pair of the random oracle game for $\mathcal{B}$, so $|\Pr[\mathcal{A}^{\mathsf{W}_2} \Rightarrow 1] - \Pr[\mathcal{A}^{\mathsf{W}_3} \Rightarrow 1]| \leq \mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B})$. For the step from W_3 to W_4, the ideal construction oracles—$(\mathsf{Rand}, \mathsf{Replay})$, instantiated per user in mu-ind, together with the New interface and the validity checks—are independent of the primitive, so a distinguisher can answer them from its own coins, run $\mathcal{A}$, and forward the at most N_{prim} primitive-interface queries; Lemma 21 bounds this step by $\mathsf{Lift}_c(N_{\mathsf{prim}})$. The triangle inequality combines the three estimates into the second bound.

The final claim follows by the monotonicity of ε_{G}. When G is parameterized by a challenge fixed before $\mathcal{A}$ runs—the everywhere-preimage target tag, or the standard preimage source X^*, whose tag $T^* = \mathsf{H}_{\mathsf{TW128}}(X^*)$ each coupled world computes—the coupling holds for each fixed challenge and the challenge-independent bound holds for the maximum or average over challenges defining $\mathbf{Adv}^{\mathsf{G}}_{\mathsf{TW128}}(\mathcal{A})$. $\square$

C Vectorized Kernel Details

This appendix details the optimized cores summarized in Section 8.1: their shared batching structure, chunk 0 fusion, instruction selection, register layout, remainder handling, and data-independent addressing.

The optimized paths share the same logical decomposition but differ in their batch widths, remainder strategy, and state writeback points:

Path	Cores and writeback
`amd64` AVX-512	Wide: one eight-state ZMM group. Remainder: masked 2–7 state AVX-512 core. Single duplex: AVX-512 permutation. Chunk 0: all active states are written back through `state8`.
`amd64` AVX2	Wide: two four-state YMM groups. Remainder: 2–7 states with dummy inactive lanes. Single duplex: general purpose register permutation. Chunk 0: all active states are written back through `state8`.
`arm64` NEON+SHA3	Wide: four two-state NEON pairs. Remainder: two-state pair core. Single duplex: NEON/SHA-3 permutation. Chunk 0: pair $(0, 1)$ is written back in the wide core, and both states are written back in the pair core.

Shared batching structure. The implementation separates the sequential root duplex from the independent leaf duplexes. The root duplex normally uses a single-duplex ($\times 1$) core, while complete leaf chunks are grouped into eight-instance batches. Every complete leaf chunk is exactly $\beta = 49\rho$ bytes, so all eight instances in a batch follow the same sequence of 49 full ρ-blocks: the first 48 close with MSG_MORE, and the last closes with MSG_LAST. A single public block index therefore drives the whole batch. The batched state is lane-major: for each of the 25 Keccak lanes, the kernel stores the corresponding 64-bit word of each active instance together in vector registers. Equivalently, for Keccak lane i the vector register holds $(S_0[i], \ldots, S_7[i])$ across the active duplex states, so one vector operation updates the same lane in several states.

Chunk 0 fusion. Chunk 0 is part of the root transcript, but its message phase has the same kernel-visible block schedule as a complete leaf chunk. The optimized paths exploit this by placing the post-AD root state into lane 0 of the first batched call and placing the first leaf chunks in the remaining lanes. When the message has at least seven complete leaf chunks, this fused call uses the $\times 8$ core; below that, it uses the same remainder machinery that drains incomplete batches. The output state from lane 0 is written back to the root duplex, which then absorbs the leaf tags produced by the same call.

Remainder and tail handling. Messages are otherwise decomposed into full eight-chunk batches plus a remainder. On `amd64`, a register-resident remainder kernel handles two to seven complete chunks in one call, reading each active chunk in place; on `arm64`, a 2-wide kernel drains the remainder two chunks at a time. A single leftover complete chunk, a partial trailing chunk, and the count-aggregation tail use the single-duplex core. The `amd64` assembly chunk kernels handle the 48 full MSG_MORE blocks and leave the final MSG_LAST block plus leaf tag extraction to shared Go code, keeping byte-granular tail logic out of assembly. The `arm64` batched kernels process the complete chunk and store back the states needed for chunk 0 fusion: pair $(0, 1)$ in the $\times 8$ kernel, and both active states in the 2-wide kernel.

`amd64`. The `amd64` implementation provides AVX-512 and AVX2 cores and selects between them once at startup from an AVX-512 feature probe requiring OS support for YMM/ZMM state, `AVX512F`, and `AVX512VL`. The single-duplex permutation has two variants: a general-purpose-register core adapted from the standard library's KECCAK-p permutation (using the lane-complement trick so that χ requires no separate negation), and an AVX-512 variant; the root duplex uses whichever matches the selected core.

The eight-way AVX-512 core packs the lane-major state into 25 ZMM registers, one per lane with all eight instances in the register's eight 64-bit slots, and performs the permutation with no cross-lane shuffles: rotations use `VPROLQ`, and `VPTERNLOGQ` fuses the θ D-term formation and the χ and-not into single three-input logic instructions. The fused chunk kernel reads the eight chunks with contiguous loads, transposes each rate block into the lane-major layout in registers (`VPUNPCKLQDQ`/`VPUNPCKHQDQ` and `VSHUFI64X2`), absorbs on the lane-major state, and transposes each output block back for a contiguous store. Since $\rho = 167 = 20 \cdot 8 + 7$, the first 20 rate lanes use this transposed path and only the trailing 7 bytes in lane 20 use a masked gather. These sequential accesses are prefetcher-friendly, whereas per-lane gathers and scatters at the chunk stride β (`VPGATHERQQ`/`VPSCATTERQQ`) would stall on cold memory and cap the long-message rate.

The register-resident kernel that drains a two-to-seven-chunk AVX-512 remainder instead uses that per-lane gather/scatter form under a mask, reading the active chunks in place: the transpose's shuffle work is constant regardless of the chunk count and would be spent partly on inactive lanes, whereas the gathers scale with the active element count.

The eight-way AVX2 core, lacking 512-bit registers, runs the eight instances as two groups of four (four 64-bit lanes per YMM register), rotating by shift-or and computing χ with `VPANDN`. The AVX2 path drains remainders with a variant of the grouped-by-four kernels that aims each inactive instance's loads and stores at a dummy block inside the kernel frame with a zero advance stride, so inactive instances touch no memory beyond the remainder and the hot loop stays branch-free.

`arm64`. The `arm64` path uses NEON plus the Armv8.2 SHA-3 extension (`FEAT_SHA3`), available on Apple Silicon and on Neoverse V1/V2 and N2. The permutation is expressed with the extension's fused instructions: `EOR3` (three-input XOR) for the θ column parities, `RAX1` (rotate-by-one and XOR) for the θ D-terms, `XAR` (XOR then rotate) to fuse the θ accumulation with the ρ rotation, and `BCAX` (bit-clear and XOR) for χ. The single-duplex core stores each Keccak lane in one 64-bit `D` lane across the 25 vector registers. The eight-way core packs two instances per 128-bit register and processes the batch of eight as four such pairs, using `ZIP1` and duplicating moves to interleave and extract the two instances of each lane in the fused chunk kernel. The same pair machinery backs a 2-wide kernel that drains a leaf-chunk remainder two chunks at a time—one instance per 64-bit half of a 128-bit register—at roughly twice the single-duplex per-chunk rate. Both batched kernels store the permutation state back into the batch after the closing block—the $\times 8$ kernel for its first pair, the 2-wide kernel for both instances—which is the writeback that permits the root duplex to occupy lane 0.

Data-independent addressing. The optimized kernels contain no secret-dependent branches and no secret-dependent memory or vector indices. The only indexed table is the round-constant array, addressed by the public round number. The AVX-512 steady-state transpose uses contiguous loads and stores; the remaining gathers and scatters, for the partial rate lane and register-resident remainder kernel, use fixed public chunk strides under masks determined by the public chunk count. The AVX2 remainder kernel redirects inactive instances to a dummy block in its stack frame with a zero advance stride, again selected only by the public chunk count. The `arm64` pair kernels use fixed lane interleaving and extraction patterns. None of these addresses depends on the key, nonce, plaintext, or decrypted plaintext.

D Test Vectors

The following selected known-answer test vectors exercise TW128 encryption and are intended for implementation conformance checks against the construction in Section 3. The complete vector set, including full byte strings for the long cases and the regeneration harness, is maintained with the reference implementation at `https://github.com/codahale/treewrap`.

All byte strings are written in hexadecimal. For each vector we list the key K, nonce U, associated data A, message M, ciphertext C, and authentication tag T; the sealed output of encryption is $C \parallel T$. The empty string is written ε. The parameters used below are $\rho = 167$ bytes, $\beta = 8183$ bytes, and 32-byte keys, nonces, and tags. The leaf index is encoded as a little-endian 64-bit integer and the phase suffixes are appended least significant bit first.

To keep the longer vectors compact, any value exceeding 32 bytes is given by its length in bytes together with its SHA-256 digest rather than in full. For generated associated data and message inputs, byte i is defined as $x_i = (17i + 31) \bmod 256$ for $i = 0, 1, \ldots$, so each such input is determined by its length alone and can be reconstructed and checked against the stated digest; a ciphertext given by its length and digest can likewise be checked by encrypting the corresponding message and hashing the result.

Vector 1. Empty associated data and empty message: the associated data and aggregation phases are both elided, and the root tag is emitted by the single closing message call with no leaves.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	ε
C	ε
T	`da65bbda0b20b1e936f5326458166633d1e5a3f7aebc0b318d24b051a4efa697`

Vector 2. Multi-block associated data with an empty message: the associated data spans two rate blocks and no message bytes are processed.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	168 bytes, SHA-256
	`4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a`
M	ε
C	ε
T	`2093878418f3fca96ad3a8d1748a9be4d2a8b518e917d988fd703dcb7bf72173`

Vector 3. Empty associated data and a single-block message: the root message phase only, no leaves.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	`1f30415263748596a7b8c9daebfc0d1e2f405162738495a6b7c8d9eafb0c1d2e`
C	`676a1441e20daea0f1ee4571ee4543842c946d59e0d3df6f1dd2f92dc4f876aa`
T	`1873332d359ff2fe5197488224af7bf6ad821875fdbd964100de424aa77342be`

Vector 4. Empty associated data and a two-block message: the root keystream is chained across two rate blocks.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	168 bytes, SHA-256
	`4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a`
C	168 bytes, SHA-256
	`df3f0a98a91a69ed474edbc52e0dc1d8e54bfd6fc514966ce7b878d48be0bf9a`
T	`aab2413074a202cfadd5eb784663b1831eb330025972d163d03e9ed5a964b765`

Vector 5. A message of exactly one chunk: the root is full, there is still no leaf, and the aggregation phase is elided so the closing root message call emits the tag.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	8183 bytes, SHA-256
	`6532a370a4ab6b5f4094dd8a6016512ab8e8416abe910bc1a0b532979224151d`
C	8183 bytes, SHA-256
	`8d58f8aa3413ea0ba1b8cc3ed4e9a334521d8dafe9694a0661fb8251049907e9`
T	`c2201637f1116c1303b8982622756770d23acb85384f4658fcb63c37e4b629dd`

Vector 6. A message of one chunk and one further byte: one root chunk and one leaf, exercising leaf tag aggregation.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	8184 bytes, SHA-256
	`c4c383bca0808e2ce44d481ddfe56efdcd7a147dcd5a14c898ef5423f2604cd7`
C	8184 bytes, SHA-256
	`ef1b2f7607ade503ab497fc61e62cef2b44c6328fa06e499718eb7da5fdd04d3`
T	`c96bea48e07a8431fd19721ffbe8afe6a73047f96d953e3281f10126eeca06a1`

Vector 7. A multi-leaf message of two chunks and one further byte: the full tree path with two leaves.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	16367 bytes, SHA-256
	`17e31f3700d0602aab8a686024ecc05b8192e80a531483c67dd90d5a26557c5e`
C	16367 bytes, SHA-256
	`6d83d8fbdc2a7c2ef590d29a36527a45019e50d1f8c8db037959081685a46908`
T	`cdc2e50ac5380419f06772a2f3af2146050bd0340b35f61b67f839351767804e`

Vector 8. Associated data spanning multiple blocks with a short message: exercises associated data absorption before message encryption.

	K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
	U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
3113	A	337 bytes, SHA-256
		`04ca7b24930bc6b5fe8b95ff1eb0933907657c1283cc3a7e69c5f7f02220f281`
	M	`1f30415263748596a7b8c9daebfc0d1e`
	C	`0dd615a0a05ed82a78c3999b9e80cbed`
	T	`22e4e6725f548ad0660041d66cf2f18dc45c69b447cdaae1f147d33a3b6aa1a9`

Vector 9. An all-ones key and nonce with a multi-chunk message.

	K	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	U	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	A	168 bytes, SHA-256
3114		`4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a`
	M	8184 bytes, SHA-256
		`c4c383bca0808e2ce44d481ddfe56efdcd7a147dcd5a14c898ef5423f2604cd7`
	C	8184 bytes, SHA-256
		`49818c320ab282296639798d840cbd60751e1623f449dcbf9423371d44145dfb`
	T	`310c9fff20e9651f8ed47eed9bb182a086ed1af1b3f1799e35307a4139865fc7`